

Title Page

**Lab Title: Lab 3 — Data Carving with XXD,
Binwalk and Scalpe**

Student Name: Athanasius Orikeze Alekwe

Student ID: 2025/FWSD/11230.

Course Name: Computer and Digital Forensics

Instructor Name: Aminu Idris

Date of Submission: 04/09/2026

Introduction

This laboratory exercise focused on practical data-carving and file-recovery techniques used when normal file-system information is missing, damaged, or no longer dependable. The work involved examining raw binary data, identifying file signatures, reconstructing files from hexadecimal data, recovering deleted content, and validating recovered evidence. The exercise also required the use of file headers and footers, metadata examination, direct sector-level analysis, embedded-file discovery, and signature-based carving.

The practical work was carried out in a controlled forensic workspace so that the original evidence could be preserved. Evidence files were downloaded into a dedicated laboratory structure, their sizes and cryptographic hashes were recorded, and the original copies were made read-only before analysis. Working copies were then used for the investigative procedures shown throughout the report.

The investigation progressed from JPEG examination with xxd and strings to file-system analysis with The Sleuth Kit, embedded-content discovery with Binwalk, and deleted-file recovery with Scalpel. MD5 and SHA-256 hashing was repeatedly used to check integrity and confirm that reconstructed or recovered files remained consistent with the data from which they were obtained.

Objectives

By completing this lab, students will demonstrate their ability to explain why data carving is required when file-system metadata is missing or unreliable; use xxd to examine binary file content and identify headers and footers; recognise JPEG SOI and EOI signatures; reconstruct binary content from a hexadecimal dump; use The Sleuth Kit to examine inode, block and sector-level data; identify and extract embedded content using Binwalk; recover deleted files from a forensic USB image using Scalpel; verify recovered evidence using cryptographic hashes; and prepare a professional forensic report containing commands, screenshots, findings and limitations.

Laboratory / Tools Used

Tool / Environment	How it was used in the laboratory
Kali Linux in VMware Workstation	Provided the working environment in which the forensic commands and analysis were performed.
wget	Downloaded the authorised evidence files used in the practical.
md5sum / sha256sum	Generated cryptographic hashes for original, reconstructed, extracted, carved, and packaged evidence.
file	Identified file types after downloading, reconstruction, extraction, and recovery.
xxd	Examined binary content, identified JPEG signatures, created a hexadecimal dump, and reconstructed the JPEG.
strings / grep	Located readable metadata-related strings, dates, software details, camera information, and other text inside the JPEG.
cmp	Compared the reconstructed JPEG with the working copy to confirm byte-for-byte equality.
The Sleuth Kit	img_stat, mmls, fsstat, fls, istat, icat, and blkcat were used to examine disk images, file systems, metadata entries, sectors, and deleted files.
7z	Inspected and extracted the 120M.7z training archive.
Binwalk	Detected embedded signatures and structures and performed automatic extraction attempts.
dd	Manually extracted the identified Office Open XML structure using offsets obtained from Binwalk.
unzip	Tested and listed the contents of the manually recovered OOXML/ZIP structure.
Scalpel	Carved EXIF- and JFIF-based JPEG files from the USB forensic image.
Kali Image Viewer	Opened recovered JPEG files to confirm that usable images had been successfully recovered.
zip / unzip -t	Created and validated the final supporting-evidence archive.

Key Findings

The hexadecimal examination of J_ub_law.jpg confirmed a standard JPEG start signature of FF D8, followed by an FF E1 APP1 marker containing EXIF information. The reconstructed copy produced the same MD5 and SHA-256 hashes as the source working copy, and cmp returned exit status 0, confirming that the reconstruction was byte-for-byte identical.

Metadata-style examination identified NIKON CORPORATION, NIKON D4, Adobe Photoshop 21.1 (Macintosh), the name-related string HOWARD KORN, the lens-related value 17.0-35.0 mm f/2.8, and date/time values including 2020:08:20 10:20:05 and 2013:10:08 18:56:21. Adobe/XMP information and an embedded camera-profile reference were also present.

The Sleuth Kit examination of Ch01InChap01.dd identified a FAT12 filesystem. A deleted Billing Letter.doc entry at metadata address 8 occupied sectors 237–283 and had a size of 24,064 bytes. Recovery with icat and direct extraction of the same 47 sectors with blkcat produced identical results, while sector 237 contained the Microsoft Compound File signature D0 CF 11 E0 A1 B1 1A E1.

The instructor-referenced File_carving.docx could not be located, including inside 120M.7z, so the permitted alternative Binwalk workflow was performed on the other supplied evidence. The USB evidence was confirmed as a raw 124,780,544-byte image with a FAT16 partition beginning at sector 128. Binwalk subsequently helped identify an Office Open XML structure at decimal offset 421888, which was manually extracted and successfully validated with unzip.

Finally, Scalpel was configured for both EXIF- and JFIF-based JPEG signatures and recovered 17 JPEG artefacts from the USB forensic image. Hashes were generated for all recovered JPEGs, and several—including 00000009.jpg, 00000010.jpg, and 00000011.jpg—were successfully opened in the Kali image viewer.

Practical Evidence

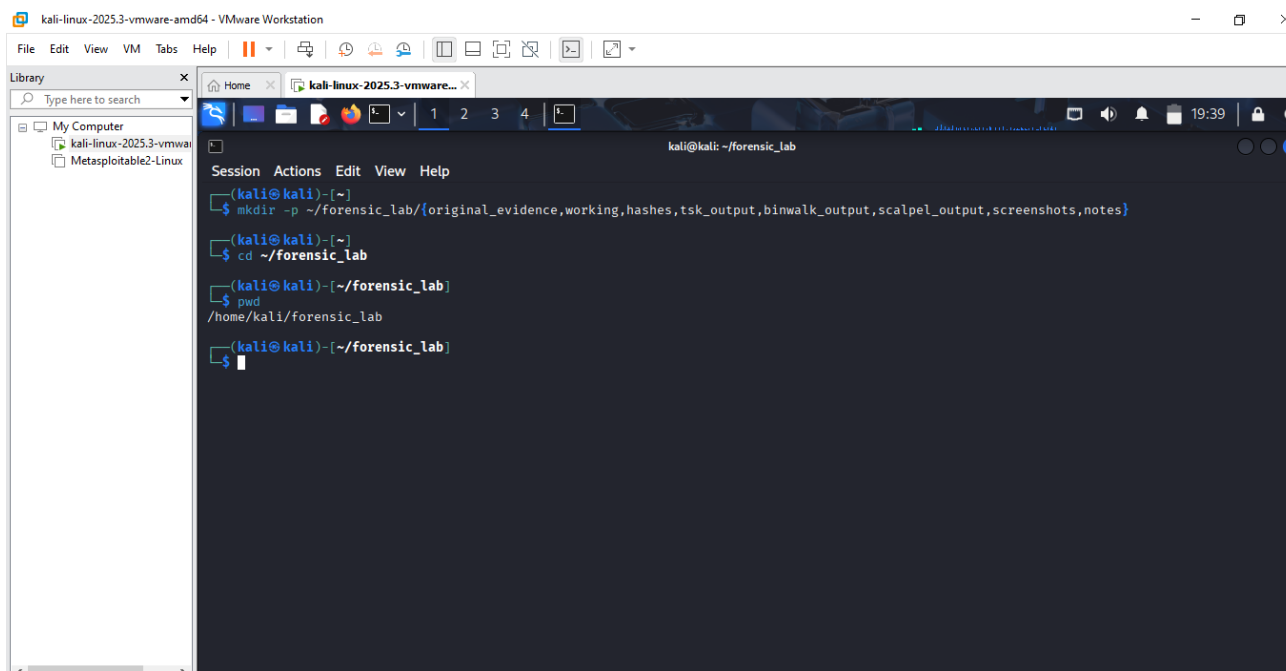


Fig 1.0 Creating the Lab folder.

Explanation

Here I created the lab folder and all its contents that are needed for this lab.

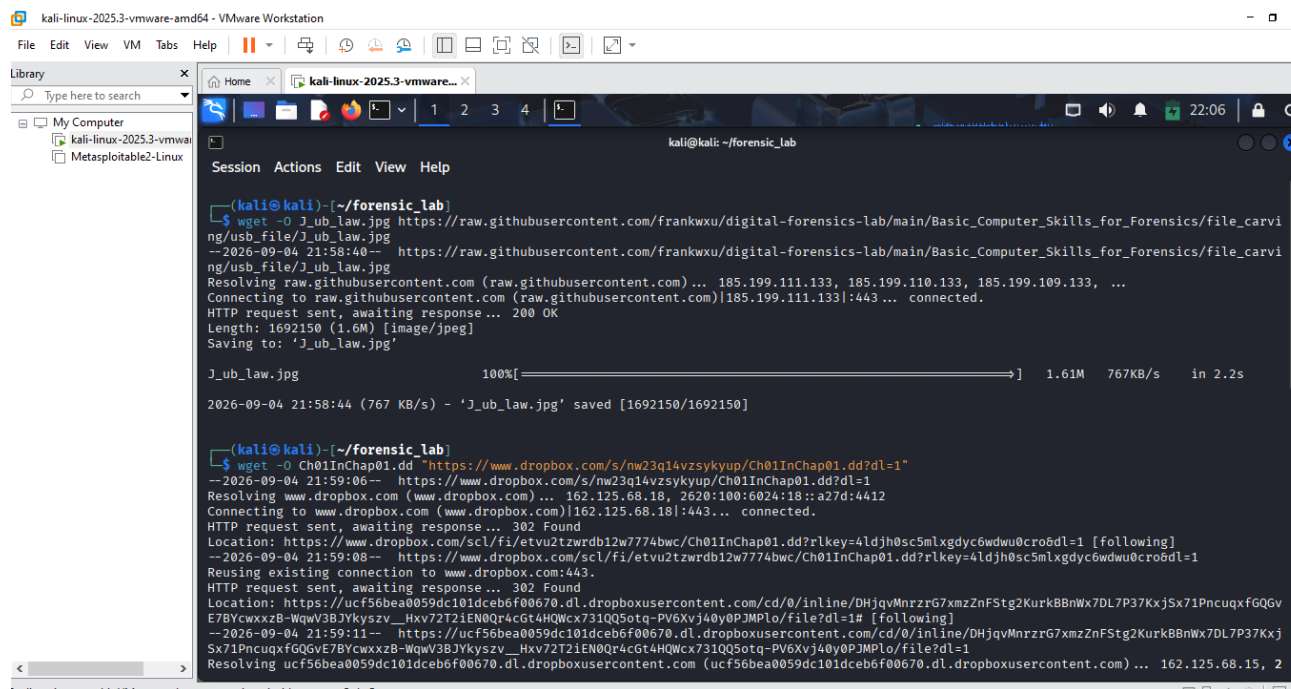


Fig 1.1 Download the authorized training evidence files.

Explanation

This screenshot records the beginning of the evidence-acquisition stage, where the authorised laboratory files were downloaded into the working environment. It provides a visible record that the practical used the evidence specified for the exercise.

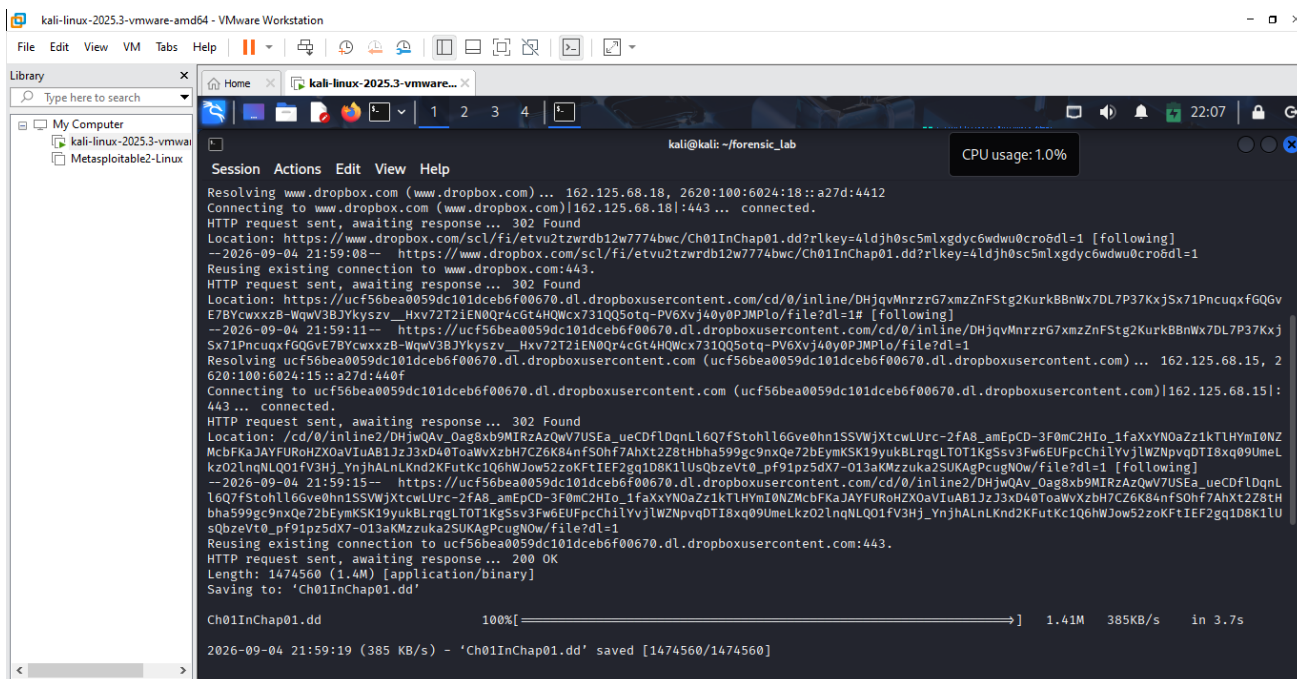


Fig 1.2 Download the authorized training evidence file page 1.

Explanation

This continuation shows the download process for Ch01InChap01.dd. The progress information demonstrates that the disk image was obtained before any forensic examination was started.

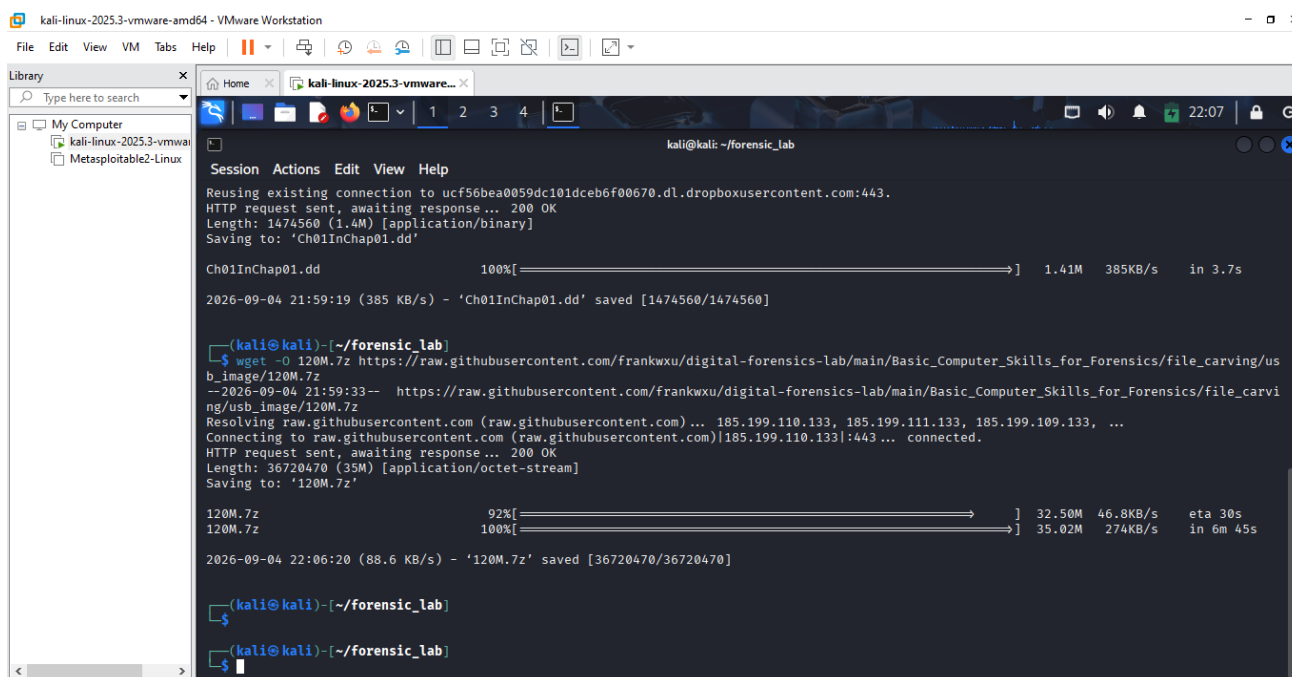
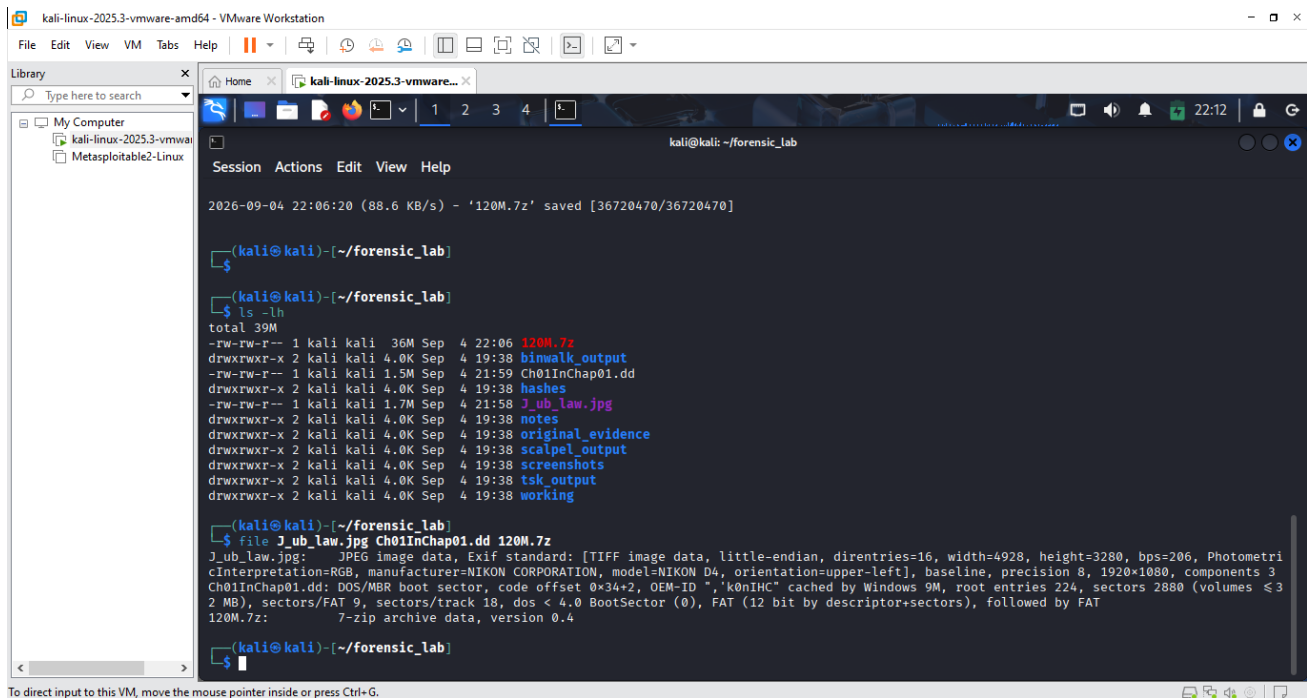


Fig 1.3 Download the authorized training evidence file page 2.

Explanation

This screenshot continues the evidence download process and shows the acquisition of the 120M.7z archive. It completes the documented acquisition of the main evidence resources used later in the laboratory.



To direct input to this VM, move the mouse pointer inside or press Ctrl+G.

Fig 1.4 Checking that all three downloaded correctly.

Explanation

The downloaded files were listed and examined with the file command to confirm that they were recognized in their expected formats. This check helped ensure that the downloads were genuine evidence files rather than incomplete or incorrect downloads.

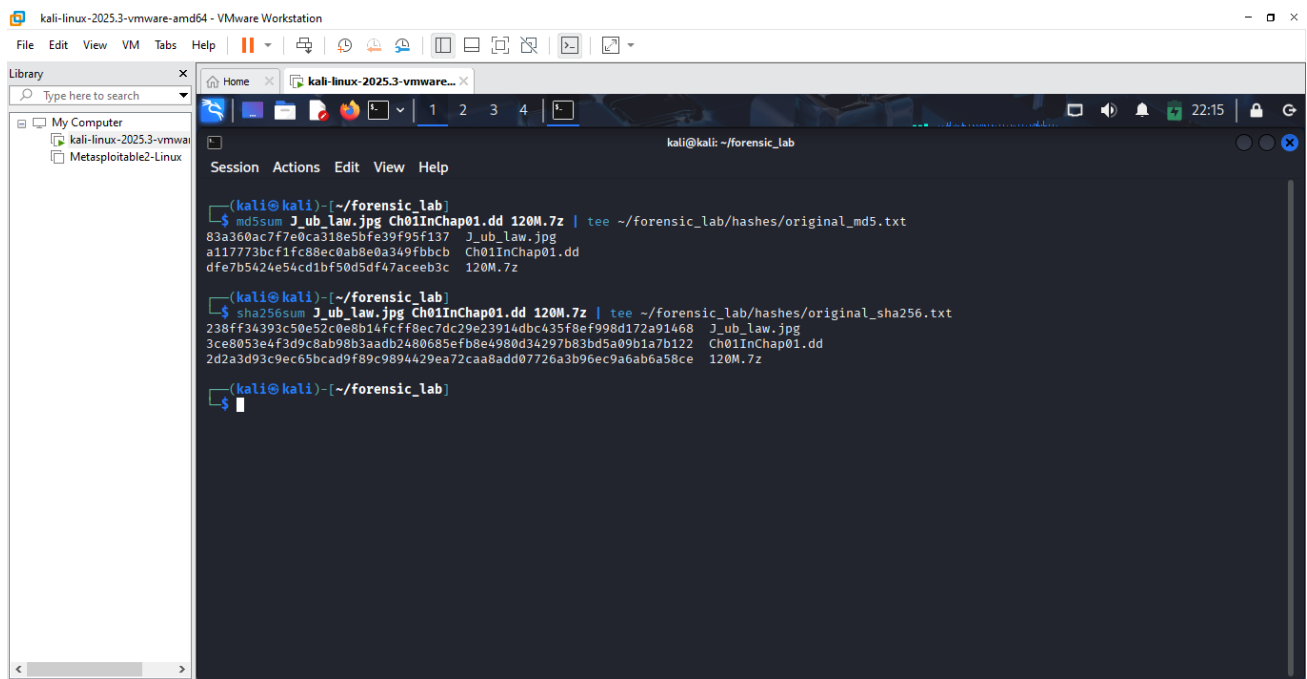
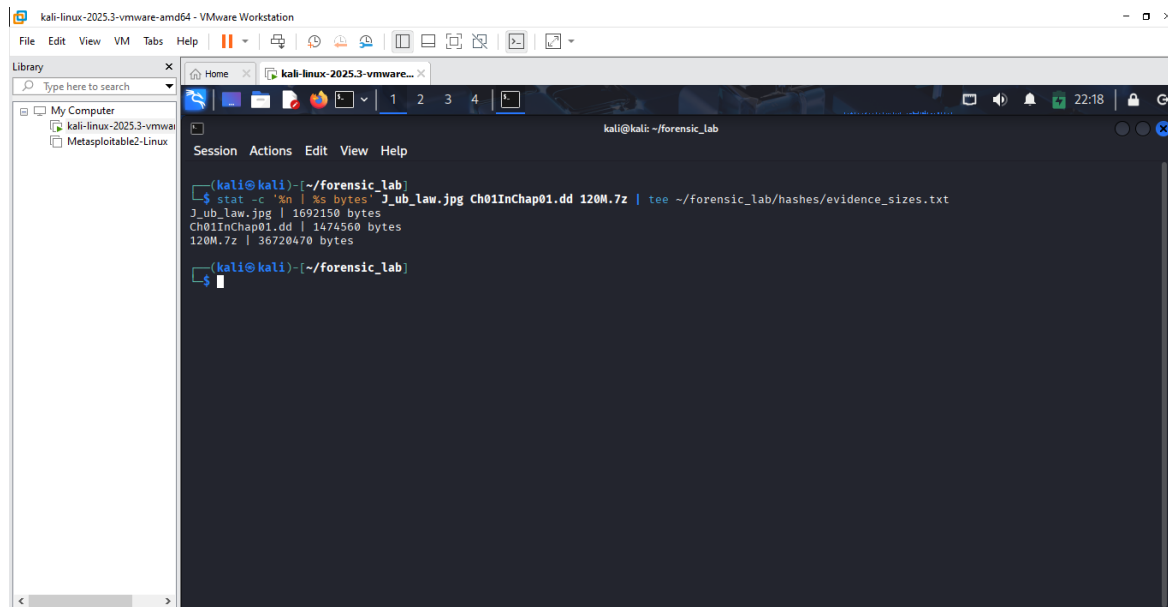


Fig 1.5 calculating the original hashes for md5sum and sha256sum.

Explanation

This screenshot records the first cryptographic integrity checks performed on the evidence. MD5 and SHA-256 values were calculated so that later copies or extracted data could be compared against known values.

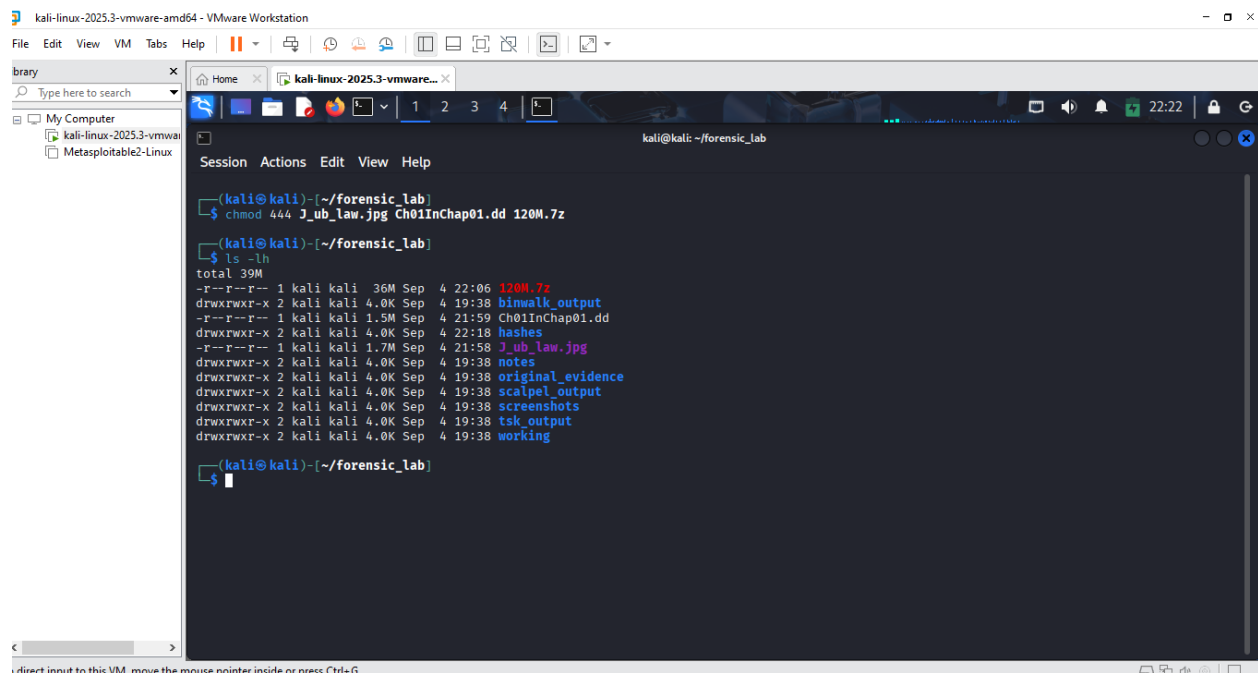


```
kali@kali: ~/forensic_lab
$ stat -c '%n | %s bytes' J_ub_law.jpg Ch01InChap01.dd 120M.7z | tee ~/forensic_lab/hashes/evidence_sizes.txt
J_ub_law.jpg | 1692150 bytes
Ch01InChap01.dd | 1474560 bytes
120M.7z | 36720470 bytes
```

Fig 1.5.1 Recording the exact size for the files.

Explanation

The exact byte size of each evidence file was recorded at this stage. This added another basic integrity and identification reference alongside the cryptographic hashes.



```
kali@kali: ~/forensic_lab
$ chmod 444 J_ub_law.jpg Ch01InChap01.dd 120M.7z
$ ls -lh
total 39M
-r--r--r-- 1 kali kali 36M Sep 4 22:06 120M.7z
drwxrwxr-x 2 kali kali 4.0K Sep 4 19:38 binwalk_output
-r--r--r-- 1 kali kali 1.5M Sep 4 21:59 Ch01InChap01.dd
drwxrwxr-x 2 kali kali 4.0K Sep 4 22:18 hashes
-r--r--r-- 1 kali kali 1.7M Sep 4 21:58 J_ub_law.jpg
drwxrwxr-x 2 kali kali 4.0K Sep 4 19:38 notes
drwxrwxr-x 2 kali kali 4.0K Sep 4 19:38 original_evidence
drwxrwxr-x 2 kali kali 4.0K Sep 4 19:38 scalpel_output
drwxrwxr-x 2 kali kali 4.0K Sep 4 19:38 screenshots
drwxrwxr-x 2 kali kali 4.0K Sep 4 19:38 tsk_output
drwxrwxr-x 2 kali kali 4.0K Sep 4 19:38 working
```

Fig 1.6 Making the originals read-only.

Explanation

The evidence files were given read-only permissions before analysis. This reduced the risk of accidental modification and helped preserve the originals while subsequent work was carried out on separate copies.

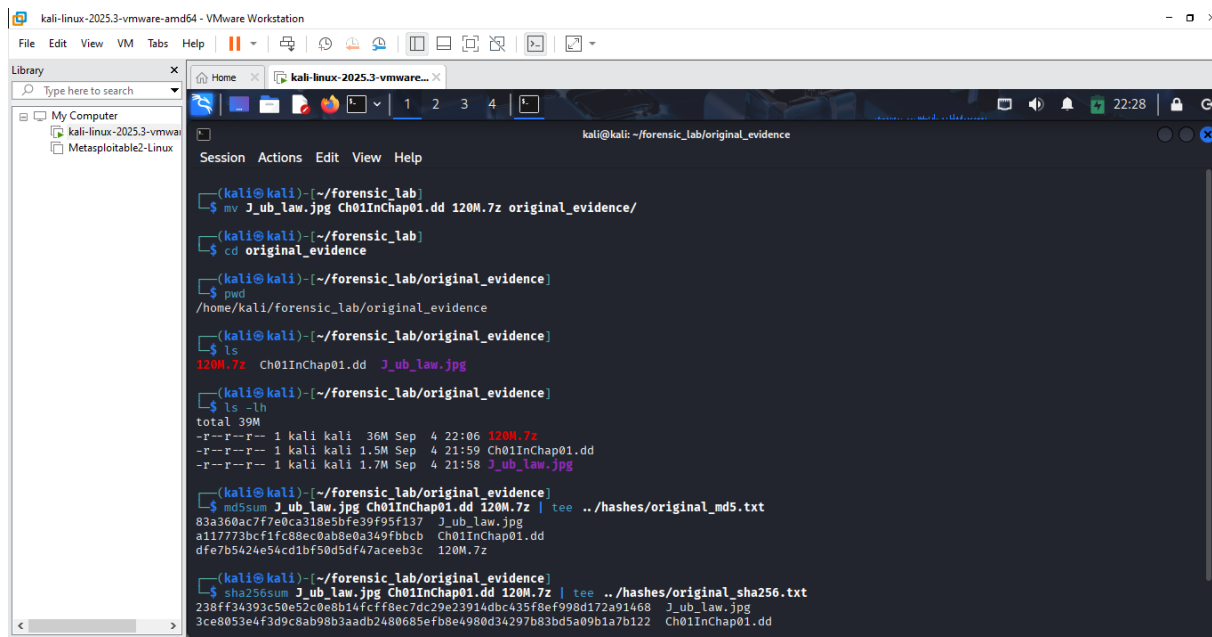


Fig 1.7 Moving the evidence into original_evidence.

Explanation

This screenshot shows the authorised files being placed in the dedicated original_evidence directory. Separating original evidence from working material helped maintain a clear distinction between source data and files created during analysis.

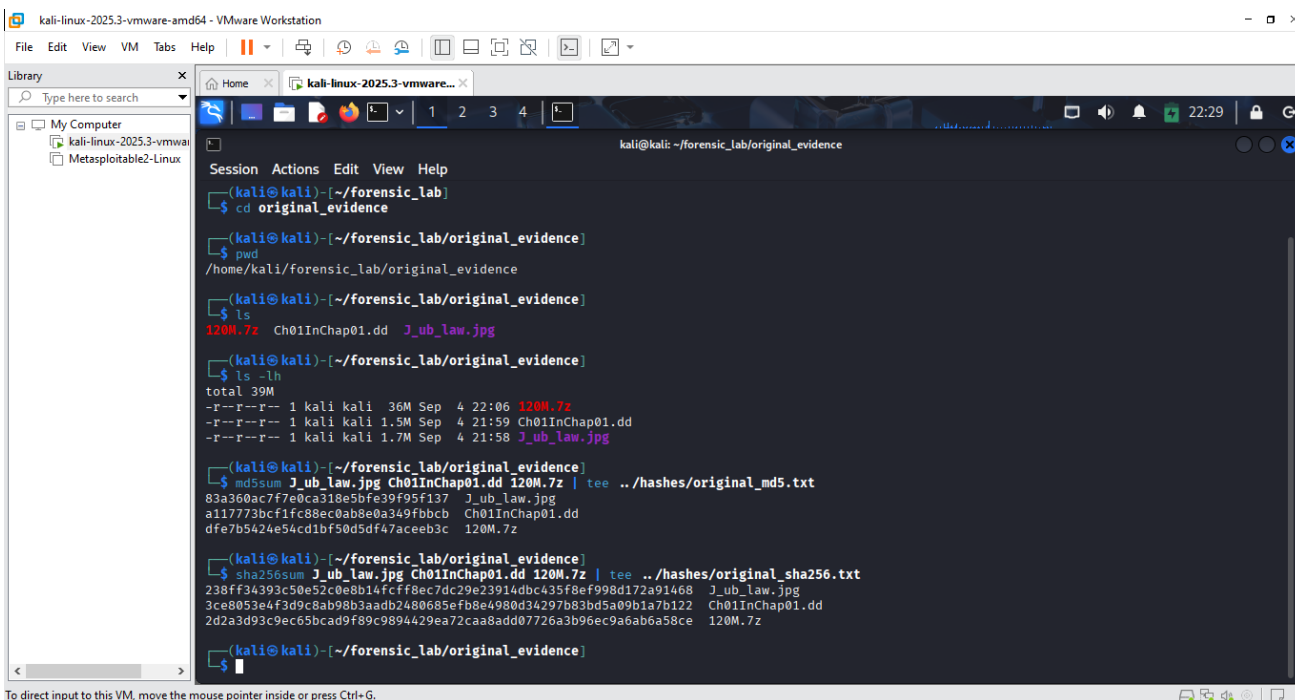
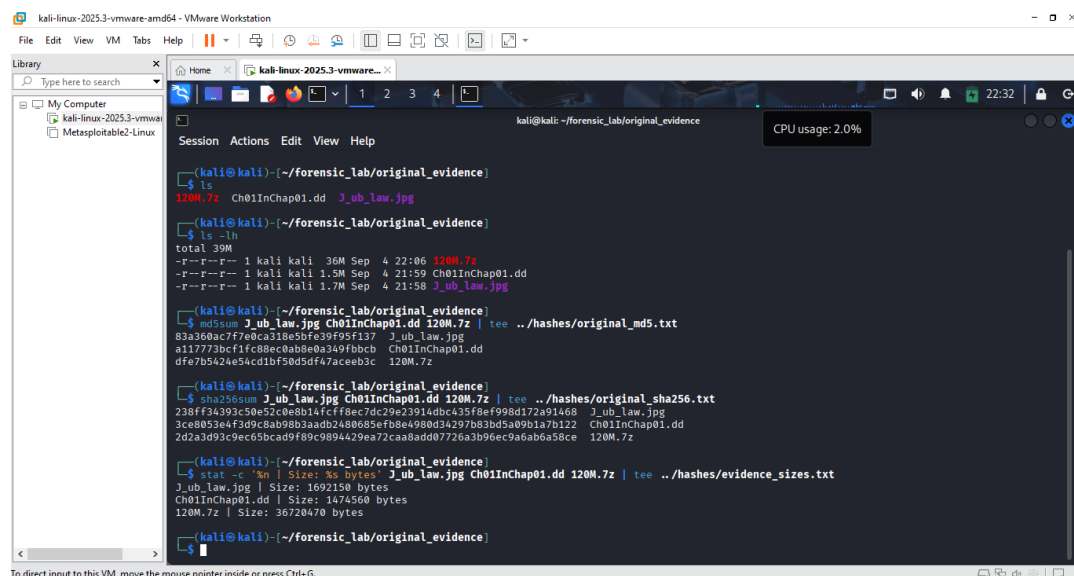


Fig 1.8 Recording the MD5 and SHA-256 hashes.

Explanation

After organizing the files inside the original-evidence directory, their cryptographic hashes were recorded again and saved into dedicated hash files. This created a permanent integrity record for the evidence used throughout the lab.



```
kali@kali:~/forensic_lab/original_evidence
$ ls
120M.7z  Ch01InChap01.dd  J_ub_law.jpg

(kali@kali:~/forensic_lab/original_evidence)
$ ls -lh
total 39M
-r--r--r-- 1 kali kali 36M Sep  4 22:06 120M.7z
-r--r--r-- 1 kali kali 1.5M Sep  4 21:59 Ch01InChap01.dd
-r--r--r-- 1 kali kali 1.7M Sep  4 21:58 J_ub_law.jpg

(kali@kali:~/forensic_lab/original_evidence)
$ md5sum J_ub_law.jpg Ch01InChap01.dd 120M.7z | tee ../hashes/original_md5.txt
83a360ac7f7e0ca318e5b9f95f137  J_ub_law.jpg
a11773bcf1fc88ec0ab8e0a349fbbcb  Ch01InChap01.dd
dfe7b542e54cd1bf50d5df47aceeb3c  120M.7z

(kali@kali:~/forensic_lab/original_evidence)
$ sha256sum J_ub_law.jpg Ch01InChap01.dd 120M.7z | tee ../hashes/original_sha256.txt
238ff34393c50e52c0e0b14fcff8ec7dc29e23914dbc435f8ef998d172a91468  J_ub_law.jpg
3ce8053e4f3d9c8ab98b3aad02480685efb8e4980d34297b83bd5a09b1a7b122  Ch01InChap01.dd
2d2a3d93c9ec65bcad9f89c9894429ea72caa8add07726a3b96ec9a6a6a58ce  120M.7z

(kali@kali:~/forensic_lab/original_evidence)
$ stat -c '%n | Size: %s bytes' J_ub_law.jpg Ch01InChap01.dd 120M.7z | tee ../hashes/evidence_sizes.txt
J_ub_law.jpg | Size: 1692150 bytes
Ch01InChap01.dd | Size: 1474560 bytes
120M.7z | Size: 36720470 bytes

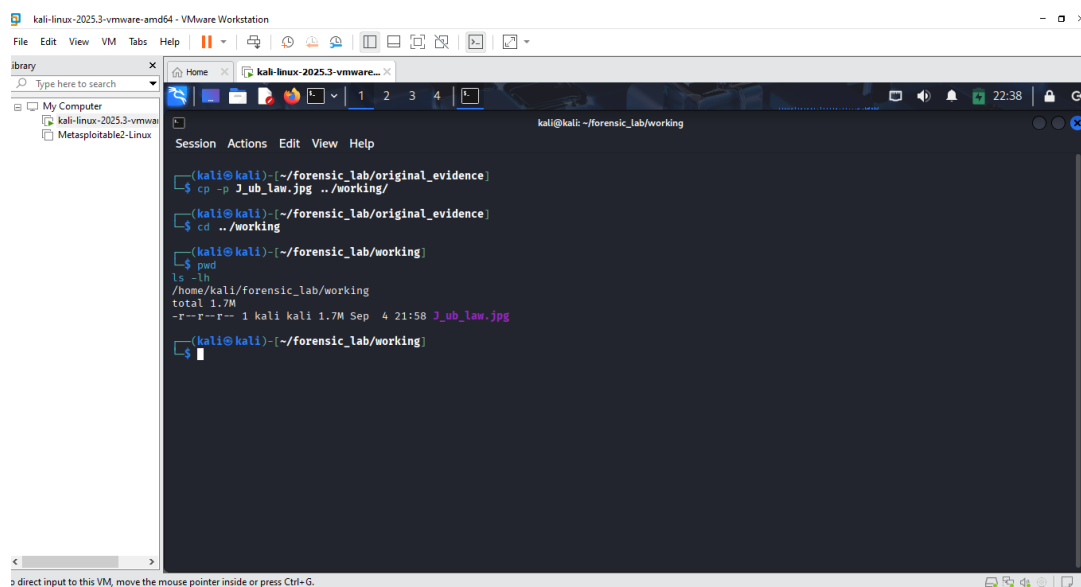
(kali@kali:~/forensic_lab/original_evidence)
$
```

Fig 1.9 Recording the exact file sizes.

Explanation

This screenshot records the exact sizes of the evidence files after they had been organised in the evidence directory. The values provided an additional reference for confirming that the files had not unexpectedly changed.

Part A - File Signatures and Hex Analysis



```
kali@kali:~/forensic_lab/working
$ cp -p J_ub_law.jpg ../working/

(kali@kali:~/forensic_lab/original_evidence)
$ cd ../working

(kali@kali:~/forensic_lab/working)
$ pwd
/home/kali/forensic_lab/working
$ ls -lh
total 1.7M
-r--r--r-- 1 kali kali 1.7M Sep  4 21:58 J_ub_law.jpg

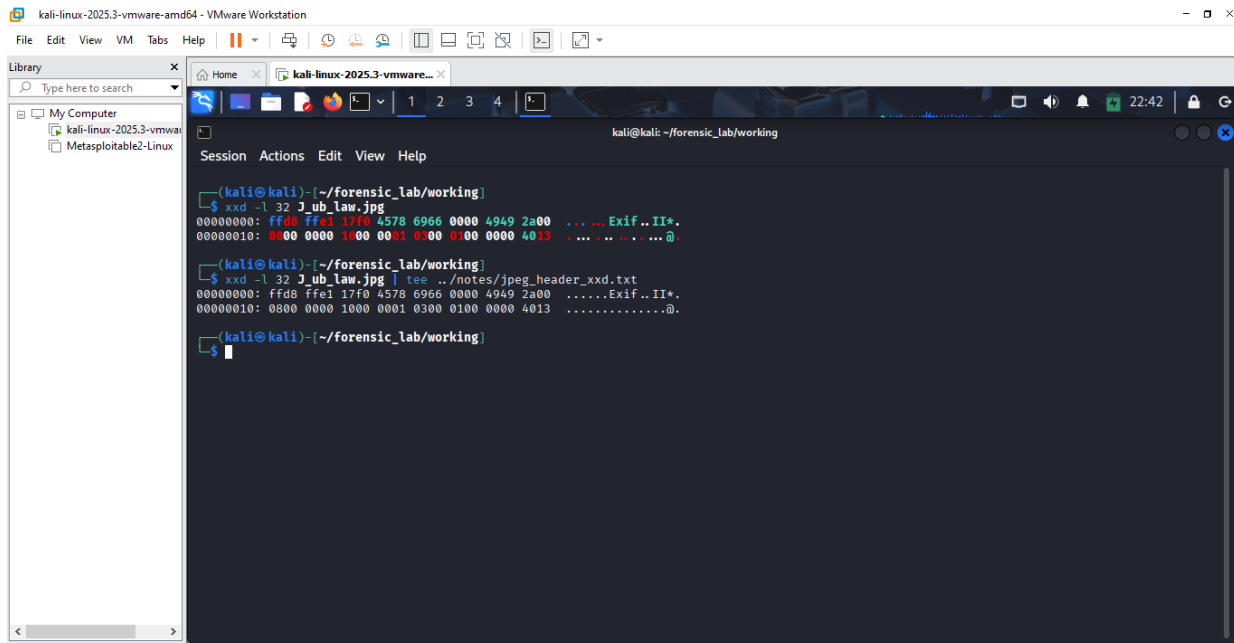
(kali@kali:~/forensic_lab/working)
$
```

Fig 1.10 Making a working copy of the JPEG.

Explanation

A separate copy of J_ub_law.jpg was created in the working directory before hexadecimal analysis began.

This ensured that the original evidence remained untouched while the JPEG was examined and reconstructed.



```
kali-linux-2025.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
Library
Type here to search
My Computer
kali-linux-2025.3-vmware-amd64
Metasploitable2-Linux

kali@kali: ~/forensic_lab/working
Session Actions Edit View Help

(kali@kali)~/forensic_lab/working
$ xxd -l 32 J_ub_lab.jpg
00000000: ffd8 ffe1 17f0 4578 6966 0000 4949 2a00  ....Exif..II*.
00000010: 0000 0000 1000 0001 0300 0100 0000 4013  ....@.....

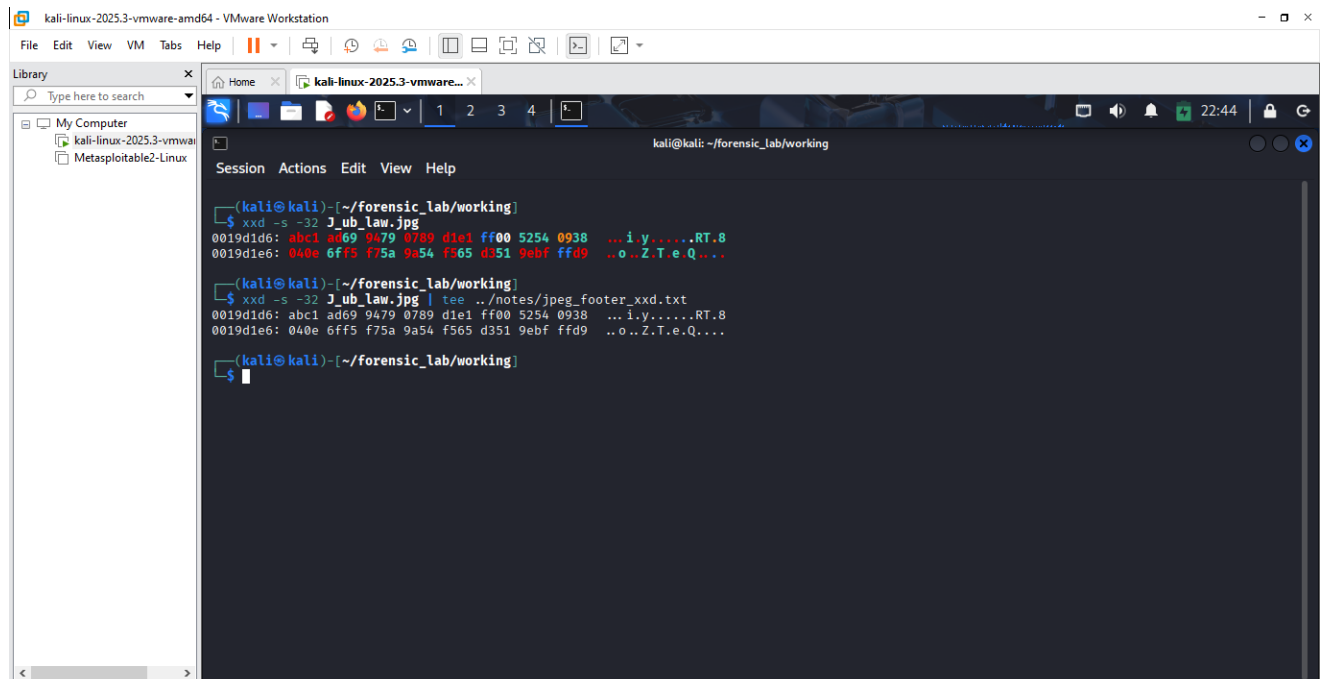
(kali@kali)~/forensic_lab/working
$ xxd -l 32 J_ub_lab.jpg | tee ../notes/jpeg_header_xxd.txt
00000000: ffd8 ffe1 17f0 4578 6966 0000 4949 2a00  ....Exif..II*.
00000010: 0000 0000 1000 0001 0300 0100 0000 4013  ....@.....

(kali@kali)~/forensic_lab/working
$
```

Fig 1.11 Examine the JPEG header with xxd.

Explanation

The beginning of the JPEG was displayed in hexadecimal form using xxd. The output revealed the standard FF D8 JPEG Start of Image marker followed by the FF E1 APP1 marker and readable Exif data.



```
kali-linux-2025.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
Library
Type here to search
My Computer
kali-linux-2025.3-vmware-amd64
Metasploitable2-Linux

kali@kali: ~/forensic_lab/working
Session Actions Edit View Help

(kali@kali)~/forensic_lab/working
$ xxd -s -32 J_ub_lab.jpg
0019d1d6: abc1 ad69 9479 0789 d1e1 ff00 5254 0938  ...i.y.....RT.8
0019d1e6: 040e 6ff5 f75a 9a54 f565 d351 9ebf ffd9  ...o..Z.T.e.Q...

(kali@kali)~/forensic_lab/working
$ xxd -s -32 J_ub_lab.jpg | tee ../notes/jpeg_footer_xxd.txt
0019d1d6: abc1 ad69 9479 0789 d1e1 ff00 5254 0938  ...i.y.....RT.8
0019d1e6: 040e 6ff5 f75a 9a54 f565 d351 9ebf ffd9  ...o..Z.T.e.Q...

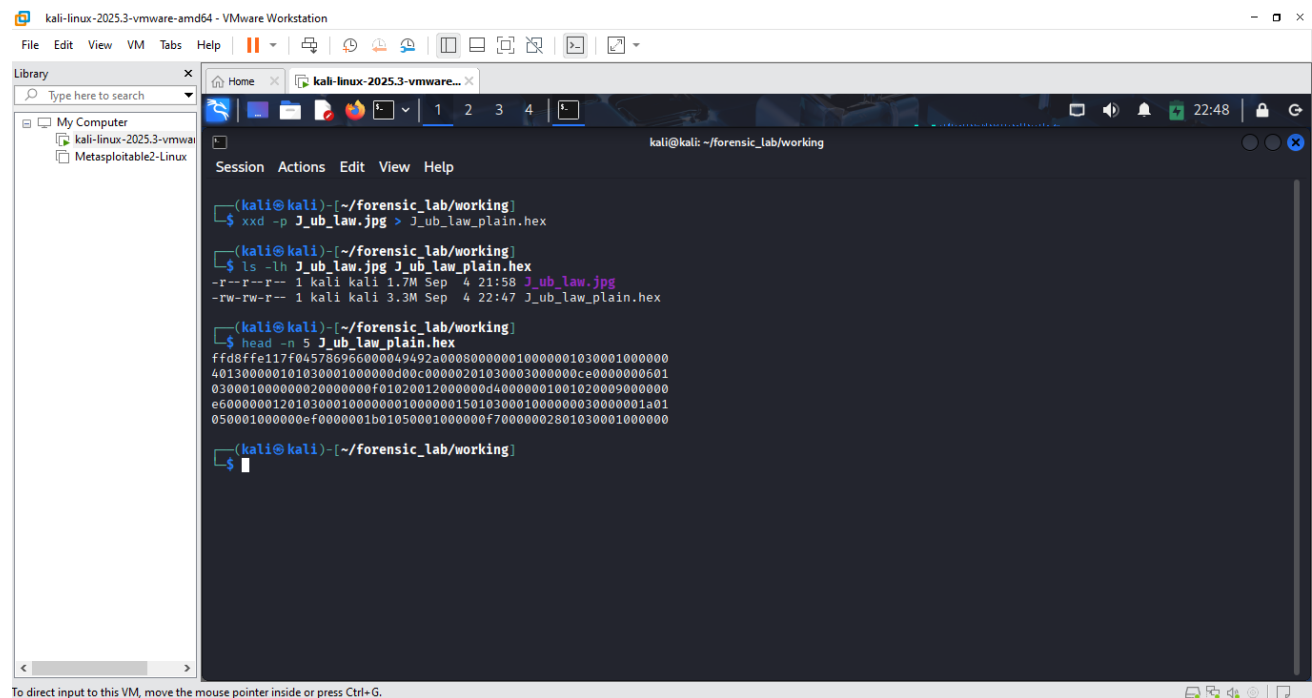
(kali@kali)~/forensic_lab/working
$
```

Fig 1.12 Examine the JPEG footer.

Explanation

The JPEG begins with the standard FF D8 SOI signature. Immediately following it is an FF E1 APP1 marker containing an EXIF structure.

The final bytes of the image were examined to locate the JPEG end marker. The FF D9 value at the end provided evidence that the expected JPEG footer was present.

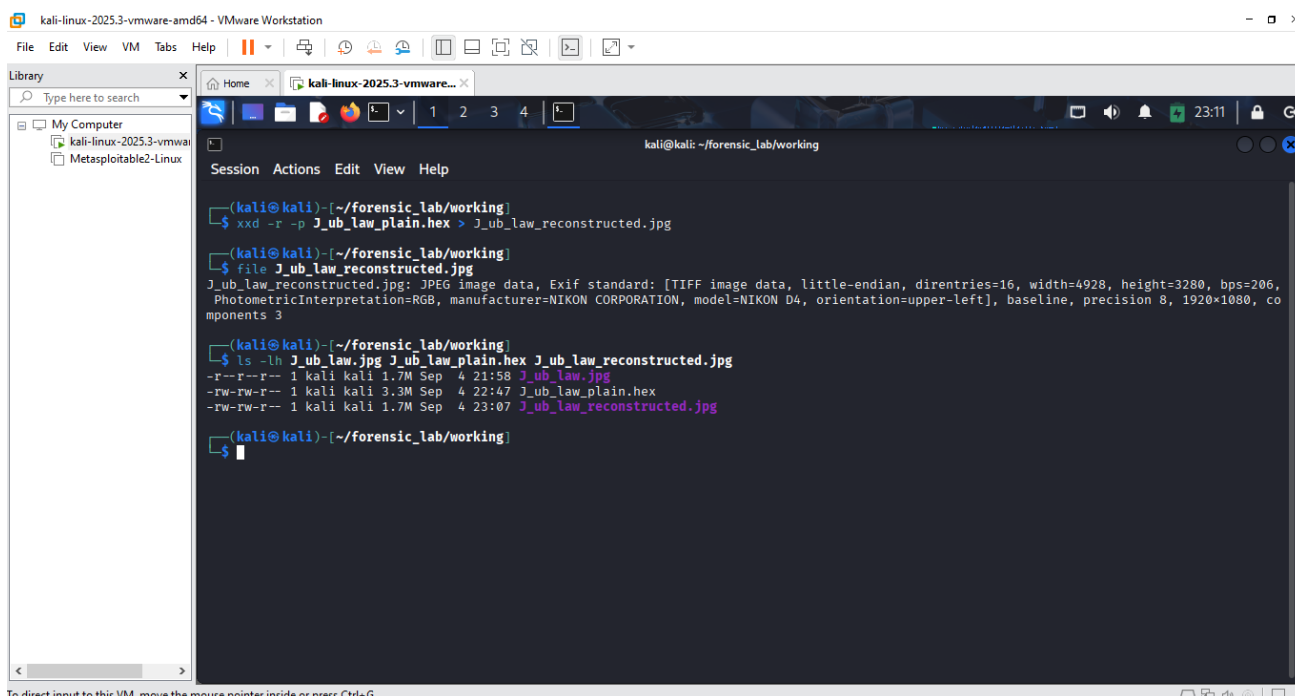


```
kali-linux-2025.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
Library
Type here to search
My Computer
kali-linux-2025.3-vmware-amd64
Metasploitable2-Linux
kali@kali: ~/forensic_lab/working
Session Actions Edit View Help
(kali@kali)~/forensic_lab/working
$ xxd -p J_ub_law.jpg > J_ub_law_plain.hex
(kali@kali)~/forensic_lab/working
$ ls -lh J_ub_law.jpg J_ub_law_plain.hex
-r--r--r-- 1 kali kali 1.7M Sep  4 21:58 J_ub_law.jpg
-rw-rw-r-- 1 kali kali 3.3M Sep  4 22:47 J_ub_law_plain.hex
(kali@kali)~/forensic_lab/working
$ head -n 5 J_ub_law_plain.hex
ffd8ffe117f045786956000049492a000800000010000001030001000000
401300000101030001000000d00c00000201030003000000ce0000000501
03000100000002000000f01020012000000d40000001001020009000000
e60000001201030001000000010000001501030001000000030000001a01
0500010000000ef0000001b01050001000000f70000002801030001000000
(kali@kali)~/forensic_lab/working
$
```

Fig 1.13 Creating a plain hexadecimal dump.

Explanation

The JPEG was converted into a plain-text hexadecimal representation. This demonstrated how binary image content can be represented as hexadecimal data while preserving the underlying byte values needed for reconstruction.



To direct input to this VM, move the mouse pointer inside or press Ctrl+G.

Fig 1.14 Reconstructing the JPEG from the hexadecimal dump.

Explanation

The plain hexadecimal dump was reversed using xxd to produce a reconstructed JPEG file. The resulting file was identified as a JPEG and could then be compared against the original working copy.

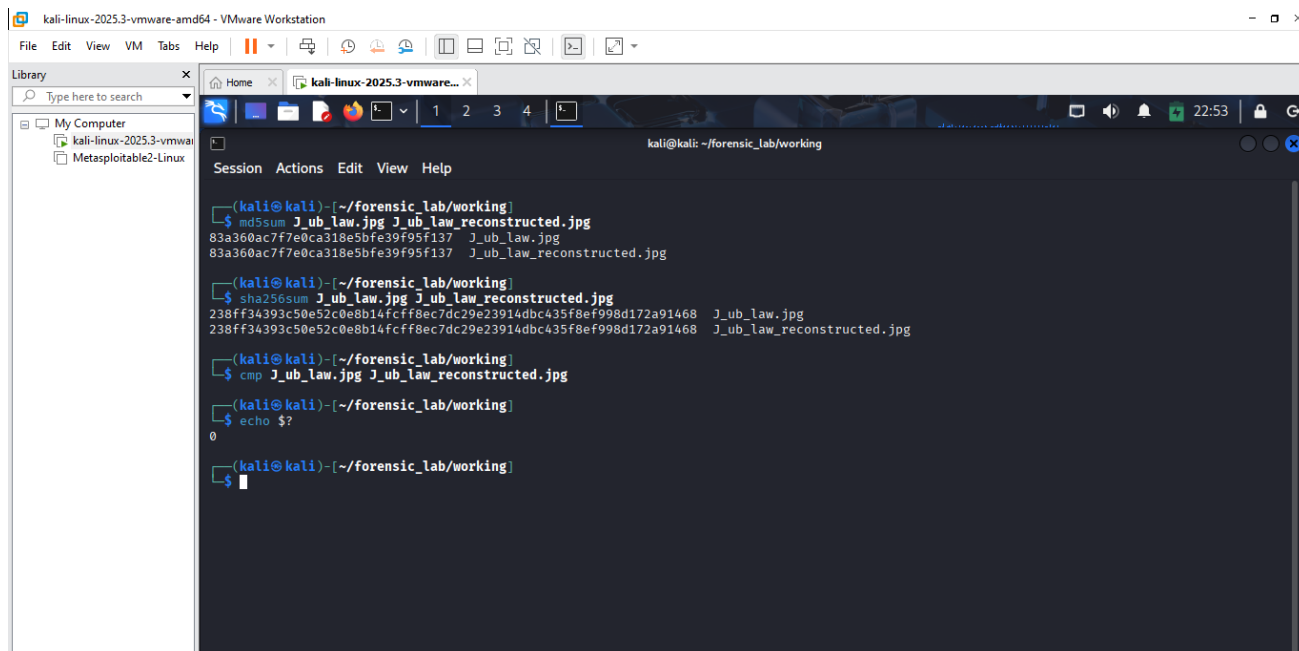


Fig 1.15 Cryptographically verify the reconstruction.

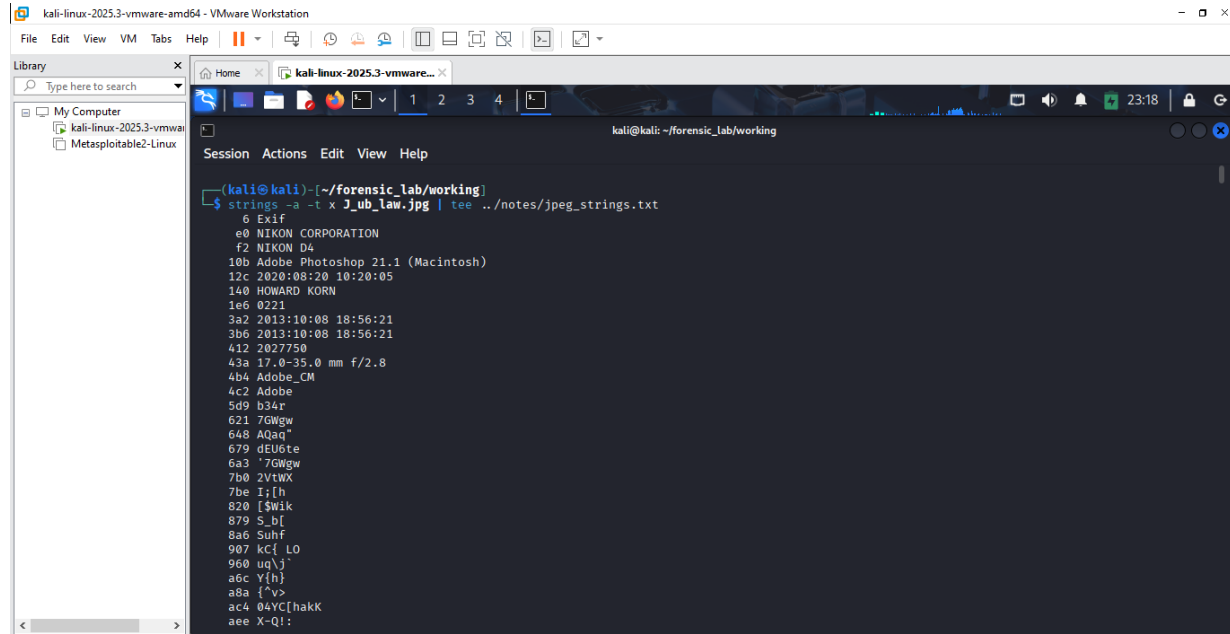
Explanation

The JPEG reconstructed from the hexadecimal dump produced identical MD5 and SHA-256 hashes to the source JPEG. A binary comparison using cmp also returned exit status 0, confirming that the reconstructed

file was identical to the source working copy.

MD5 and SHA-256 values for the working JPEG and reconstructed JPEG were identical. A cmp comparison also returned exit status 0, demonstrating that the reconstructed file matched the source working copy byte for byte.

Part B — examining metadata-related strings inside

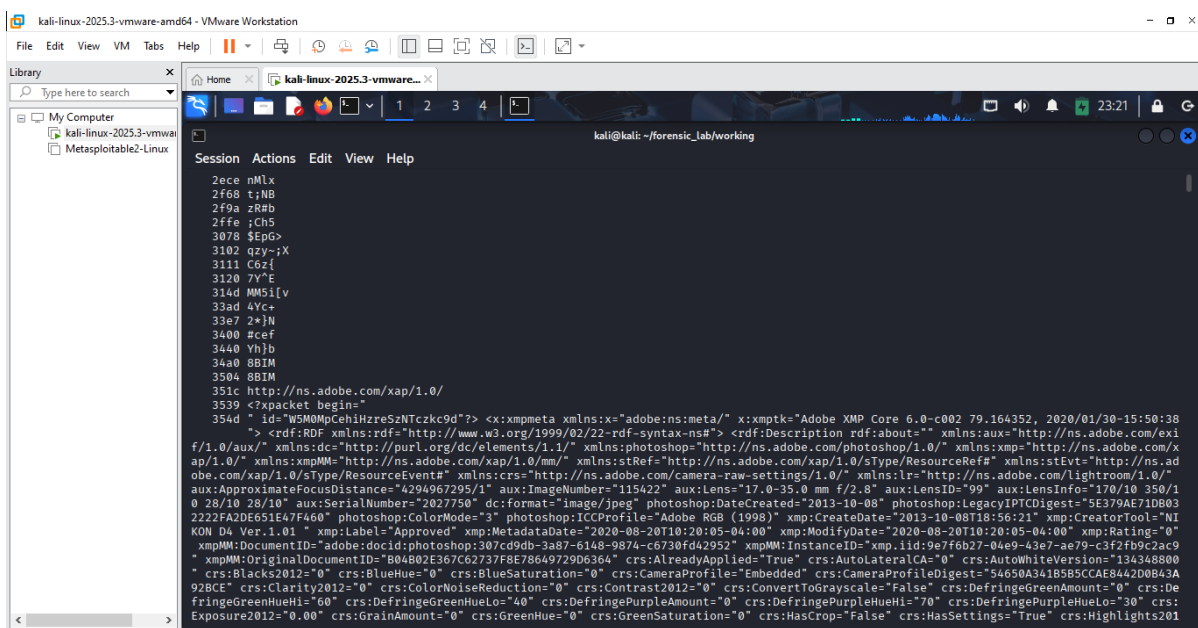


```
kali@kali: ~/forensic_lab/working
$ strings -t -x J_ub_law.jpg | tee ../notes/jpeg_strings.txt
6 Exif
e0 NIKON CORPORATION
f2 NIKON D4
10b Adobe Photoshop 21.1 (Macintosh)
12c 2020:08:20 10:20:05
140 HOWARD KORN
1e6 0221
3a2 2013:10:08 18:56:21
3b6 2013:10:08 18:56:21
412 2027750
43a 17.0-35.0 mm f/2.8
4b4 Adobe_CM
4c2 Adobe
5d9 b34r
621 7GWgw
648 A0aq"
679 dEU6te
6a3 '7GWgw
7b0 2VtWX
7be I;[h
820 [$Wik
879 S_b[
8a6 Suhf
907 xC[ LO
900 uq\j
a6c Y[h]
a8a {^v}
ac4 04YC[hakk
aee X-Q!;
```

Fig 1.16 The metadata-like content examination.

Explanation

The strings command was used to extract readable text from the JPEG together with hexadecimal offsets. The output immediately revealed EXIF-related data and several useful strings, including camera, software, name, date, and lens-related information.



```
kali@kali: ~/forensic_lab/working
$ strings -t -x J_ub_law.jpg | tee ../notes/jpeg_strings.txt
...
351c http://ns.adobe.com/xap/1.0/
3539 <?packet begin="
354d " id="WSM0MpCehiHzreSzNTczkc9d"?> <x:xmpmeta xmlns:x="adobe:meta/" x:xmpptk="Adobe XMP Core 6.0-c002 79.164352, 2020/01/30-15:50:38
"> <rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#" > <rdf:Description rdf:about="" xmlns:aux="http://ns.adobe.com/exi
f/1.0/aux/" xmlns:dc="http://purl.org/dc/elements/1.1/" xmlns:photoshop="http://ns.adobe.com/photoshop/1.0/" xmlns:xmp="http://ns.adobe.com/x
ap/1.0/" xmlns:xmpMM="http://ns.adobe.com/xap/1.0/mm/" xmlns:stRef="http://ns.adobe.com/xap/1.0/stype/ResourceRef#" xmlns:stEvt="http://ns.ad
obe.com/xap/1.0/stype/ResourceEvent#" xmlns:crs="http://ns.adobe.com/camera-raw-settings/1.0/" xmlns:lr="http://ns.adobe.com/Lightroom/1.0/"
aux:ApproximateFocusDistance="4294967295/1" aux:ImageNumber="115422" aux:Lens="17.0-35.0 mm f/2.8" aux:LensID="99" aux:LensInfo="170/10 350/1
0 28/10 28/10" aux:SerialNumbers="2027750" dc:format="image/jpeg" photoshop:DataCreated="2013-10-08" photoshop:LegacyIPTCDigest="5E379AE71DB03
222FA2DE651E47F460" photoshop:ColorMode="3" photoshop:ICCPProfile="Adobe RGB (1998)" xmp:CreateDate="2013-10-08T18:56:21" xmp:CreatorTool="NI
KON D4 Ver.1.01" xmp:Label="Approved" xmp:MetadataDate="2020-08-20T10:20:05-04:00" xmp:ModifyDate="2020-08-20T10:20:05-04:00" xmp:Rating="0"
xmpMM:DocumentID="adobe:docid:photoshop:307cd9db-3a87-6148-9874-c6730fd42952" xmpMM:InstanceID="xmp.iid:9e7f6b27-04e9-43e7-ae79-c3f2fb9c2ac9"
xmpMM:OriginalDocumentID="B04802E367C62737F8E78649729D6364" crs:AlreadyApplied="True" crs:AutoLateralCA="0" crs:AutoWhiteVersion="134348800"
crs:Blacks2012="0" crs:BlueHue="0" crs:BlueSaturation="0" crs:CameraProfile="Embedded" crs:CameraProfileDigest="54650A3418585CAE842D0B43A
920EF" crs:Clarity2012="0" crs:ColorNoiseReduction="0" crs:Contrast2012="0" crs:ConvertToGrayscale="False" crs:DeFringeGreenAmount="0" crs:De
fringeGreenHueHi="60" crs:DeFringeGreenHueLo="40" crs:DeFringePurpleAmount="0" crs:DeFringePurpleHueHi="70" crs:DeFringePurpleHueLo="30" crs:
Exposure2012="0.00" crs:GrainAmount="0" crs:GreenHue="0" crs:GreenSaturation="0" crs:HasCrop="False" crs:HasSettings="True" crs:Highlights201
```

Fig 1.17 The metadata-like content examination page 1.

Explanation

This screenshot continues the readable-string examination and shows a larger section of embedded XML/XMP-style information. It demonstrates that considerably more information was stored within the JPEG than was visible from the image alone.

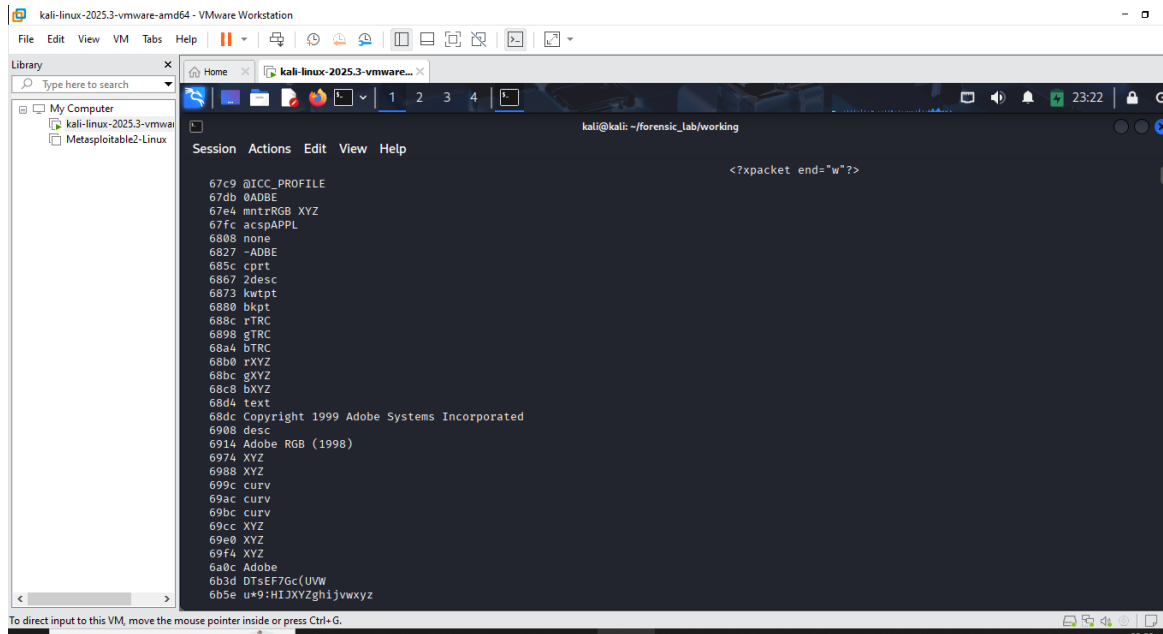


Fig 1.18 The metadata-like content examination page 2.

Explanation

Further readable strings were displayed from the JPEG, including Adobe-related information. The output also contained a readable Adobe copyright string, which became part of the documented metadata findings.

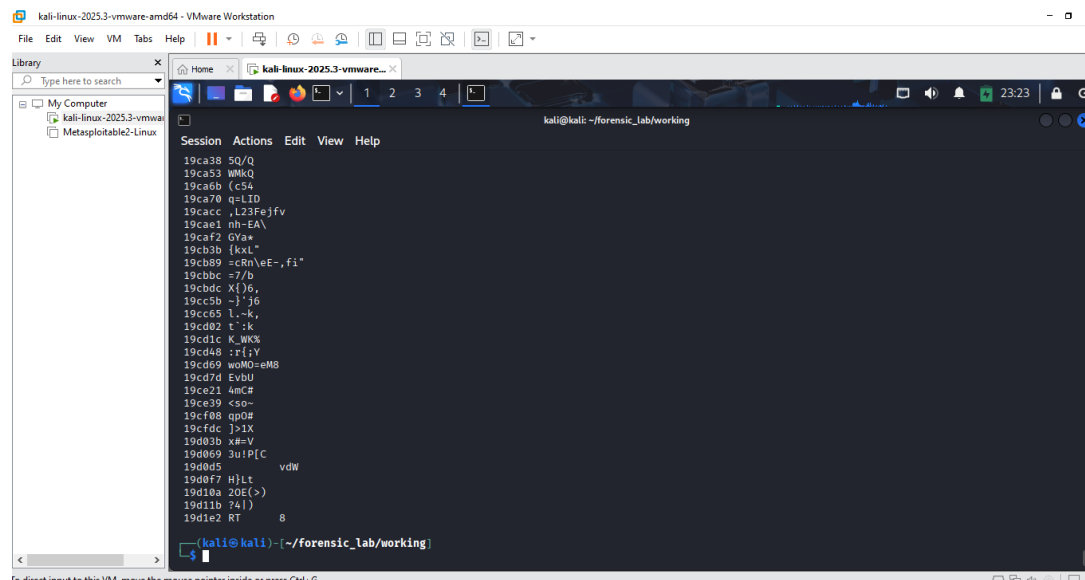


Fig 1.19 The metadata-like content examination page 3.

Explanation

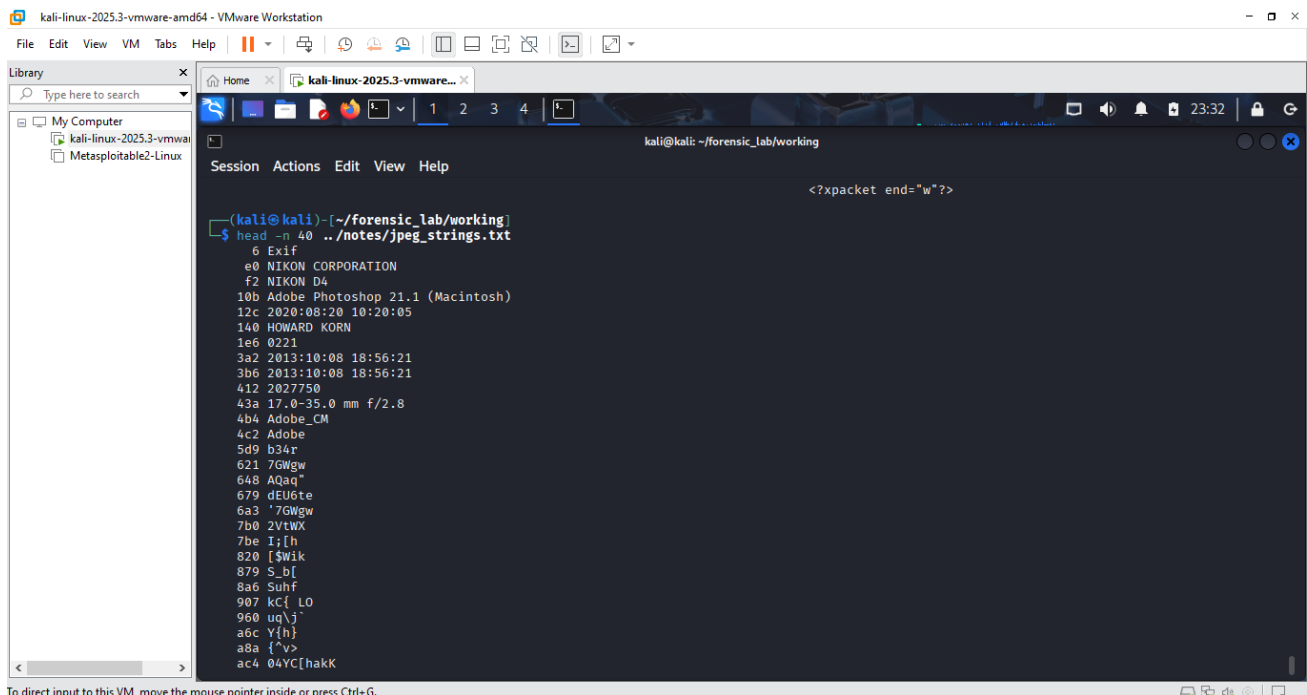
This screenshot records the continuation of the string output through later offsets in the JPEG. Much of this

```
kali-linux-2023.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
Library
Type here to search
My Computer
kali-linux-2023.3-vmware
Metasploitable2-Linux
kali@kali: ~/forensic_lab/working
Session Actions Edit View Help
19cae1 nh-EA\
19cafa 0va~
19cb3b [kxL"
19cbb9 =cRn\oE-,fi"
19cbbc 7/b
19cbcc X()6,
19cc5b -j"j6
19cc65 L-,k,
19cd02 t":k
19cd1c K_WK%
19cd48 rfr:Y
19cd69 woMnD-qM8
19cd7d EvbU
19ce21 4mC#
19ce39 cso~
19cf08 gpd%
19cf4c j>1X
19d03b xM=V
19d069 3uIP[C
19d0d5
vdW
19d0f7 Nltt
19d10a 20E(>)
19d11b 74()
19d1e2 RT 8
[kali@kali] ~/forensic_lab/working
$ grep -iE 'make model camera software artist|author|copyright|comment|date|time|description' ../notes/jpeg_strings.txt
354d id="W5M0MmpCehiHrreS2NTczkc9d"?> <x:xmpmeta xmlns:x="adobe:ns:meta/" x:xmp:ts="Adobe XMP Core 6.0-c002 79.164352, 2020/01/30-15:50:38
"> <rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"> <rdf:Description rdf:about="" xmlns:aux="http://ns.adobe.com/exi
f/1.0/aux/" xmlns:dc="http://purl.org/dc/elements/1.1/" xmlns:photoshop="http://ns.adobe.com/photoshop/1.0/" xmlns:xmp="http://ns.adobe.com/x
ap/1.0/" xmlns:xmpMM="http://ns.adobe.com/xap/1.0/mm/" xmlns:stRef="http://ns.adobe.com/xap/1.0/stdio/ResourceRef#" xmlns:stEvt="http://ns.ad
obe.com/xap/1.0/stdio/ResourceEvent#" xmlns:crs="http://ns.adobe.com/xmpcrs-raw-settings/1.0/" xmlns:lr="http://ns.adobe.com/Lightroom/1.0/"
```

Explanation

Explanation

This screenshot shows additional XMP information associated with Adobe software and processing history. It also displays the copyright-related text recovered from within the file.

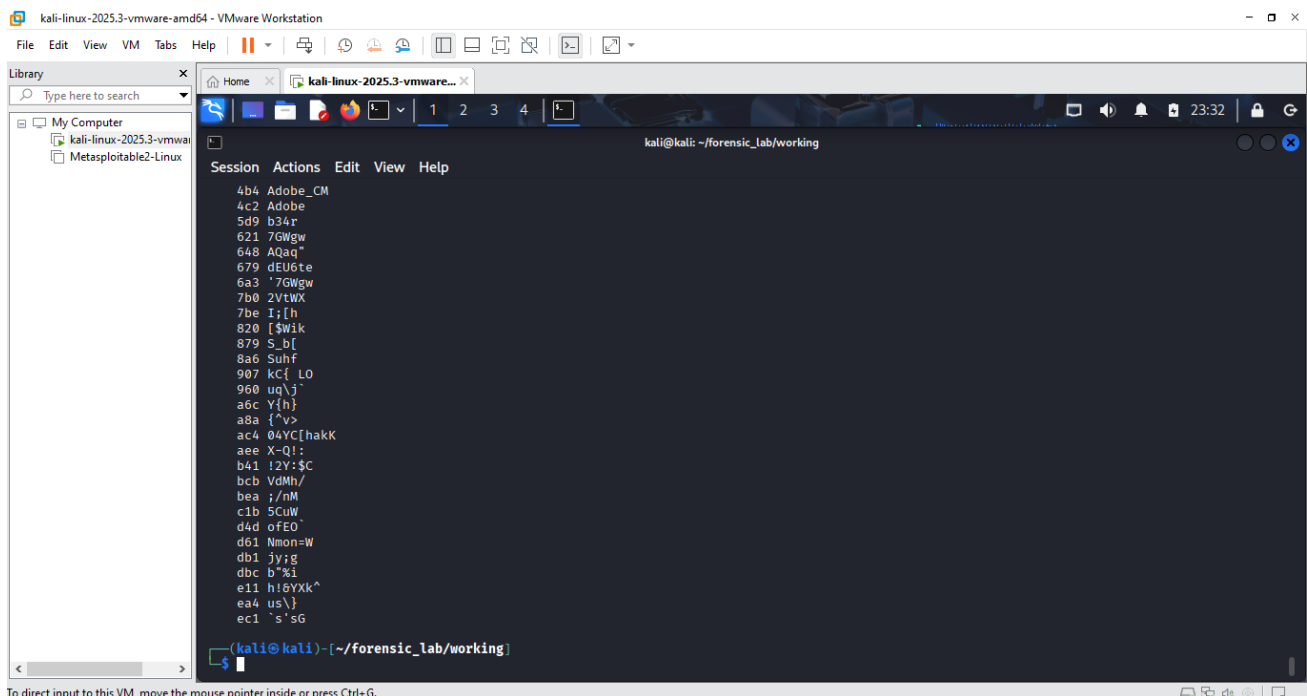


To direct input to this VM, move the mouse pointer inside or press Ctrl+G.

Fig 1.24 The metadata-like content examination page 8.

Explanation

The beginning of the saved strings output clearly displays several important findings, including NIKON CORPORATION, NIKON D4, Adobe Photoshop 21.1 (Macintosh), HOWARD KORN, date/time values, and the lens-related string.

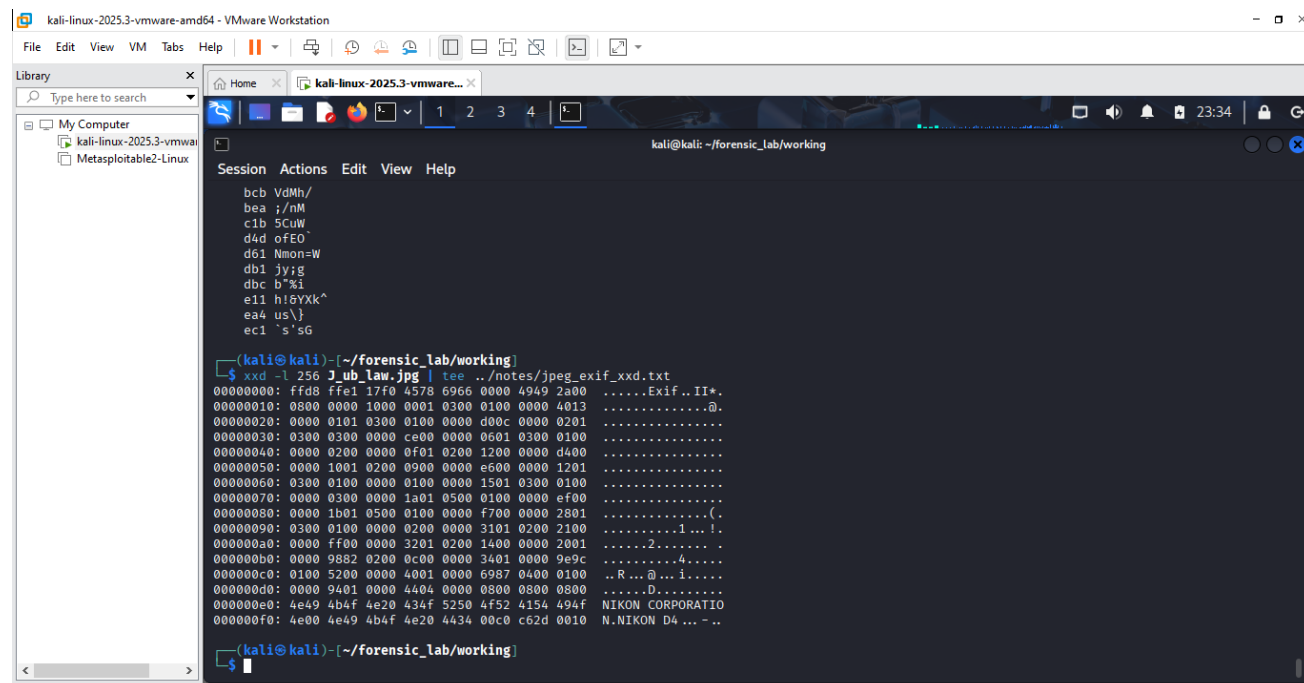


To direct input to this VM, move the mouse pointer inside or press Ctrl+G.

Fig 1.25 The metadata-like content examination page 9.

Explanation

This screenshot shows the continuation of the extracted strings beyond the main readable metadata fields. It documents that the examination was not limited to a single matched value but covered the wider file contents.

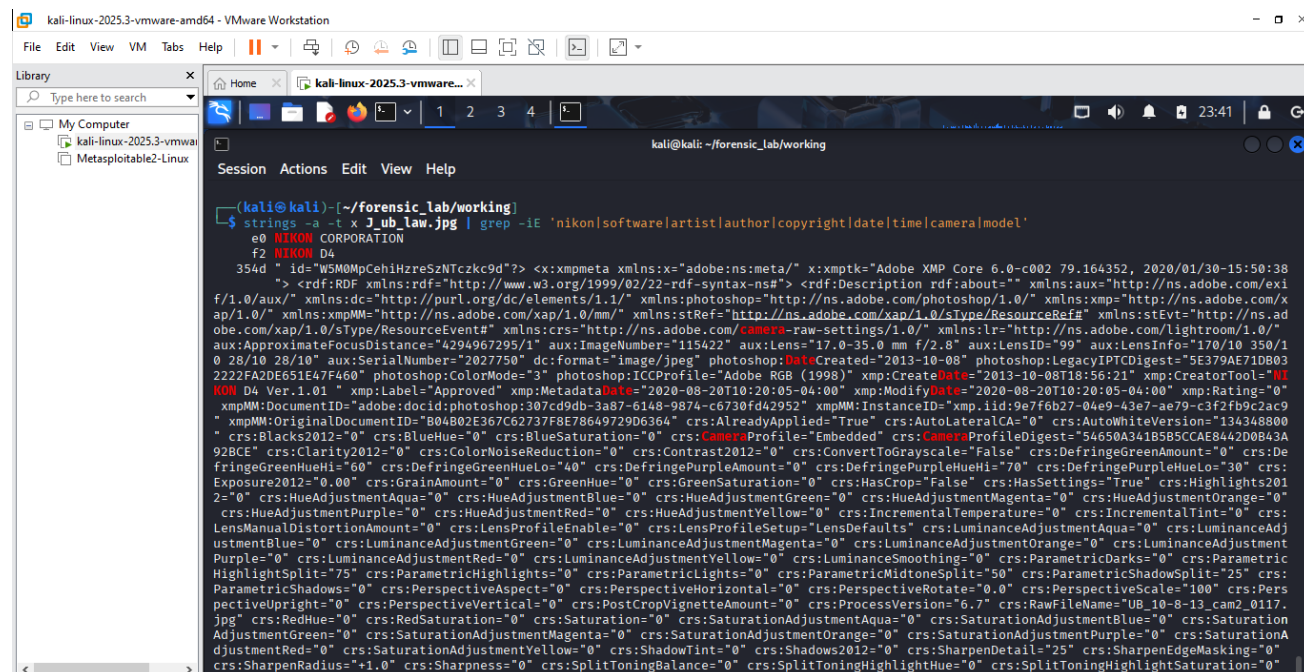


```
kali@kali: ~/forensic_lab/working
$ xxd -l 256 J_ub_law.jpg | tee ../notes/jpeg_exif_xxd.txt
00000000: ffd8 ffe1 17f0 4578 6966 0000 4949 2a00  ....Exif..II*.
00000010: 0800 0000 1000 0001 0300 0100 0000 4013  ....@.....
00000020: 0000 0101 0300 0100 0000 d00c 0000 0201  ....
00000030: 0300 0300 0000 ce00 0000 0601 0300 0100  ....
00000040: 0000 0200 0000 0f01 0200 1200 0000 d400  ....
00000050: 0000 1001 0200 0900 0000 e600 0000 1201  ....
00000060: 0300 0100 0000 0100 0000 1501 0300 0100  ....
00000070: 0000 0300 0000 1a01 0500 0100 0000 ef00  ....
00000080: 0000 1b01 0500 0100 0000 f700 0000 2801  ....
00000090: 0300 0100 0000 0200 0000 3101 0200 2100  ....1...!
000000a0: 0000 ffd0 0000 3201 0200 1400 0000 2001  ....2.....
000000b0: 0000 9882 0200 0c00 0000 3401 0000 9e9c  ....4.....
000000c0: 0100 5200 0000 4001 0000 6987 0400 0100  ..R...1....
000000d0: 0000 9401 0000 4404 0000 0800 0800 0800  ....D.....
000000e0: 4e49 4b4f 4e20 434f 5250 4f52 4154 494f  NIKON CORPORATION
000000f0: 4e00 4e49 4b4f 4e20 4434 00c0 c62d 0010  N.NIKON D4 ...-
```

Fig 1.26 The metadata-like content examination page 10.

Explanation

The first 256 bytes of the JPEG were inspected with xxd, where readable camera information could be seen within the EXIF area. NIKON CORPORATION and NIKON D4 were visible in the hexadecimal output.



```
kali@kali: ~/forensic_lab/working
$ strings -a -t x J_ub_law.jpg | grep -iE 'nikon|software|artist|author|copyright|date|time|camera|model'
e0 NIKON CORPORATION
f2 NIKON D4
354d " id="WSM0MpCehiHzreSzNTczkc9d"?> <x:xmpmeta xmlns:x="adobe:meta/" x:xmp:tk="Adobe XMP Core 6.0-c002 79.164352, 2020/01/30-15:50:38
"> <rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#" <rdf:Description rdf:about="" xmlns:aux="http://ns.adobe.com/exi
f/1.0/aux/" xmlns:dc="http://purl.org/dc/elements/1.1/" xmlns:photoshop="http://ns.adobe.com/photoshop/1.0/" xmlns:xmp="http://ns.adobe.com/x
ap/1.0/" xmlns:xmpMM="http://ns.adobe.com/xap/1.0/mm/" xmlns:stRef="http://ns.adobe.com/xap/1.0/stype/ResourceRef#" xmlns:stEvt="http://ns.ad
obe.com/xap/1.0/stype/ResourceEvent#" xmlns:crs="http://ns.adobe.com/camera-raw-settings/1.0/" xmlns:lr="http://ns.adobe.com/lightroom/1.0/"
aux:ApproximateFocusDistance="4294967295/1" aux:ImageNumber="115422" aux:Lens="17.0-35.0 mm f/2.8" aux:LensID="99" aux:LensInfo="170/10 350/1
0 28/10 28/10" aux:SerialNumber="2027750" dc:format="image/jpeg" photoshop:DataCreated="2013-10-08" photoshop:LegacyIPTCDigest="5E379AE71DB03
222fA20F651E7F460" photoshop:ColorMode="3" photoshop:ICCPProfile="Adobe RGB (1998)" xmp:CreateDate="2013-10-08T19:56:21" xmp:CreatorTool="VI
RON D4 Ver.1.01" xmp:Label="Approved" xmp:MetadataDate="2020-08-20T10:20:05-04:00" xmp:ModifyDate="2020-08-20T10:20:05-04:00" xmp:Rating="0"
xmpMM:DocumentID="adobe:docid:photoshop:307ced9b-3a87-6148-9874-c6730fd42952" xmpMM:InstanceID="xmp.iid:9e7feb2b-04e9-43e7-ae79-c3f2fb9c2ac9
" xmpMM:OriginalDocumentID="B04B02E367C62737F8E78649729D6364" crs:AlreadyApplied="True" crs:AutoLateralCA="0" crs:AutoWhiteVersions="134348800
" crs:Black2012="0" crs:BlueHue="0" crs:BlueSaturation="0" crs:CameraProfile="Embedded" crs:CameraProfileDigest="54650A34185B5SCAE8A42D0B43A
92BCE" crs:Clarity2012="0" crs:ColorNoiseReduction="0" crs:Contrast2012="0" crs:ConvertToGrayscale="False" crs:DefringeGreenAmount="0" crs:De
fringeGreenHueHi="60" crs:DefringeGreenHueLo="40" crs:DefringePurpleHueHi="70" crs:DefringePurpleHueLo="30" crs:
Exposure2012="0.00" crs:GrainAmount="0" crs:GreenHue="0" crs:GreenSaturation="0" crs:HasCrop="False" crs:HasSettings="True" crs:Highlights201
2="0" crs:HueAdjustmentAqua="0" crs:HueAdjustmentBlue="0" crs:HueAdjustmentGreen="0" crs:HueAdjustmentMagenta="0" crs:HueAdjustmentOrange="0" crs:
HueAdjustmentPurple="0" crs:HueAdjustmentRed="0" crs:HueAdjustmentYellow="0" crs:IncrementalTemperature="0" crs:IncrementalTint="0" crs:
LensManualDistortionAmount="0" crs:LensProfileEnable="0" crs:LensProfileSetups="LensDefaults" crs:LuminanceAdjustmentAqua="0" crs:LuminanceAdj
ustmentBlue="0" crs:LuminanceAdjustmentGreen="0" crs:LuminanceAdjustmentMagenta="0" crs:LuminanceAdjustmentOrange="0" crs:LuminanceAdjustment
Purple="0" crs:LuminanceAdjustmentRed="0" crs:LuminanceAdjustmentYellow="0" crs:LuminanceSmoothing="0" crs:ParametricDarkness="0" crs:Parametric
HighlightSplit="75" crs:ParametricHighlights="0" crs:ParametricLights="0" crs:ParametricMidtoneSplit="50" crs:ParametricShadowSplit="25" crs:
ParametricShadows="0" crs:PerspectiveAspect="0" crs:PerspectiveHorizontal="0" crs:PerspectiveRotate="0" crs:PerspectiveScale="100" crs:Pers
pectiveUpright="0" crs:PerspectiveVertical="0" crs:PostCropVignetteAmount="0" crs:ProcessVersion="6.7" crs:RawFileName="UB_10-8-13_cam2_0117.
jpg" crs:RedHue="0" crs:RedSaturation="0" crs:Saturation="0" crs:SaturationAdjustmentAqua="0" crs:SaturationAdjustmentBlue="0" crs:Saturation
AdjustmentGreen="0" crs:SaturationAdjustmentMagenta="0" crs:SaturationAdjustmentOrange="0" crs:SaturationAdjustmentPurple="0" crs:SaturationA
djustmentRed="0" crs:SaturationAdjustmentYellow="0" crs:ShadowTint="0" crs:Shadows2012="0" crs:SharpenDetails="25" crs:SharpenEdgeMasking="0"
crs:SharpenRadius="+1.0" crs:Sharpness="0" crs:SplitToningBalance="0" crs:SplitToningHighlightHue="0" crs:SplitToningHighlightSaturation="0"
```

Fig 1.27 The metadata-like content examination page 11.

Explanation

A targeted metadata search brought together Nikon, software, date, camera, and other matching strings from the JPEG. The screenshot provides a focused view of the information that was considered relevant to the investigation.

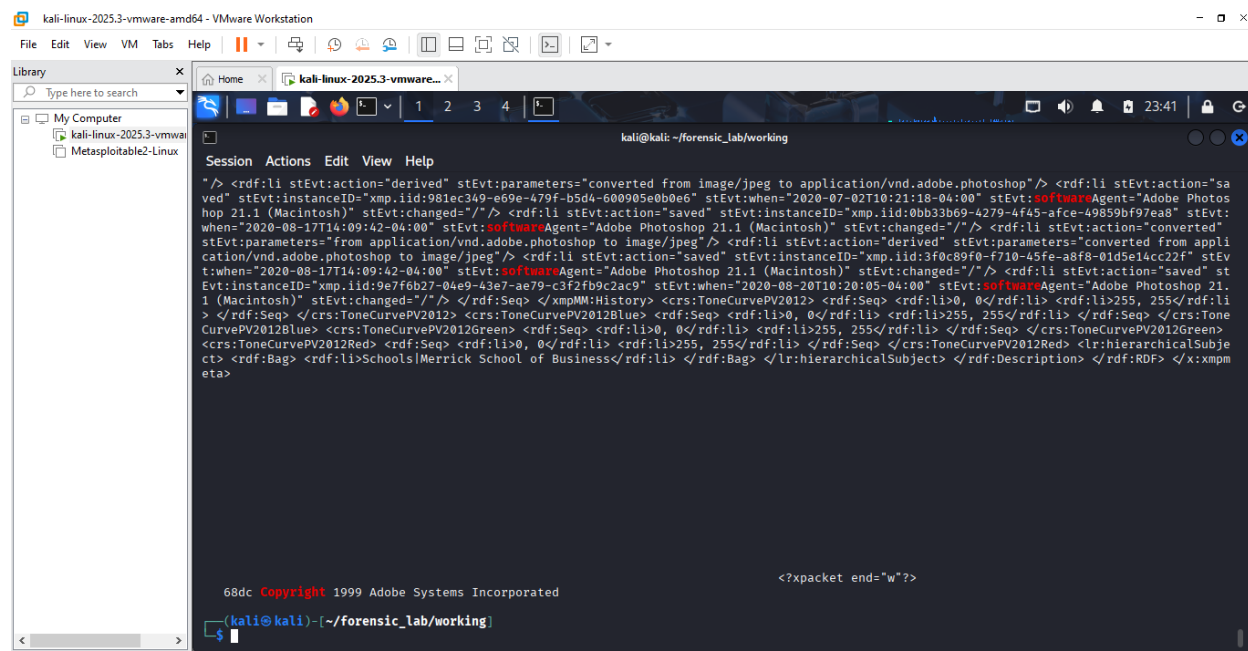


Fig 1.28 The metadata-like content examination page 12.

Explanation

This continuation displays more of the XMP and Adobe-related metadata identified by the targeted search. The presence of processing and software information supported the conclusion that the image contained embedded editing-related metadata.

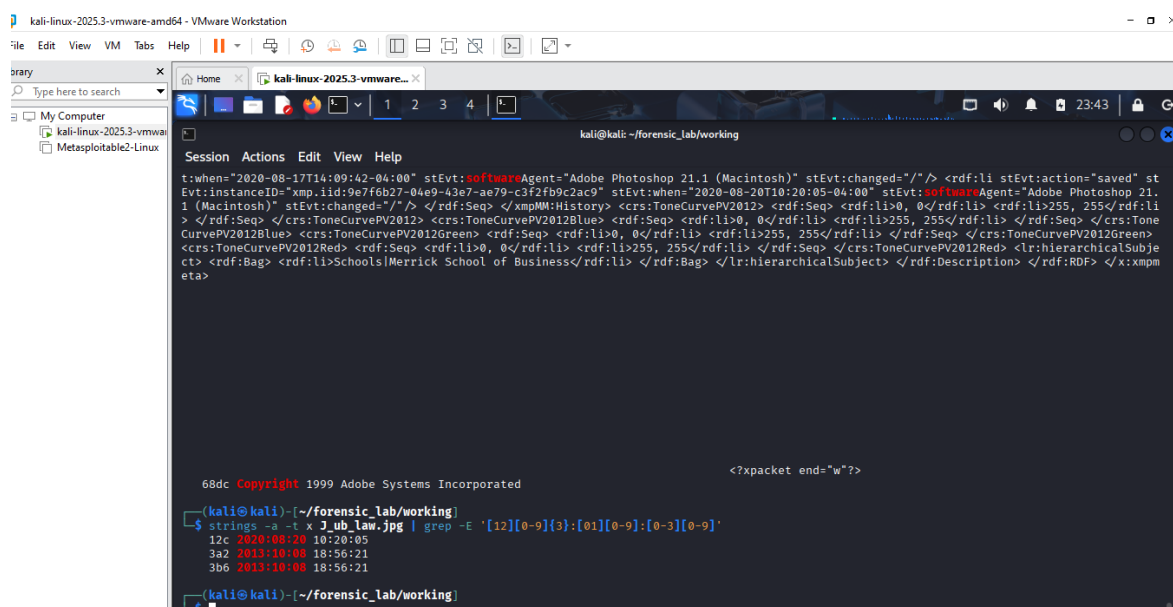


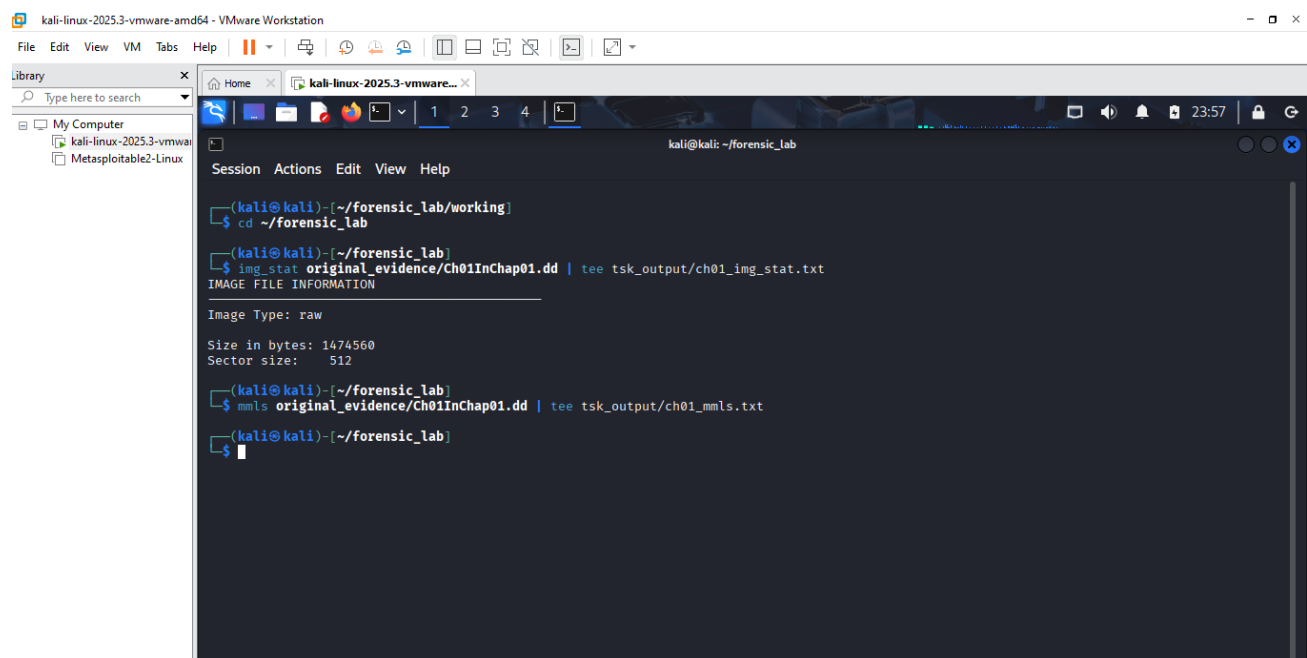
Fig 1.29 The metadata-like content examination page 13.

The final metadata screenshot records the date search and additional metadata output. Together, the metadata examination identified Nikon camera information, Adobe Photoshop information, the name-related string, lens information, and multiple date/time values.

Explanation

Examination of J_ub_law.jpg using xxd and strings identified an EXIF structure and several readable metadata-related strings. The camera manufacturer was identified as NIKON CORPORATION and the model as NIKON D4. The file contained references to Adobe Photoshop 21.1 (Macintosh), a name-related string HOWARD KORN, the lens-related string 17.0-35.0 mm f/2.8, and date/time values including 2020:08:20 10:20:05 and 2013:10:08 18:56:21. Adobe/XMP metadata and an embedded camera-profile reference were also observed. Only information actually present in the examined data was recorded.

Part C - Direct Data-Unit Examination with The Sleuth Kit



```
kali-linux-2025.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
Library
Type here to search
My Computer
kali-linux-2025.3-vmware-amd64
Metasploitable2-Linux

kali@kali: ~/forensic_lab
Session Actions Edit View Help
(kali@kali)-[~/forensic_lab/working]
$ cd ~/forensic_lab
(kali@kali)-[~/forensic_lab]
$ img_stat original_evidence/Ch01InChap01.dd | tee tsk_output/ch01_img_stat.txt
IMAGE FILE INFORMATION
Image Type: raw
Size in bytes: 1474560
Sector size: 512
(kali@kali)-[~/forensic_lab]
$ mmls original_evidence/Ch01InChap01.dd | tee tsk_output/ch01_mmls.txt
(kali@kali)-[~/forensic_lab]
$
```

Fig 2. Examining the Ch01InChap01.dd.

img_stat identified the evidence as a raw image with a size of 1,474,560 bytes and 512-byte sectors. mmls produced no partition table, so the investigation did not assume an artificial partition offset.

Explanation

mmls was executed against Ch01InChap01.dd; no partition table was returned. Therefore, no arbitrary partition offset was assumed, and subsequent filesystem analysis was performed directly against the image.

```
kali@kali: ~/forensic_lab
$ fsstat original_evidence/Ch01InChap01.dd | tee tsk_output/ch01_fsstat.txt
FILE SYSTEM INFORMATION
-----
File System Type: FAT12

OEM Name: .k0nIHC
Volume ID: 0x0
Volume Label (Boot Sector):
Volume Label (Root Directory):
File System Type Label: *3*MI

Sectors before file system: 0

File System Layout (in sectors)
Total Range: 0 - 2879
* Reserved: 0 - 0
** Boot Sector: 0
* FAT 0: 1 - 9
* FAT 1: 10 - 18
* Data Area: 19 - 2879
** Root Directory: 19 - 32
** Cluster Area: 33 - 2879

METADATA INFORMATION
-----
Range: 2 - 45782
Root Directory: 2

CONTENT INFORMATION
-----
Sector Size: 512
Cluster Size: 512
```

Fig 2.1 Examining the Ch01InChap01.dd page 2.

Explanation

fsstat identified the file system as FAT12 and displayed its layout, including FAT areas, the root directory, data area, and cluster region. This provided the structural information needed for later file and sector examination.

```
kali@kali: ~/forensic_lab
File System Type Label: *3*MI

Sectors before file system: 0

File System Layout (in sectors)
Total Range: 0 - 2879
* Reserved: 0 - 0
** Boot Sector: 0
* FAT 0: 1 - 9
* FAT 1: 10 - 18
* Data Area: 19 - 2879
** Root Directory: 19 - 32
** Cluster Area: 33 - 2879

METADATA INFORMATION
-----
Range: 2 - 45782
Root Directory: 2

CONTENT INFORMATION
-----
Sector Size: 512
Cluster Size: 512
Total Cluster Range: 2 - 2848

FAT CONTENTS (in sectors)
-----
33-236 (204) -> EOF
285-311 (27) -> EOF

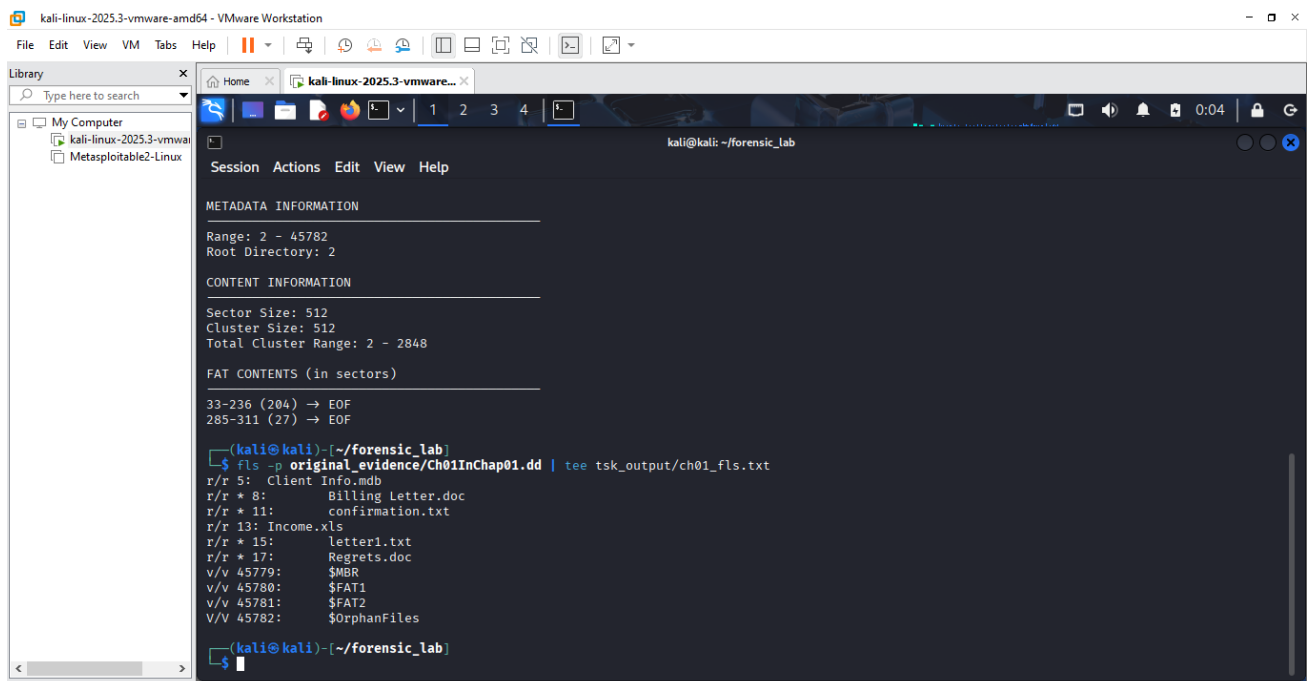
(kali@kali)~/forensic_lab
$
```

Fig 2.2 Examining the Ch01InChap01.dd page 3.

Explanation

This continuation records the remaining FAT12 layout and content information, including sector and cluster

sizes. It completed the initial file-system description of the image.

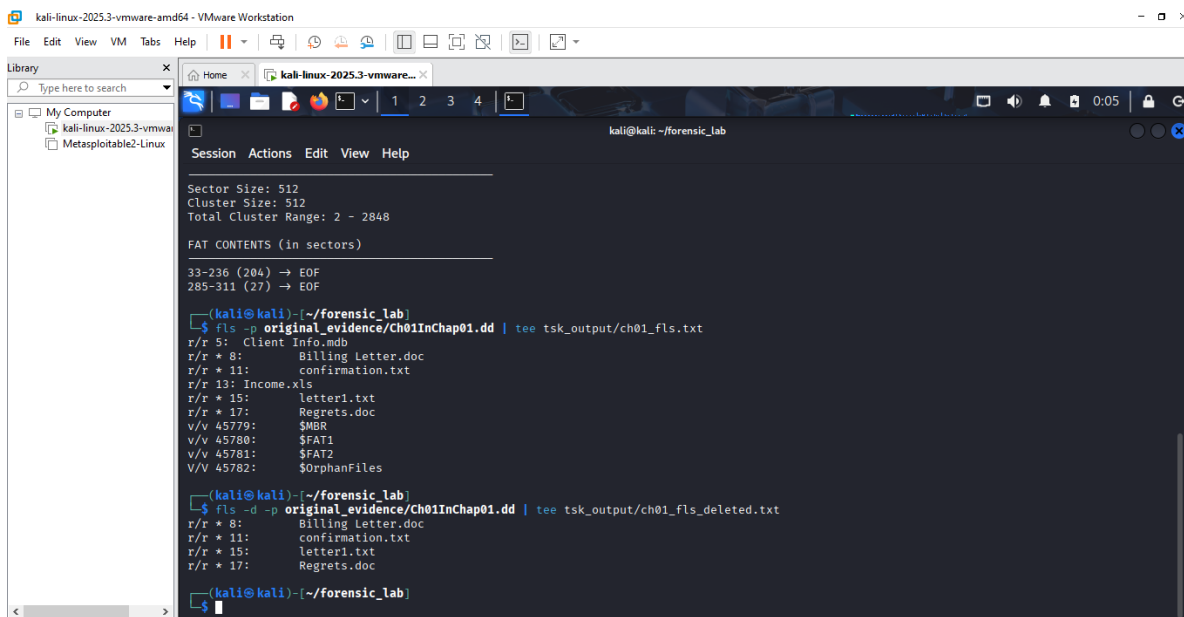


```
kali@kali: ~/forensic_lab
$ fls -p original_evidence/Ch01InChap01.dd | tee tsk_output/ch01_fls.txt
r/r 5: Client Info.mdb
r/r * 8: Billing Letter.doc
r/r * 11: confirmation.txt
r/r 13: Income.xls
r/r * 15: letter1.txt
r/r * 17: Regrets.doc
v/v 45779: $MBR
v/v 45780: $FAT1
v/v 45781: $FAT2
V/V 45782: $OrphanFiles
```

Fig 2.3 Examining the Ch01InChap01.dd page 4.

Explanation

fls was used to list directory entries from the FAT12 image. The output included normal and deleted files together with their metadata addresses, giving specific values that could be used for subsequent recovery.



```
kali@kali: ~/forensic_lab
$ fls -d -p original_evidence/Ch01InChap01.dd | tee tsk_output/ch01_fls_deleted.txt
r/r * 8: Billing Letter.doc
r/r * 11: confirmation.txt
r/r * 15: letter1.txt
r/r * 17: Regrets.doc
```

Fig 2.4 Examining the Ch01InChap01.dd page 5.

Explanation

The deleted-file listing was narrowed to entries marked as deleted. Files such as Billing Letter.doc, confirmation.txt, letter1.txt, and Regrets.doc were visible with their actual metadata addresses.

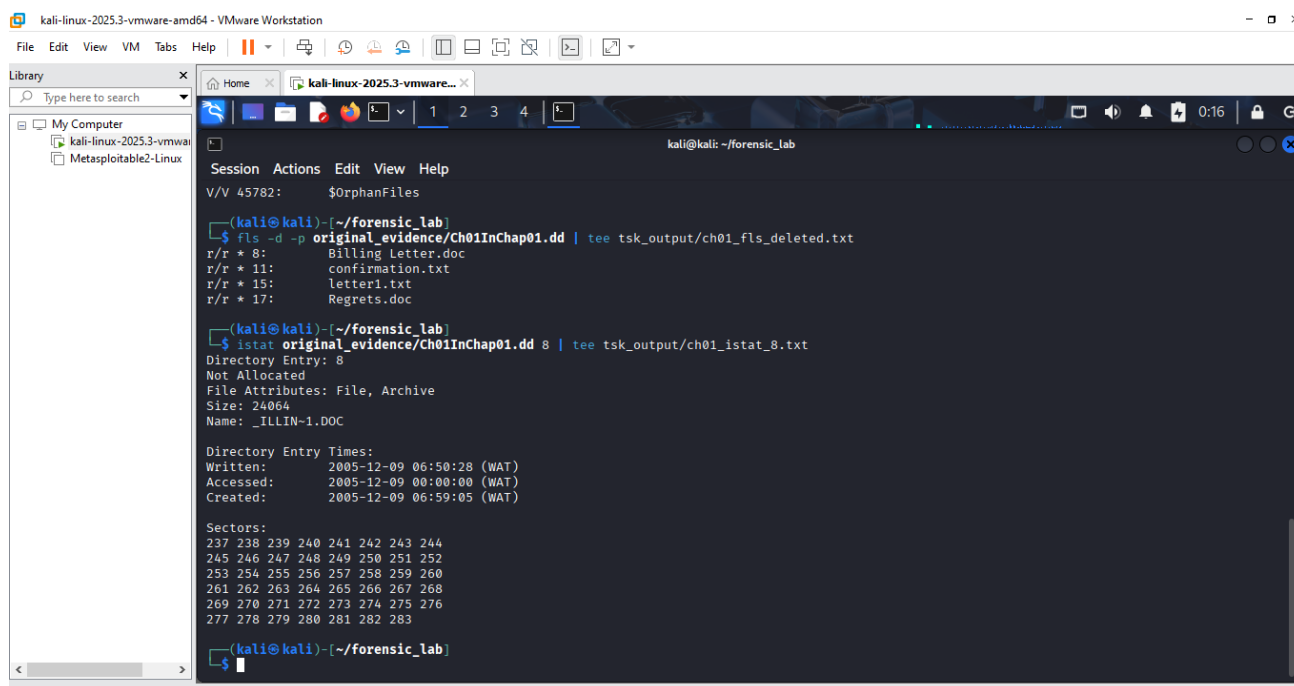


Fig 2.5 inspecting one deleted file with istat.

Explanation

Metadata entry 8 was examined with istat. The output showed that the entry was not allocated, had a size of 24,064 bytes, and occupied sectors 237 through 283.

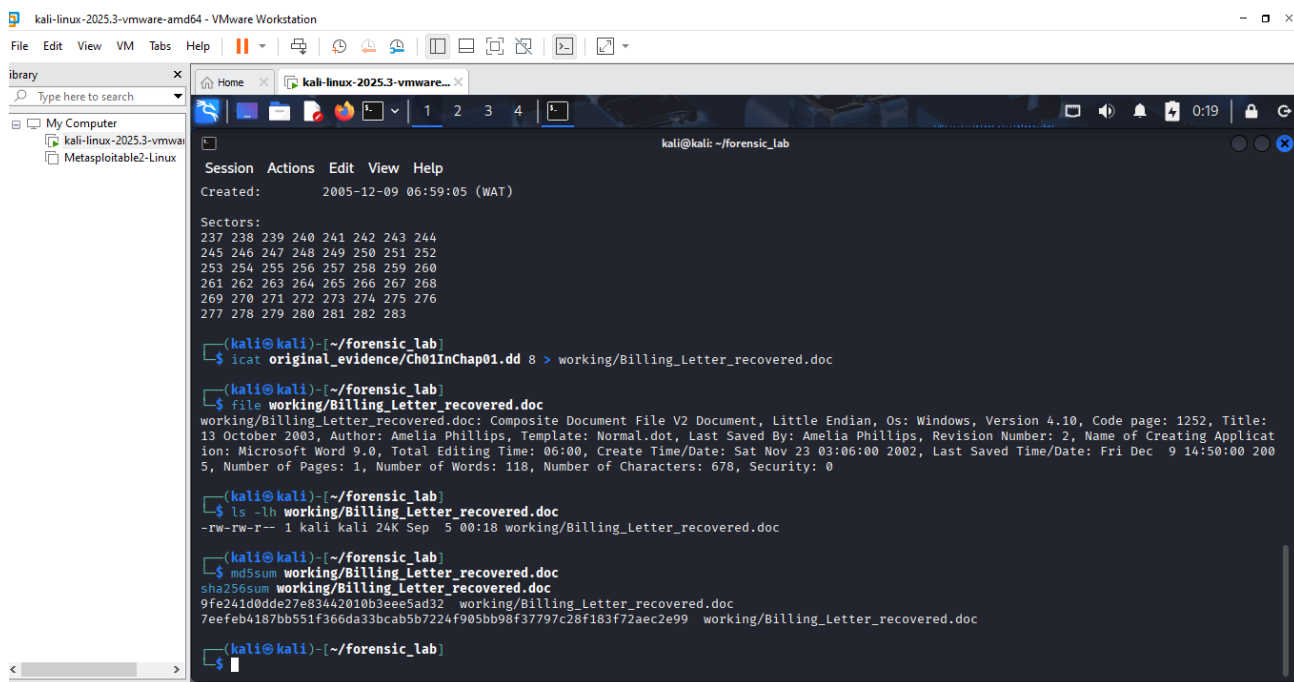
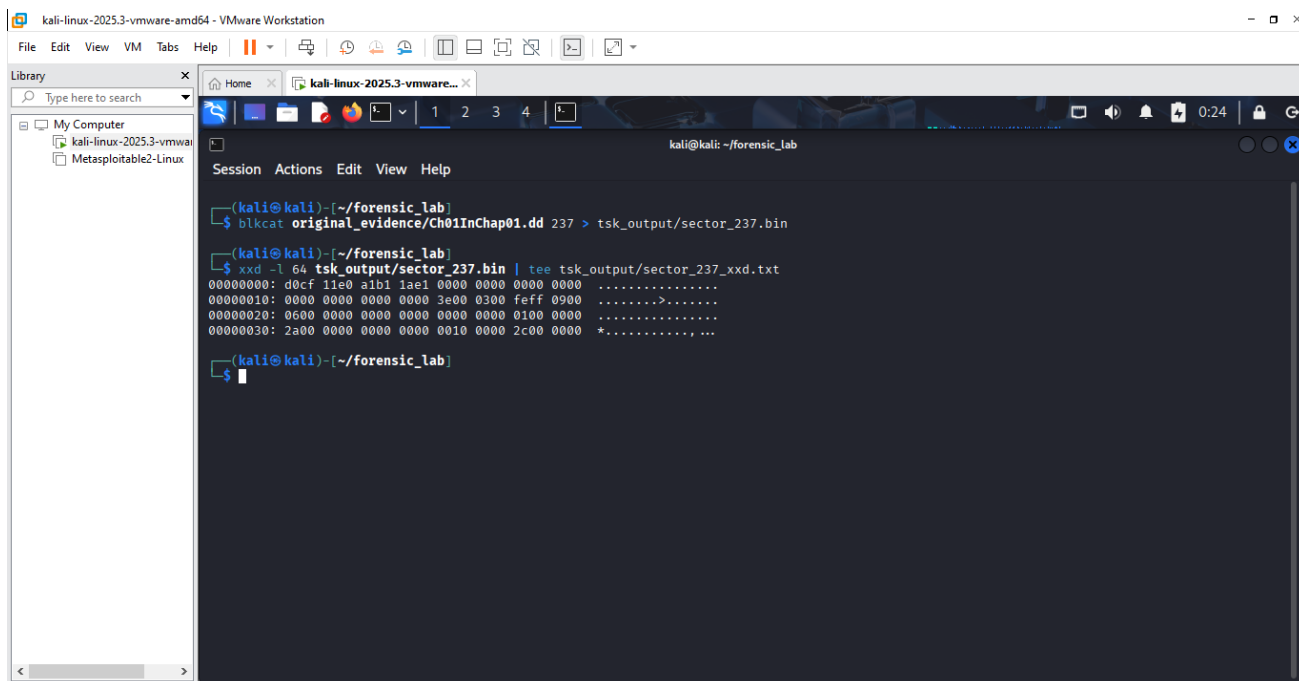


Fig 2.6 Extracting it with icat.

Explanation

icat was used with metadata address 8 to recover the deleted document. The recovered file was recognised as a Microsoft Compound Document, and MD5 and SHA-256 hashes were generated to preserve its integrity

information.



```
kali@kali: ~/forensic_lab
$ blkcat original_evidence/Ch01InChap01.dd 237 > tsk_output/sector_237.bin

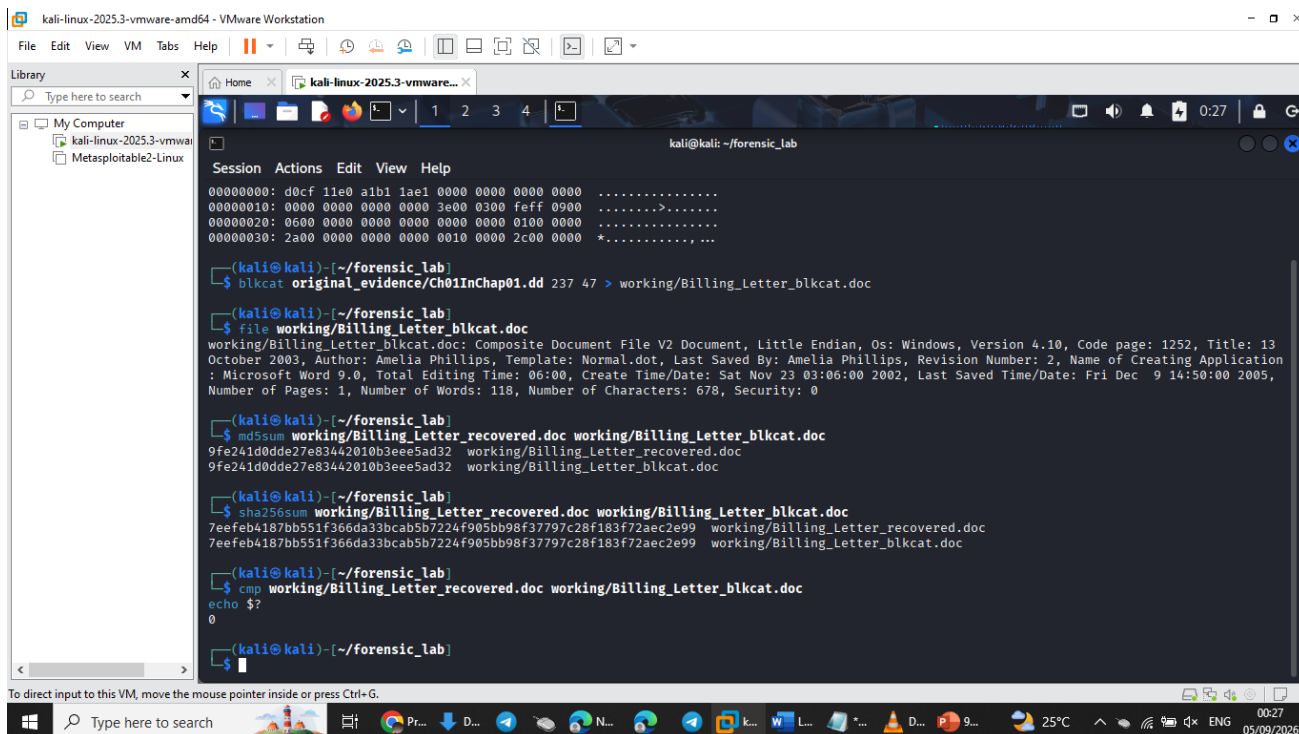
(kali@kali) ~/forensic_lab
$ xxd -l 64 tsk_output/sector_237.bin | tee tsk_output/sector_237_xxd.txt
00000000: d0cf 11e0 a1b1 1ae1 0000 0000 0000 0000 .....
00000010: 0000 0000 0000 0000 3e00 0300 feff 0900 .....>.....
00000020: 0600 0000 0000 0000 0000 0000 0100 0000 .....
00000030: 2a00 0000 0000 0000 0010 0000 2c00 0000 *....., ...

(kali@kali) ~/forensic_lab
$
```

Fig 2.7 Completing the blkcat requirement.

Explanation

Sector 237, identified from the istat result, was extracted directly with blkcat and examined with xxd. The output began with D0 CF 11 E0 A1 B1 1A E1, matching the Microsoft Compound File signature.



```
kali@kali: ~/forensic_lab
$ blkcat original_evidence/Ch01InChap01.dd 237 > working/Billing_Letter_blkcat.doc

(kali@kali) ~/forensic_lab
$ file working/Billing_Letter_blkcat.doc
working/Billing_Letter_blkcat.doc: Composite Document File V2 Document, Little Endian, Os: Windows, Version 4.10, Code page: 1252, Title: 13
October 2003, Author: Amelia Phillips, Template: Normal.dot, Last Saved By: Amelia Phillips, Revision Number: 2, Name of Creating Application
: Microsoft Word 9.0, Total Editing Time: 06:00, Create Time/Date: Sat Nov 23 03:06:00 2002, Last Saved Time/Date: Fri Dec 9 14:50:00 2005,
Number of Pages: 1, Number of Words: 118, Number of Characters: 678, Security: 0

(kali@kali) ~/forensic_lab
$ md5sum working/Billing_Letter_recovered.doc working/Billing_Letter_blkcat.doc
9fe241d0dde27e83442010b3eee5ad32 working/Billing_Letter_recovered.doc
9fe241d0dde27e83442010b3eee5ad32 working/Billing_Letter_blkcat.doc

(kali@kali) ~/forensic_lab
$ sha256sum working/Billing_Letter_recovered.doc working/Billing_Letter_blkcat.doc
7eeFeb4187b551f366da33bcab5b7224f905bb98f37797c28f183f72aec2e99 working/Billing_Letter_recovered.doc
7eeFeb4187b551f366da33bcab5b7224f905bb98f37797c28f183f72aec2e99 working/Billing_Letter_blkcat.doc

(kali@kali) ~/forensic_lab
$ cmp working/Billing_Letter_recovered.doc working/Billing_Letter_blkcat.doc
echo $?
0

(kali@kali) ~/forensic_lab
$
```

Fig 2.8 Completing the blkcat requirement page 1.

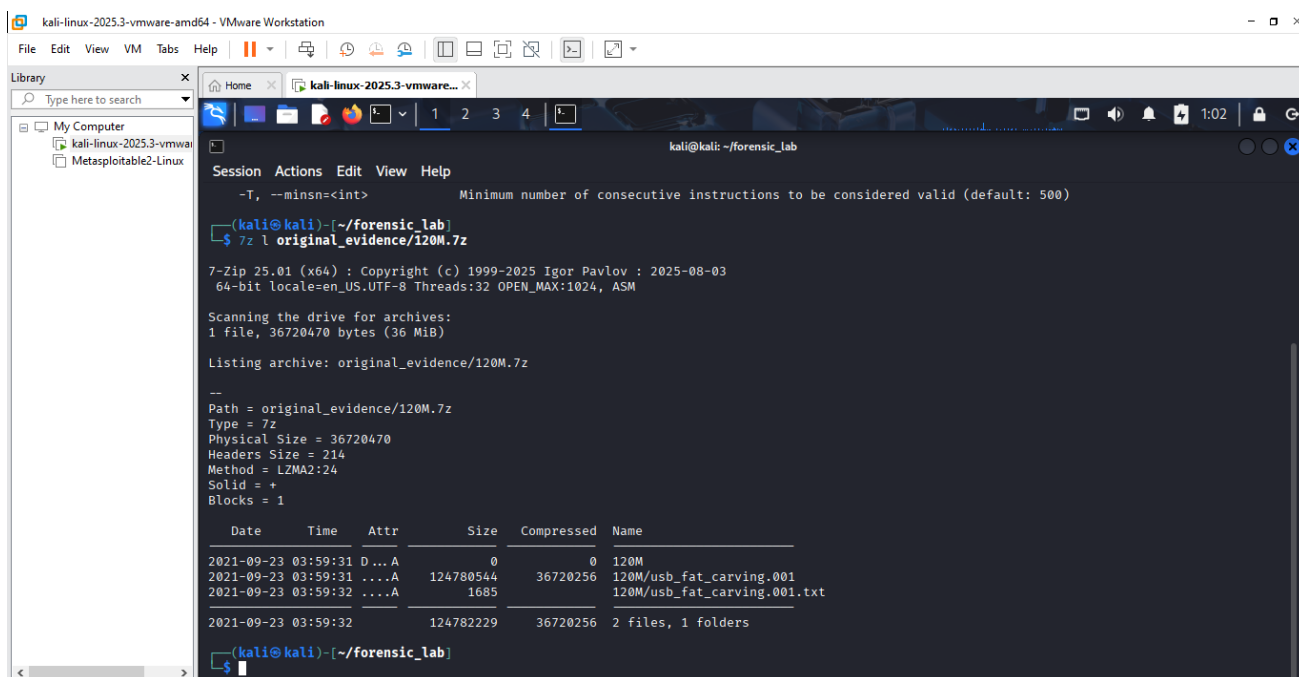


Fig 3.0 Searching for the doc file page 1.

Explanation

The contents of 120M.7z were inspected using 7z l. The archive contained usb_fat_carving.001 and its associated text file, but the required File_carving.docx was not listed.

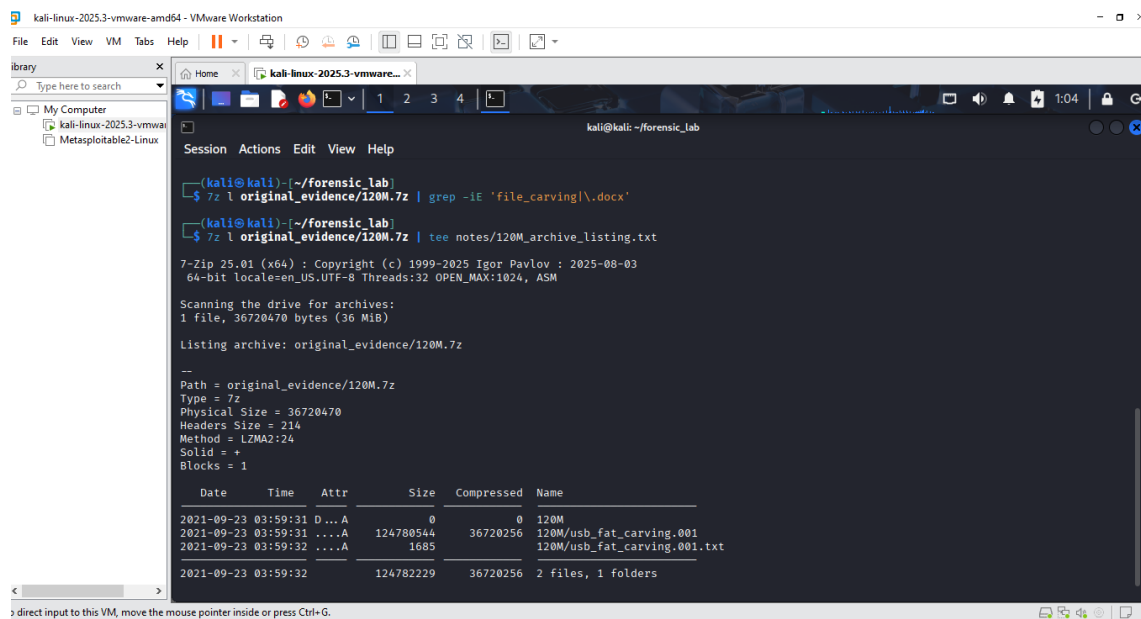


Fig 3.1 Searching for the doc file page 2.

Explanation

Binwalk detected JPEG/EXIF data, a little-endian TIFF structure, and an Adobe copyright string inside J_ub_law.jpg. No recognised Binwalk signatures were returned for Ch01InChap01.dd, so the investigation continued with the supplied USB forensic image.

Limitation – File_carving.docx:

The laboratory instructions reference an instructor-provided File_carving.docx for the demonstrated manual and automatic Binwalk extraction workflow. A recursive search of the available working directories did not locate the file. The supplied 120M.7z archive was also inspected using 7z l; it contained usb_fat_carving.001 and an associated text file, but no File_carving.docx. Consequently, the DOCX-specific workflow could not be reproduced. In accordance with the laboratory instructions, Binwalk practice was instead completed using the other authorised evidence files.

Complete the Binwalk practice using the other supplied lab evidence.

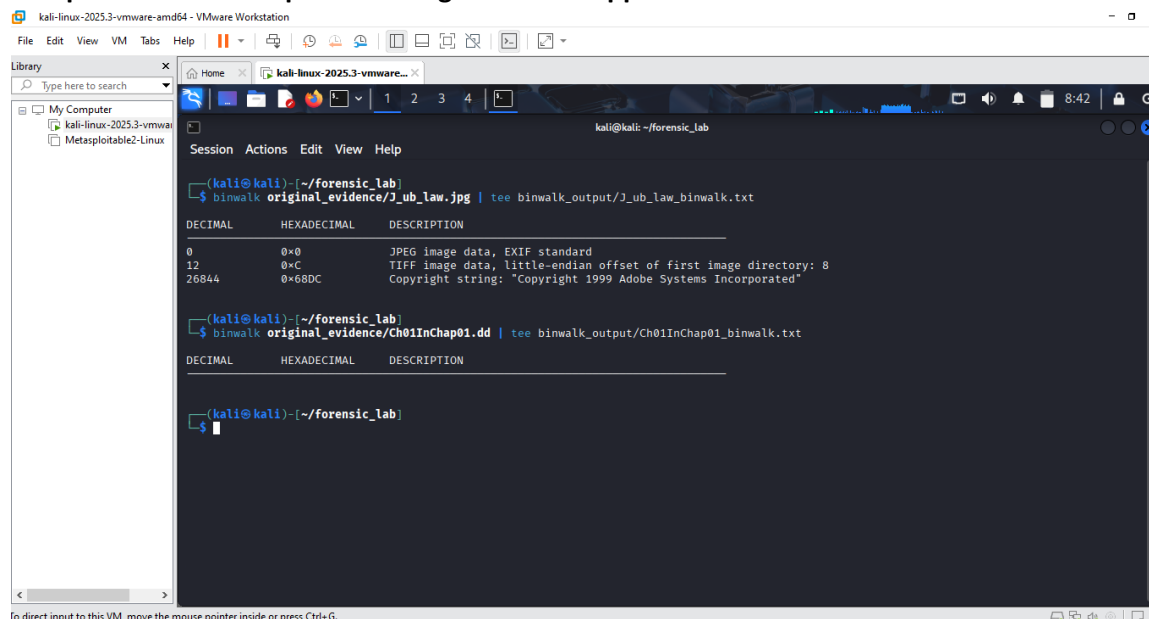


Fig 3.1 Using binwalk for both authorize files.

DECIMAL	HEXADECIMAL	DESCRIPTION
0	0x0	JPEG image data, EXIF standard
12	0xC	TIFF image data, little-endian offset of first image directory: 8
26844	0x68DC	Copyright string: "Copyright 1999 Adobe Systems Incorporated"

File_carving.docx was not available among the supplied evidence or within 120M.7z. In accordance with the laboratory instructions, Binwalk analysis was therefore performed using the other authorised evidence. Binwalk identified JPEG/EXIF data at offset 0, an embedded little-endian TIFF structure at decimal offset 12 (0xC), and an Adobe copyright string at decimal offset 26,844 (0x68DC) within J_ub_law.jpg. No recognised Binwalk signatures were reported for Ch01InChap01.dd. Further Binwalk examination and extraction was performed on the supplied USB forensic image after preparation.

Part E — Prepare the USB image

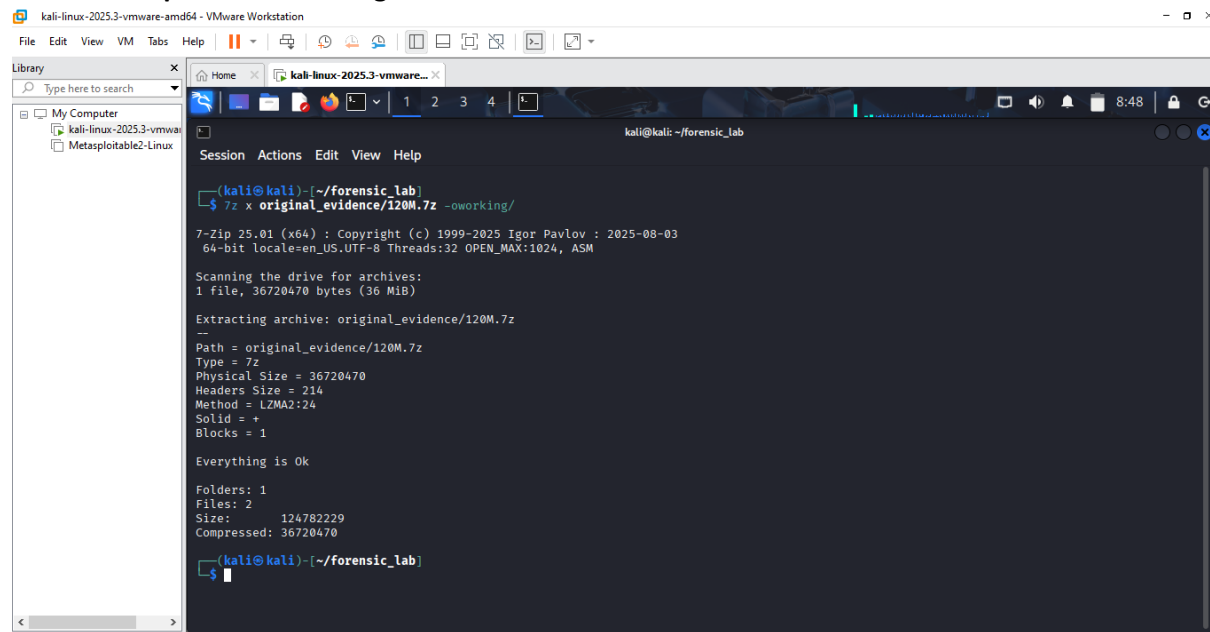


Fig 3.2 Extracting into working and keeping the .7z untouched.

Explanation

The 120M.7z archive was extracted into the working area rather than modifying the preserved archive. This produced the USB forensic image required for the remaining analysis.

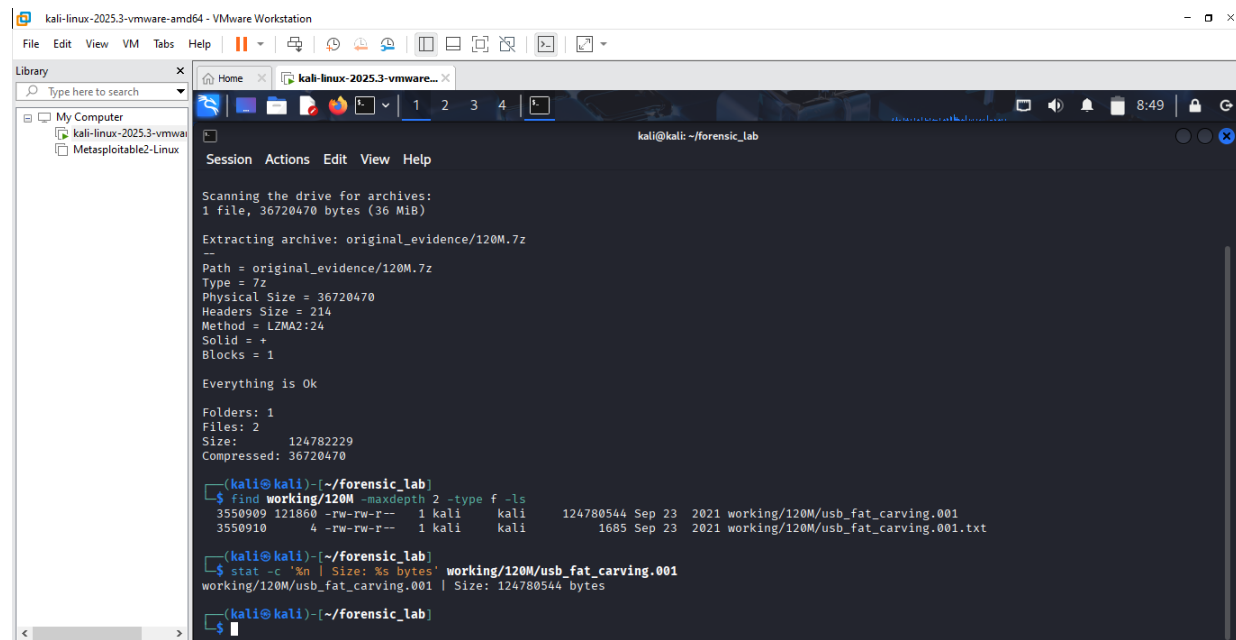


Fig 3.3 Seeing exactly what was extracted and identifying the image type.

Explanation

The extracted contents were listed, showing `usb_fat_carving.001` and its accompanying text file. The image was then examined to identify its type before further analysis.

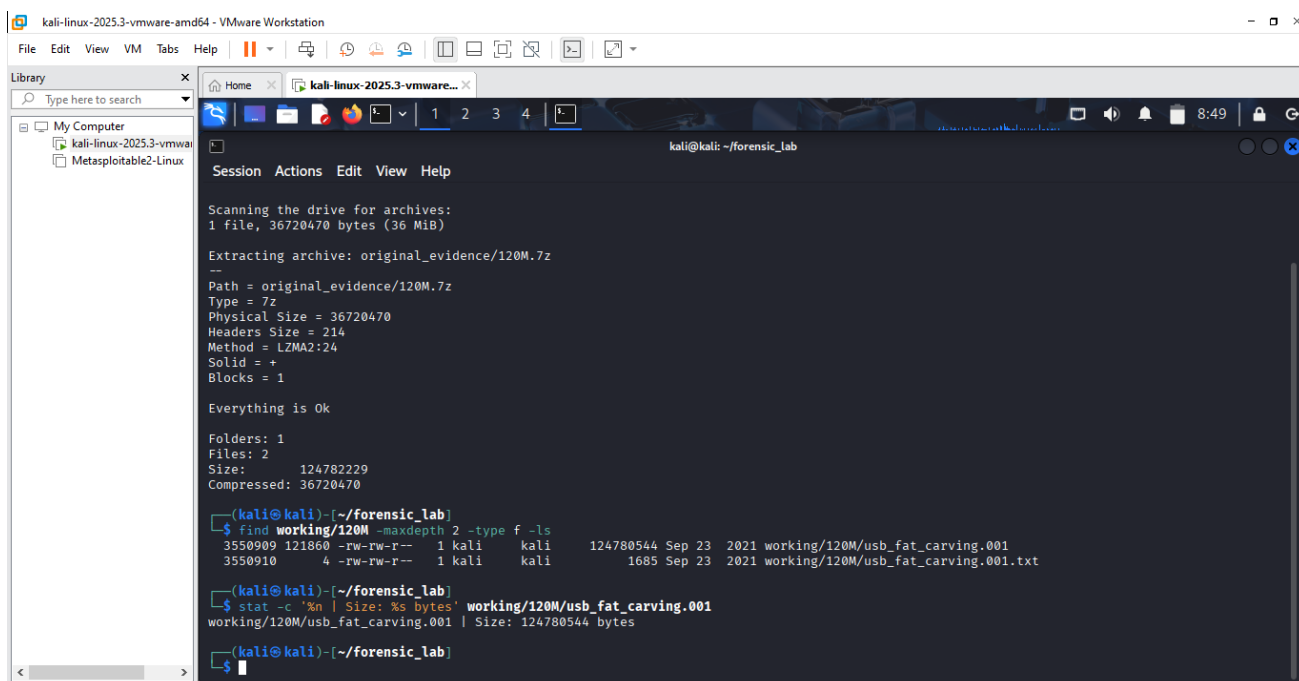


Fig 3.4 Recording the exact size for usb_fat_carving.001.

Explanation

The extracted USB image size was recorded as 124,780,544 bytes. Recording the exact size provided another reference point for evidence identification and integrity.

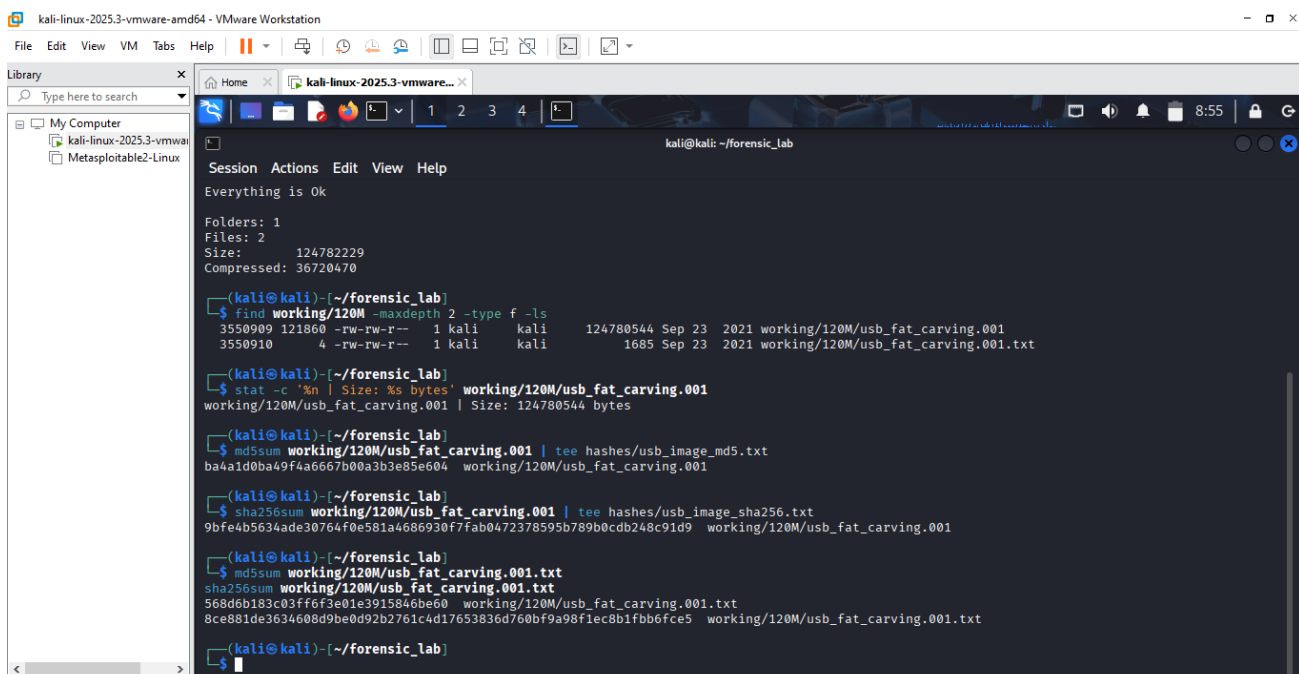
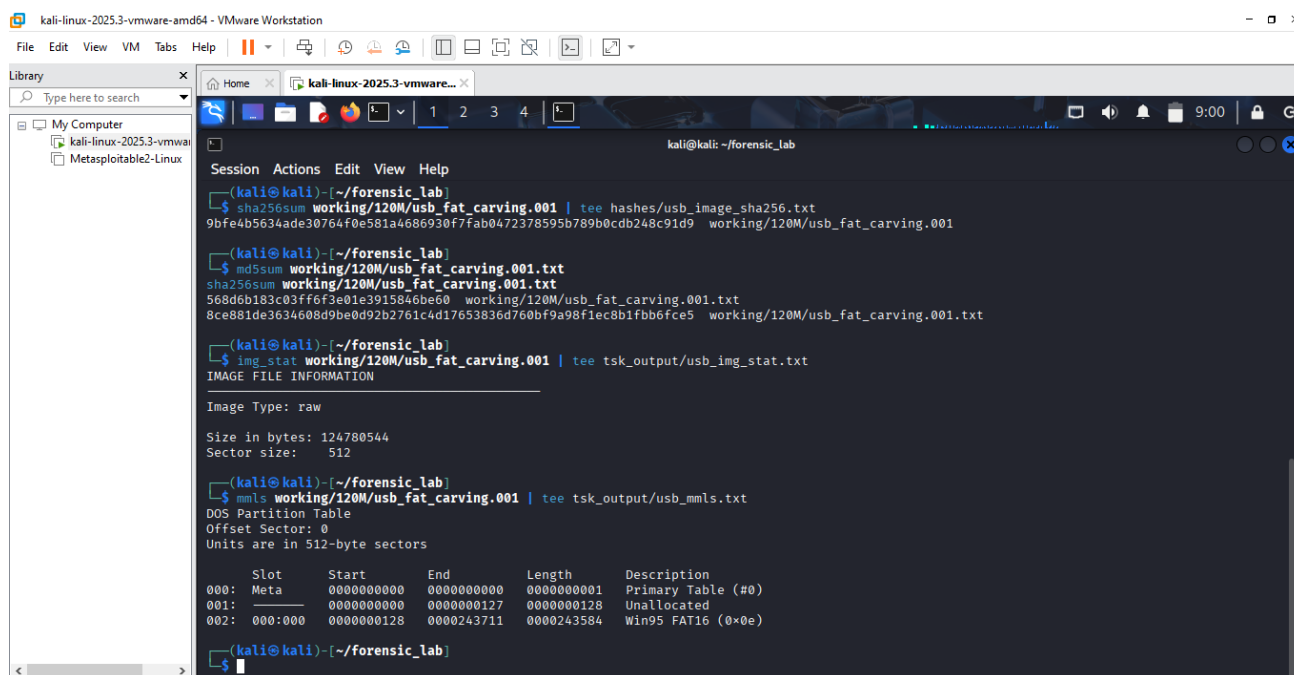


Fig 3.5 Hashing the extracted forensic image with md5sum and sha256sum.

Explanation

MD5 and SHA-256 hashes were generated for usb_fat_carving.001 and its accompanying text file. The USB image MD5 was ba4a1d0ba49f4a6667b00a3b3e85e604, while its SHA-256 was 9bfe4b5634ade30764f0e581a4686930f7fab0472378595b789b0cdb248c91d9.



```
(kali@kali)-[~/forensic_lab]
$ sha256sum working/120M/usb_fat_carving.001 | tee hashes/usb_image_sha256.txt
9bfe4b5634ade30764f0e581a4686930f7fab0472378595b789b0cdb248c91d9  working/120M/usb_fat_carving.001

(kali@kali)-[~/forensic_lab]
$ md5sum working/120M/usb_fat_carving.001.txt
sha256sum working/120M/usb_fat_carving.001.txt
568d6b183c03ff6f3e01e3915846be60  working/120M/usb_fat_carving.001.txt
8ce881de3634608d9be0d92b2761c4d17653836d760bf9a98f1ec8b1fbb6fce5  working/120M/usb_fat_carving.001.txt

(kali@kali)-[~/forensic_lab]
$ img_stat working/120M/usb_fat_carving.001 | tee tsk_output/usb_img_stat.txt
IMAGE FILE INFORMATION
-----
Image Type: raw
Size in bytes: 124780544
Sector size: 512

(kali@kali)-[~/forensic_lab]
$ mmls working/120M/usb_fat_carving.001 | tee tsk_output/usb_mmls.txt
DOS Partition Table
Offset Sector: 0
Units are in 512-byte sectors

    Slot      Start      End      Length    Description
000:  Meta      0000000000  0000000000  0000000001  Primary Table (#0)
001:  _____ 0000000000  0000000127  0000000128  Unallocated
002:  000:000  0000000128  0000243711  0000243584  Win95 FAT16 (0x0e)

(kali@kali)-[~/forensic_lab]
$
```

Fig 3.6 Determining the partition structure.

Explanation

img_stat and mmls were used to examine the image structure. mmls showed 128 unallocated sectors followed by a Win95 FAT16 partition beginning at sector **128**, confirming the offset required for later TSK commands.

mmls output shows:

Slot	Start	End	Length	Description
001	0	127	128	Unallocated
002	128	243711	243584	Win95 FAT16 (0x0e)

USB image preparation

Image size: 124,780,544 bytes

Image type: Raw

Sector size: 512 bytes

MD5: ba4a1d0ba49f4a6667b00a3b3e85e604

SHA-256: 9bfe4b5634ade30764f0e581a4686930f7fab0472378595b789b0cdb248c91d9

```
kali@kali: ~/forensic_lab
002: 000:000 0000000128 0000243711 0000243584 Win95 FAT16 (0x0e)

(kali@kali)~/forensic_lab
$ fsstat -o 128 working/120M/usb_fat_carving.001 | tee tsk_output/usb_fsstat.txt
FILE SYSTEM INFORMATION

File System Type: FAT16

OEM Name: MSDOS5.0
Volume ID: 0x94105672
Volume Label (Boot Sector): NO NAME
Volume Label (Root Directory): USB
File System Type Label: FAT16

Sectors before file system: 128

File System Layout (in sectors)
Total Range: 0 - 243583
* Reserved: 0 - 3
** Boot Sector: 0
* FAT 0: 4 - 241
* FAT 1: 242 - 479
* Data Area: 480 - 243583
** Root Directory: 480 - 511
** Cluster Area: 512 - 243583

METADATA INFORMATION

Range: 2 - 3889670
Root Directory: 2

CONTENT INFORMATION
```

Fig 3.7 Inspecting the FAT16 filesystem at the confirmed offset.

Explanation

fsstat was executed using offset 128 and identified the file system as FAT16 with the volume label USB. The output also began to show the FAT and data-area layout.

```
(kali@kali)~/forensic_lab
$ fsstat -o 128 working/120M/usb_fat_carving.001 | tee tsk_output/usb_fsstat.txt
FILE SYSTEM INFORMATION

File System Type: FAT16

OEM Name: MSDOS5.0
Volume ID: 0x94105672
Volume Label (Boot Sector): NO NAME
Volume Label (Root Directory): USB
File System Type Label: FAT16

Sectors before file system: 128

File System Layout (in sectors)
Total Range: 0 - 243583
* Reserved: 0 - 3
** Boot Sector: 0
* FAT 0: 4 - 241
* FAT 1: 242 - 479
* Data Area: 480 - 243583
** Root Directory: 480 - 511
** Cluster Area: 512 - 243583

METADATA INFORMATION

Range: 2 - 3889670
Root Directory: 2

CONTENT INFORMATION

Sector Size: 512
Cluster Size: 2048
Total Cluster Range: 2 - 60769

FAT CONTENTS (in sectors)

512-515 (4) → EOF
516-519 (4) → EOF
520-523 (4) → EOF
70976-71211 (236) → EOF
71212-71215 (4) → EOF

(kali@kali)~/forensic_lab
$
```

Fig 3.8 Inspecting the FAT16 filesystem at the confirmed offset page 1.

Explanation

The remaining fsstat output shows a 512-byte sector size, 2048-byte cluster size, FAT locations, root directory, data area, and cluster area. These values describe the structure of the USB filesystem used in the later recovery work.

Filesystem: FAT16

OEM: MSDOS5.0

Volume label: USB

Sectors before filesystem: 128

Sector size: 512 bytes

Cluster size: 2048 bytes

Filesystem sector range: 0–243583

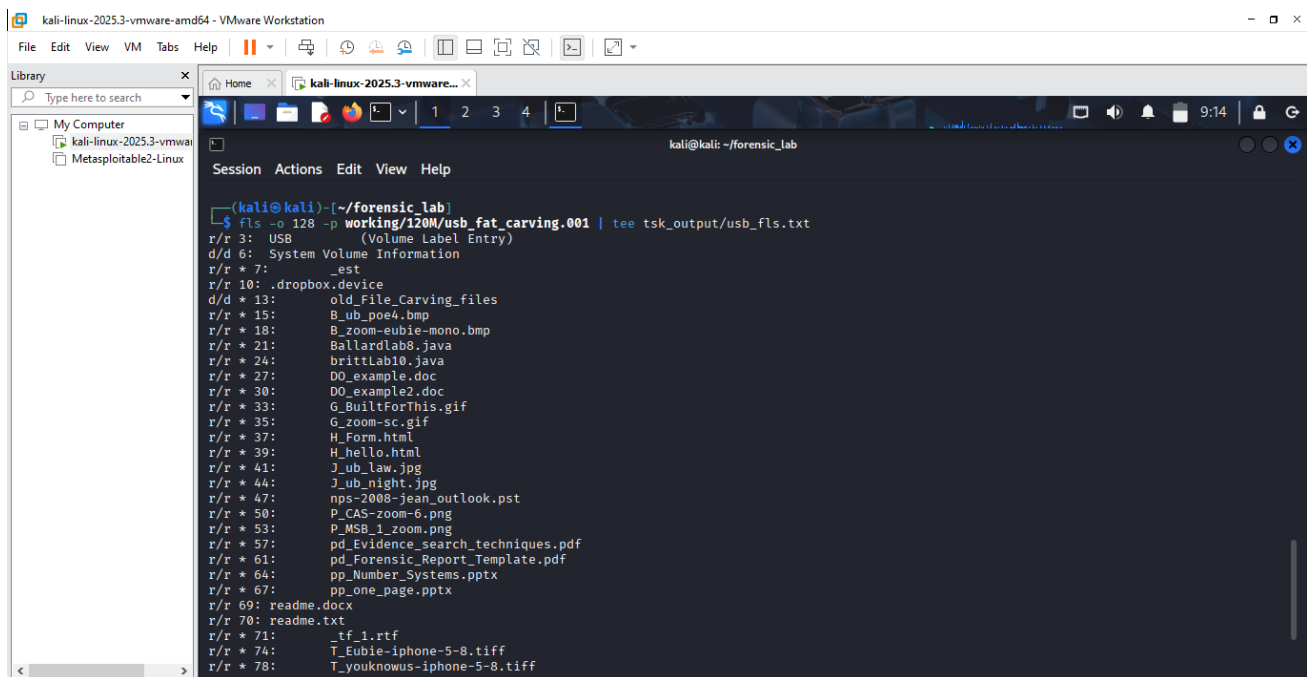
FAT 0: sectors 4–241

FAT 1: sectors 242–479

Data area: sectors 480–243583

Root directory: sectors 480–511

Cluster area: sectors 512–243583



```
kali@kali: ~/forensic_lab
$ fls -o 128 -p working/120M/usb_fat_carving.001 | tee tsk_output/usb_fls.txt
r/r 3: USB
d/d 6: System Volume Information
r/r * 7: _est
r/r 10: .dropbox.device
d/d * 13: old_File_Carving_files
r/r * 15: B_ub_poe4.bmp
r/r * 18: B_zoom-eubie-mono.bmp
r/r * 21: Ballardlab8.java
r/r * 24: brittLab10.java
r/r * 27: DO_example.doc
r/r * 30: DO_example2.doc
r/r * 33: G_BuiltForThis.gif
r/r * 35: G_zoom-sc.gif
r/r * 37: H_Form.html
r/r * 39: H_hello.html
r/r * 41: J_ub_law.jpg
r/r * 44: J_ub_night.jpg
r/r * 47: nps-2008-jean_outlook.pst
r/r * 50: P_CAS-zoom-6.png
r/r * 53: P_MSB_1_zoom.png
r/r * 57: pd_Evidence_search_techniques.pdf
r/r * 61: pd_Forensic_Report_Template.pdf
r/r * 64: pp_Number_Systems.pptx
r/r * 67: pp_one_page.pptx
r/r 69: readme.docx
r/r 70: readme.txt
r/r * 71: _tf_1.rtf
r/r * 74: T_Eubie-iphone-5-8.tiff
r/r * 78: T_youknowus-iphone-5-8.tiff
```

Fig 3.9 listing the files.

Explanation

fls was run at the confirmed offset to display files and directories found on the USB image. The listing included a range of document, image, audio, archive, and other file types.

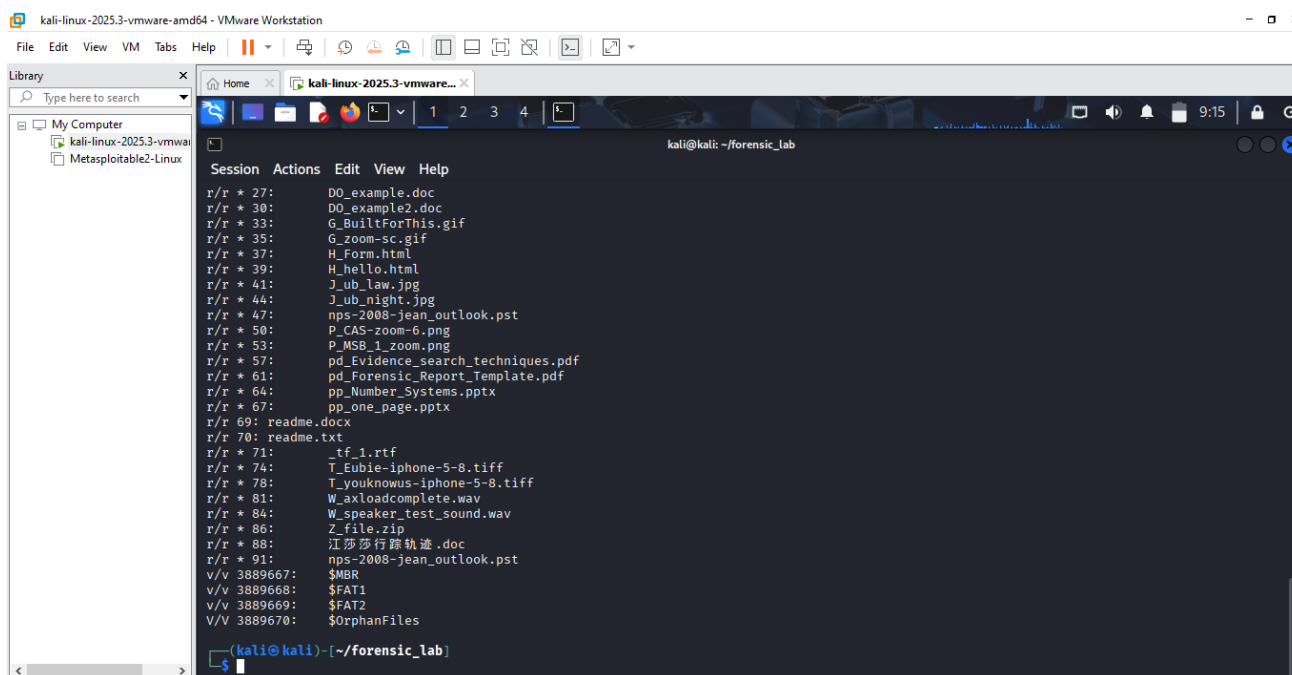


Fig 4.0 listing the files page 1.

Explanation

This screenshot continues the fls output and records additional files and special FAT entries. Together with the previous screenshot, it provides a more complete view of the accessible file-system contents.

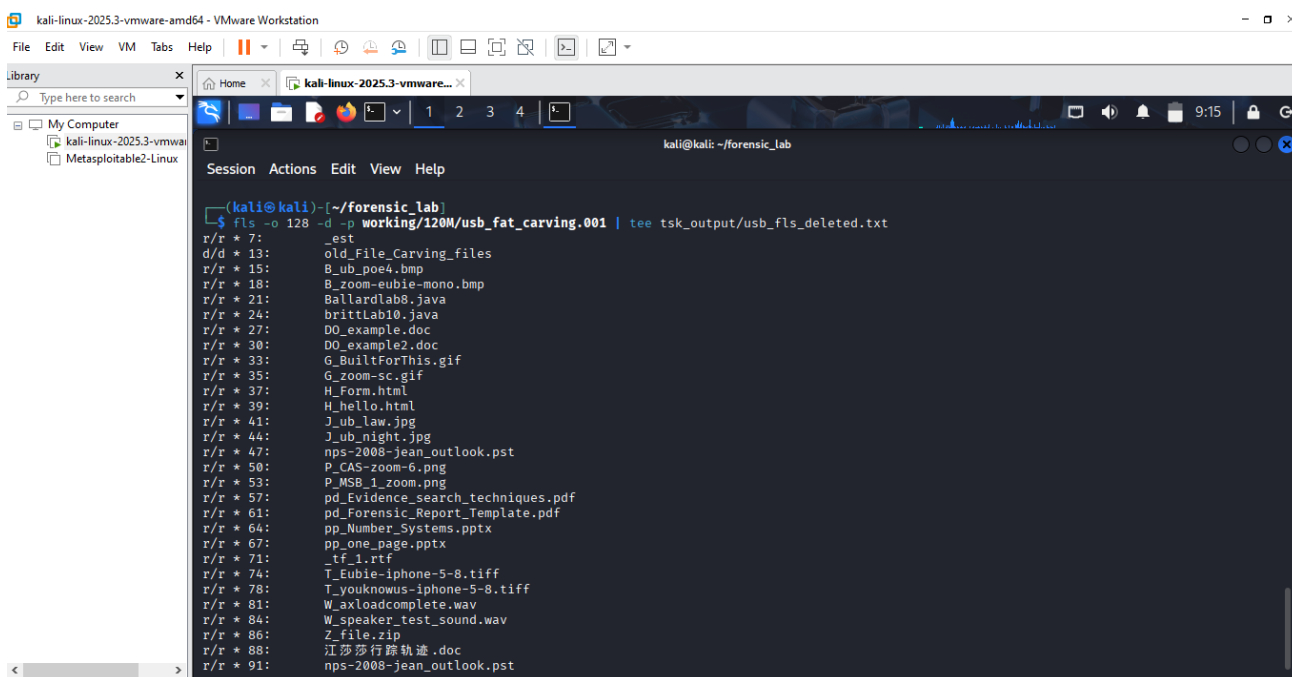


Fig 4.1 Showing deleted entries.

Explanation

fls was run specifically to display deleted entries. The results showed several deleted files, including JPEGs that could later be recovered through signature-based carving.

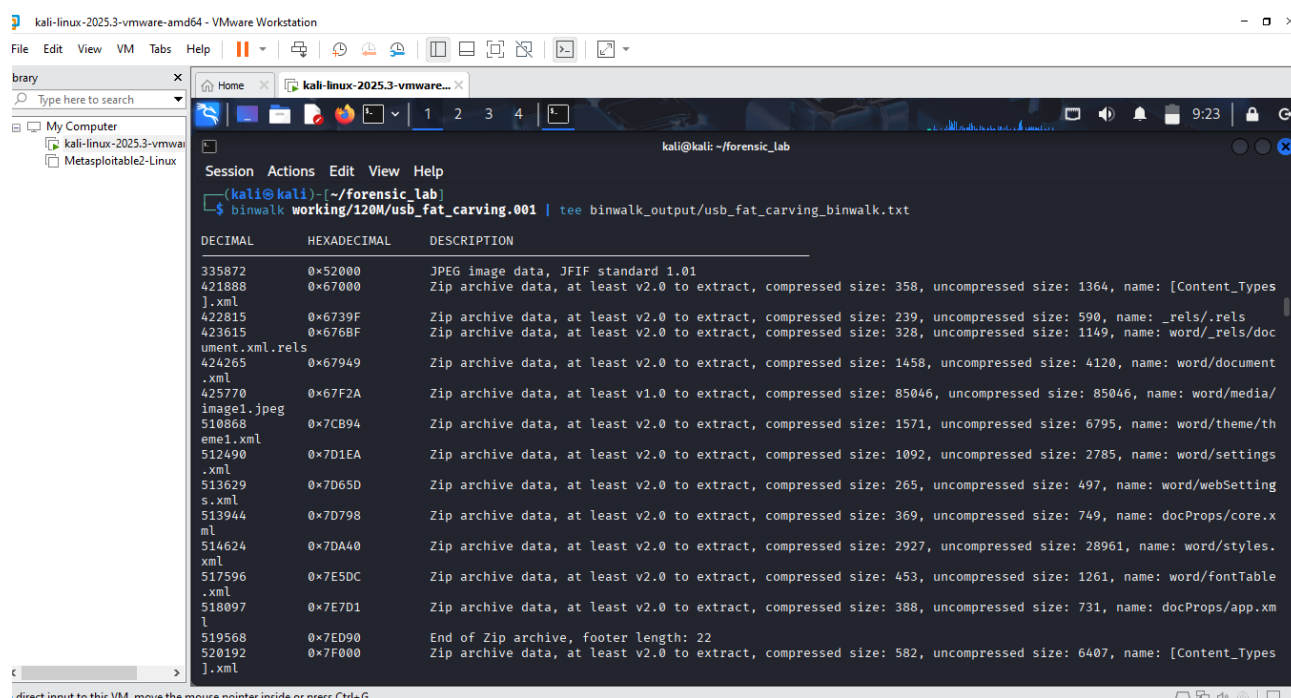


Fig 4.2 Examining fat_carving.001 using binwalk.

Explanation

Binwalk identified numerous embedded signatures in the USB image, including JPEG and ZIP/Office-related structures. The output also showed internal Word OOXML components such as [Content_Types].xml and word/document.xml.

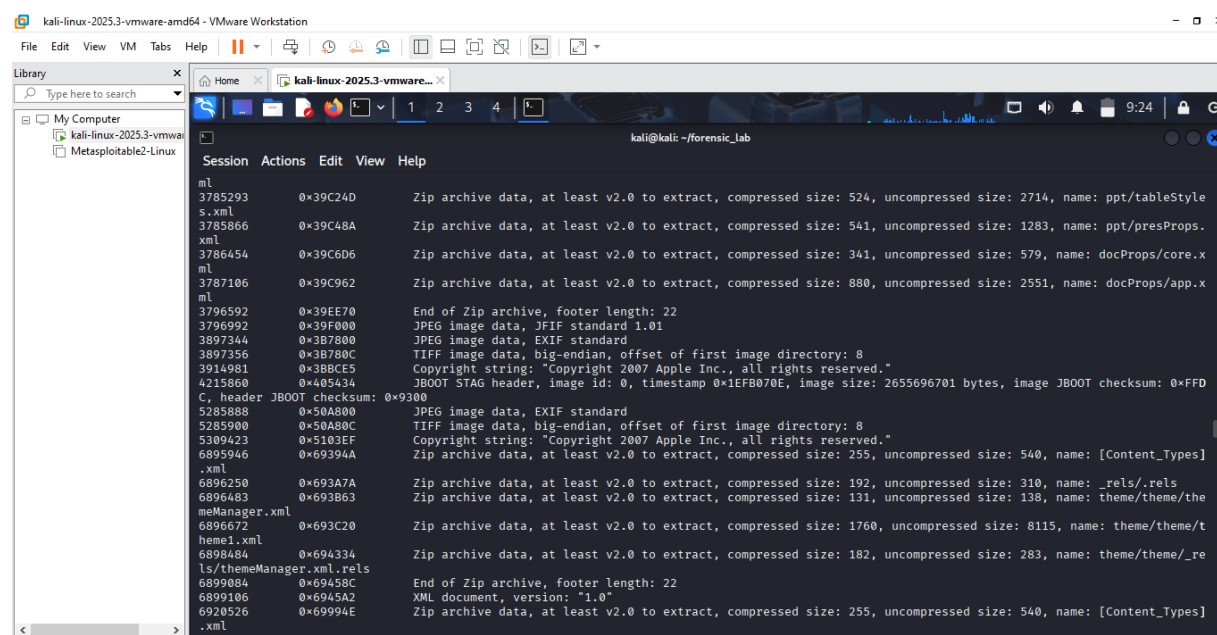


Fig 4.3 Examining fat_carving.001 using binwalk page 1.

Explanation

This continuation shows further signatures and embedded structures at additional offsets. JPEG, TIFF/EXIF,

ZIP, XML, and copyright-related data were among the items detected.

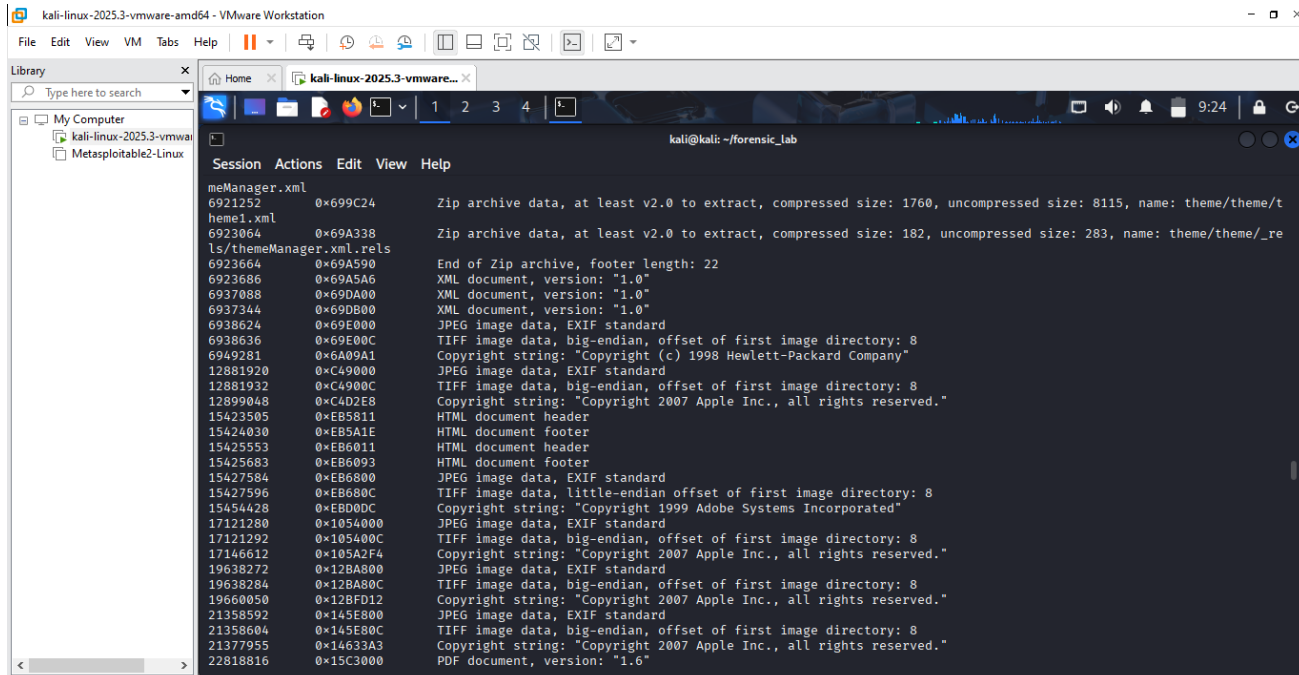


Fig 4.4 Examining fat_carving.001 using binwalk page 2.

Explanation

More embedded content was identified deeper in the image, including image structures, HTML headers and footers, PDF content, and additional metadata-related strings. This demonstrated that many different file signatures were present within the raw image.

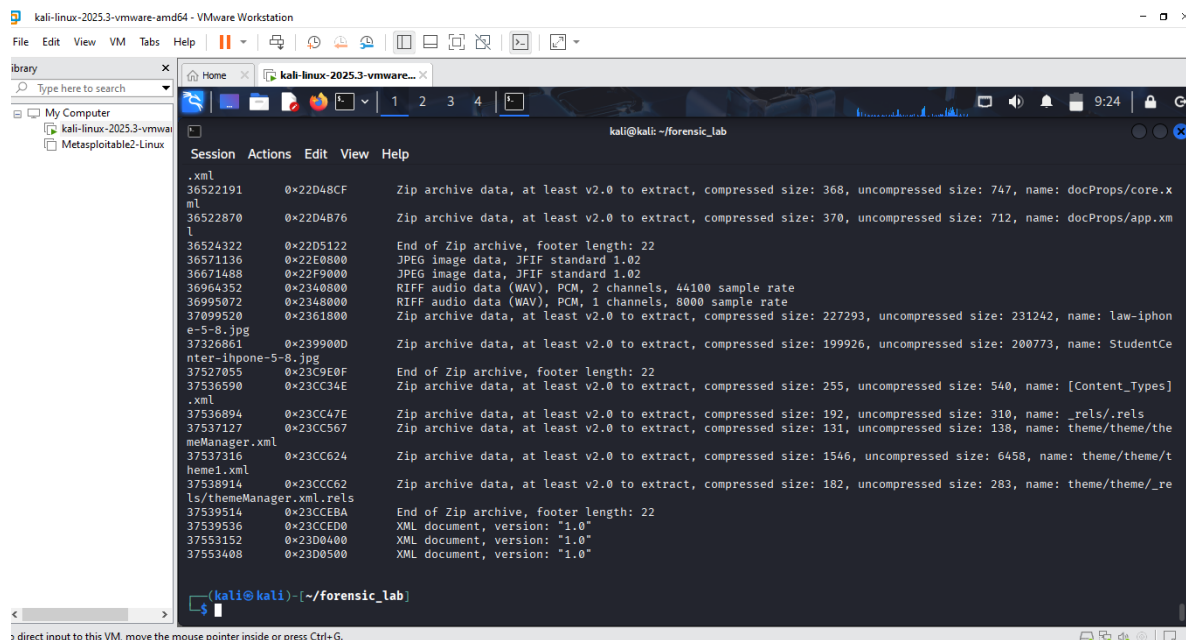
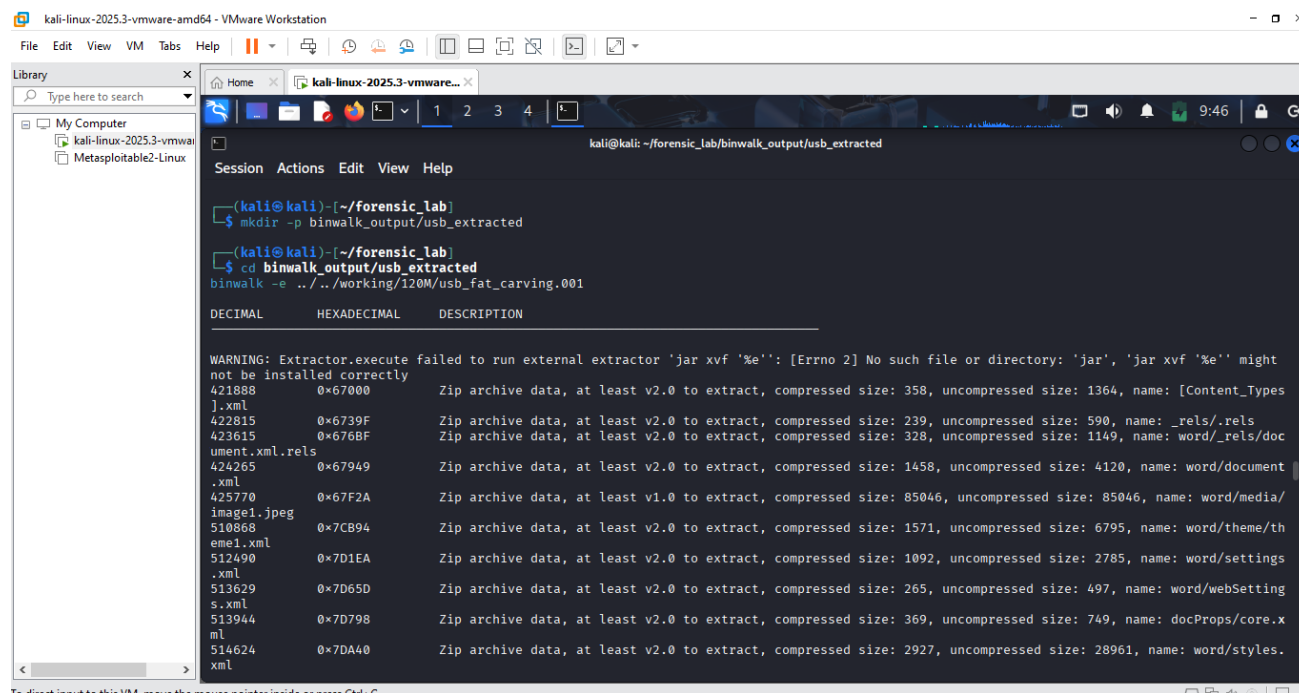


Fig 4.5 Examining fat_carving.001 using binwalk page 3.

Explanation

The final part of the Binwalk scan shows still more ZIP, image, audio, and XML-related signatures. The broad range of results provided useful offsets for both automatic and manual extraction attempts.



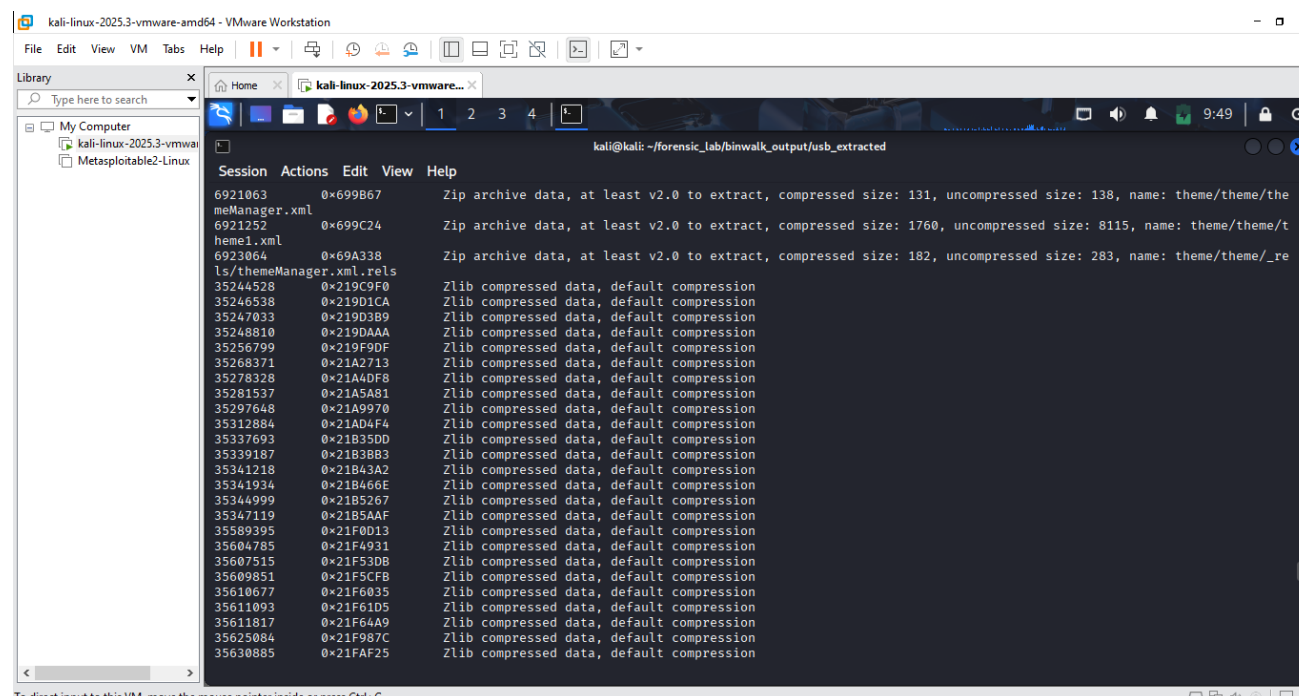
```
kali@kali: ~/forensic_lab
$ mkdir -p binwalk_output/usb_extracted
$ cd binwalk_output/usb_extracted
binwalk -e ../working/120M/usb_fat_carving.001
```

DECIMAL	HEXADECIMAL	DESCRIPTION
WARNING: Extractor.execute failed to run external extractor 'jar xvf %e': [Errno 2] No such file or directory: 'jar', 'jar xvf %e' might not be installed correctly		
421888	0x67000	Zip archive data, at least v2.0 to extract, compressed size: 358, uncompressed size: 1364, name: [Content_Types].xml
422815	0x6739F	Zip archive data, at least v2.0 to extract, compressed size: 239, uncompressed size: 590, name: _rels/.rels
423615	0x676BF	Zip archive data, at least v2.0 to extract, compressed size: 328, uncompressed size: 1149, name: word/_rels/document.xml.rels
424265	0x67949	Zip archive data, at least v2.0 to extract, compressed size: 1458, uncompressed size: 4120, name: word/document.xml
425770	0x67F2A	Zip archive data, at least v1.0 to extract, compressed size: 85046, uncompressed size: 85046, name: word/media/image1.jpeg
510868	0x7CB94	Zip archive data, at least v2.0 to extract, compressed size: 1571, uncompressed size: 6795, name: word/theme/theme1.xml
512490	0x7D1EA	Zip archive data, at least v2.0 to extract, compressed size: 1092, uncompressed size: 2785, name: word/settings.xml
513629	0x7D65D	Zip archive data, at least v2.0 to extract, compressed size: 265, uncompressed size: 497, name: word/webSettings.xml
513944	0x7D798	Zip archive data, at least v2.0 to extract, compressed size: 369, uncompressed size: 749, name: docProps/core.xml
514624	0x7DA40	Zip archive data, at least v2.0 to extract, compressed size: 2927, uncompressed size: 28961, name: word/styles.xml

Fig 4.6 Automatic Binwalk extraction.

Explanation

Binwalk automatic extraction was run against the USB image. The process began extracting recognised structures but also reported that the external jar extractor was unavailable.



```
kali@kali: ~/forensic_lab/binwalk_output/usb_extracted
```

6921063	0x699B67	Zip archive data, at least v2.0 to extract, compressed size: 131, uncompressed size: 138, name: theme/theme/themeManager.xml
6921252	0x699C24	Zip archive data, at least v2.0 to extract, compressed size: 1760, uncompressed size: 8115, name: theme/theme/theme1.xml
6923864	0x69A338	Zip archive data, at least v2.0 to extract, compressed size: 182, uncompressed size: 283, name: theme/theme/_rels/themeManager.xml.rels
35244528	0x219C9F0	Zlib compressed data, default compression
35244538	0x219D1CA	Zlib compressed data, default compression
35247033	0x219D3B9	Zlib compressed data, default compression
35248810	0x219DAAA	Zlib compressed data, default compression
35256799	0x219F9DF	Zlib compressed data, default compression
35268371	0x21A2713	Zlib compressed data, default compression
35278328	0x21A4DF8	Zlib compressed data, default compression
35281537	0x21A5A81	Zlib compressed data, default compression
35297648	0x21A9970	Zlib compressed data, default compression
35312884	0x21AD4F4	Zlib compressed data, default compression
35337693	0x21B35DD	Zlib compressed data, default compression
35339187	0x21B3BB3	Zlib compressed data, default compression
35341218	0x21B43A2	Zlib compressed data, default compression
35341934	0x21B466E	Zlib compressed data, default compression
35344999	0x21B5267	Zlib compressed data, default compression
35347119	0x21B5AAF	Zlib compressed data, default compression
35589395	0x21F0D13	Zlib compressed data, default compression
35604785	0x21F4931	Zlib compressed data, default compression
35607515	0x21F53DB	Zlib compressed data, default compression
35609851	0x21F6CFB	Zlib compressed data, default compression
35610677	0x21F6D35	Zlib compressed data, default compression
35611093	0x21F6D05	Zlib compressed data, default compression
35611817	0x21F64A9	Zlib compressed data, default compression
35625084	0x21F987C	Zlib compressed data, default compression
35630885	0x21FAF25	Zlib compressed data, default compression

Fig 4.7 Automatic Binwalk extraction page 1.

Explanation

The automatic process continued to identify and extract compressed structures. Numerous zlib-related objects were detected during this stage.

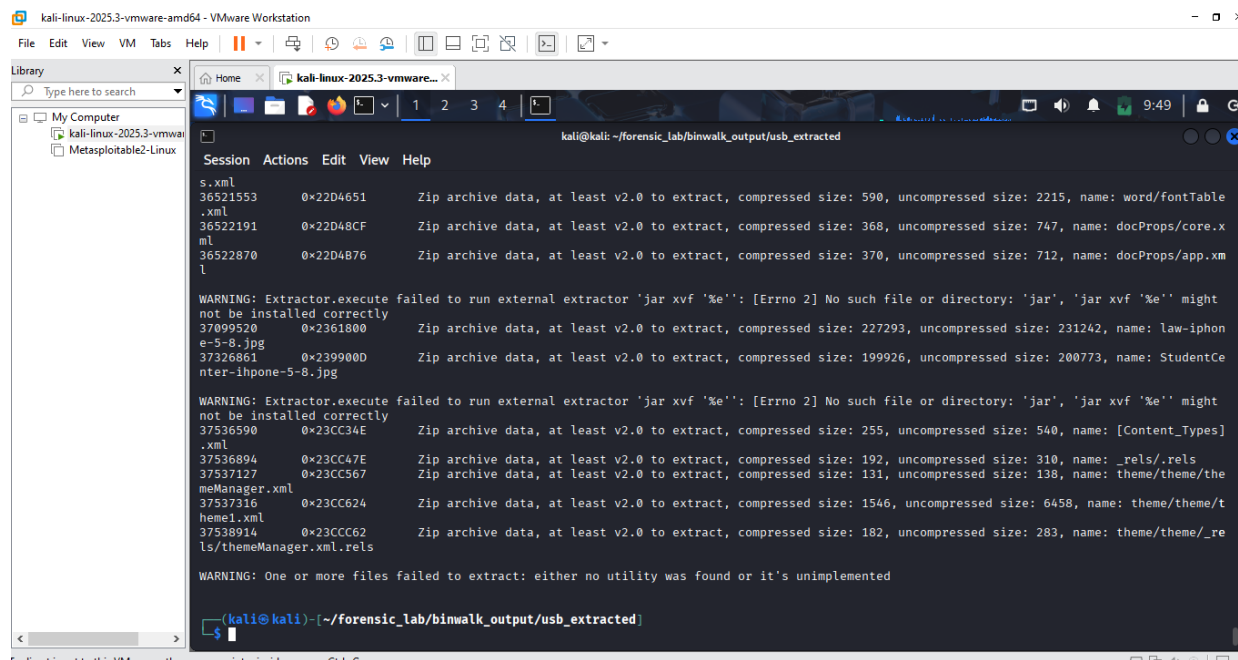


Fig 4.8 Automatic Binwalk extraction page 2.

Explanation

This screenshot records additional extractor warnings, including instances where the required external utility could not be found. Binwalk therefore completed only the extraction operations supported by the available utilities.

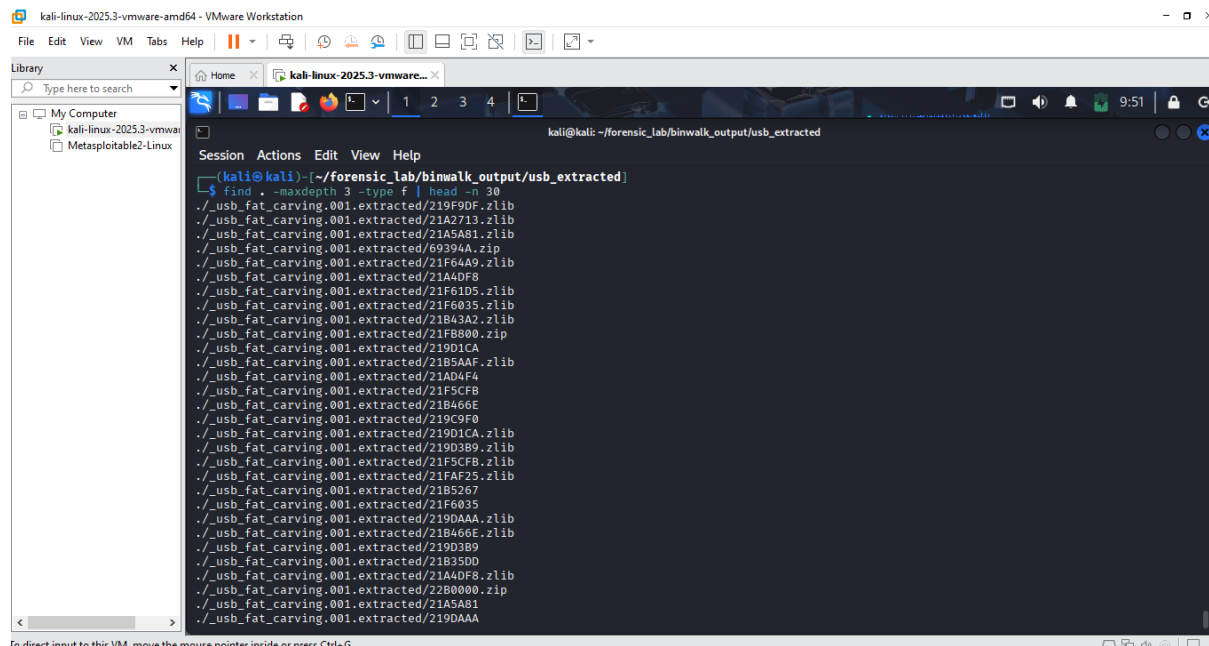


Fig 4.9 Automatic Binwalk extraction page 3.

Explanation

This continuation shows more of the automatically generated extraction files. It documents the large number of derived objects created during the Binwalk process.

```
kali@kali: ~/forensic_lab/binwalk_output/usb_extracted
$ find . -maxdepth 3 -type f | head -n 30
./usb_fat_carving.001.extracted/219F9DF.zlib
./usb_fat_carving.001.extracted/21A2713.zlib
./usb_fat_carving.001.extracted/21A5A81.zlib
./usb_fat_carving.001.extracted/69394A.zip
./usb_fat_carving.001.extracted/21F64A9.zlib
./usb_fat_carving.001.extracted/21A4DF8
./usb_fat_carving.001.extracted/21F61D5.zlib
./usb_fat_carving.001.extracted/21F6035.zlib
./usb_fat_carving.001.extracted/21B43A2.zlib
./usb_fat_carving.001.extracted/21F8800.zip
./usb_fat_carving.001.extracted/219D1CA
./usb_fat_carving.001.extracted/21B5AAF.zlib
./usb_fat_carving.001.extracted/21AD4F4
./usb_fat_carving.001.extracted/21F5CFB
./usb_fat_carving.001.extracted/21B466E
./usb_fat_carving.001.extracted/219C9F0
./usb_fat_carving.001.extracted/219D1CA.zlib
./usb_fat_carving.001.extracted/219D3B9.zlib
./usb_fat_carving.001.extracted/21F5CFB.zlib
./usb_fat_carving.001.extracted/21FAF25.zlib
./usb_fat_carving.001.extracted/21B5267
./usb_fat_carving.001.extracted/21F6035
./usb_fat_carving.001.extracted/219DAAA.zlib
./usb_fat_carving.001.extracted/21B466E.zlib
./usb_fat_carving.001.extracted/219D3B9
./usb_fat_carving.001.extracted/21B35D0
./usb_fat_carving.001.extracted/21A4DF8.zlib
./usb_fat_carving.001.extracted/22B0000.zip
./usb_fat_carving.001.extracted/21A5A81
./usb_fat_carving.001.extracted/219DAAA
```

Fig 5.0 Automatic Binwalk extraction page 4.

Explanation

This continuation shows more of the automatically generated extraction files. It documents the large number of derived objects created during the Binwalk process.

```
kali@kali: ~/forensic_lab/binwalk_output/usb_extracted
$ find . -maxdepth 3 -type f | head -n 30
./usb_fat_carving.001.extracted/21A2713.zlib
./usb_fat_carving.001.extracted/21A5A81.zlib
./usb_fat_carving.001.extracted/69394A.zip
./usb_fat_carving.001.extracted/21F64A9.zlib
./usb_fat_carving.001.extracted/21A4DF8
./usb_fat_carving.001.extracted/21F61D5.zlib
./usb_fat_carving.001.extracted/21F6035.zlib
./usb_fat_carving.001.extracted/21B43A2.zlib
./usb_fat_carving.001.extracted/21F8800.zip
./usb_fat_carving.001.extracted/219D1CA
./usb_fat_carving.001.extracted/21B5AAF.zlib
./usb_fat_carving.001.extracted/21AD4F4
./usb_fat_carving.001.extracted/21F5CFB
./usb_fat_carving.001.extracted/21B466E
./usb_fat_carving.001.extracted/219C9F0
./usb_fat_carving.001.extracted/219D1CA.zlib
./usb_fat_carving.001.extracted/219D3B9.zlib
./usb_fat_carving.001.extracted/21F5CFB.zlib
./usb_fat_carving.001.extracted/21FAF25.zlib
./usb_fat_carving.001.extracted/21B5267
./usb_fat_carving.001.extracted/21F6035
./usb_fat_carving.001.extracted/219DAAA.zlib
./usb_fat_carving.001.extracted/21B466E.zlib
./usb_fat_carving.001.extracted/219D3B9
./usb_fat_carving.001.extracted/21B35D0
./usb_fat_carving.001.extracted/21A4DF8.zlib
./usb_fat_carving.001.extracted/22B0000.zip
./usb_fat_carving.001.extracted/21A5A81
./usb_fat_carving.001.extracted/219DAAA
```

Fig 5.0.1 Automatic Binwalk extraction page 5.

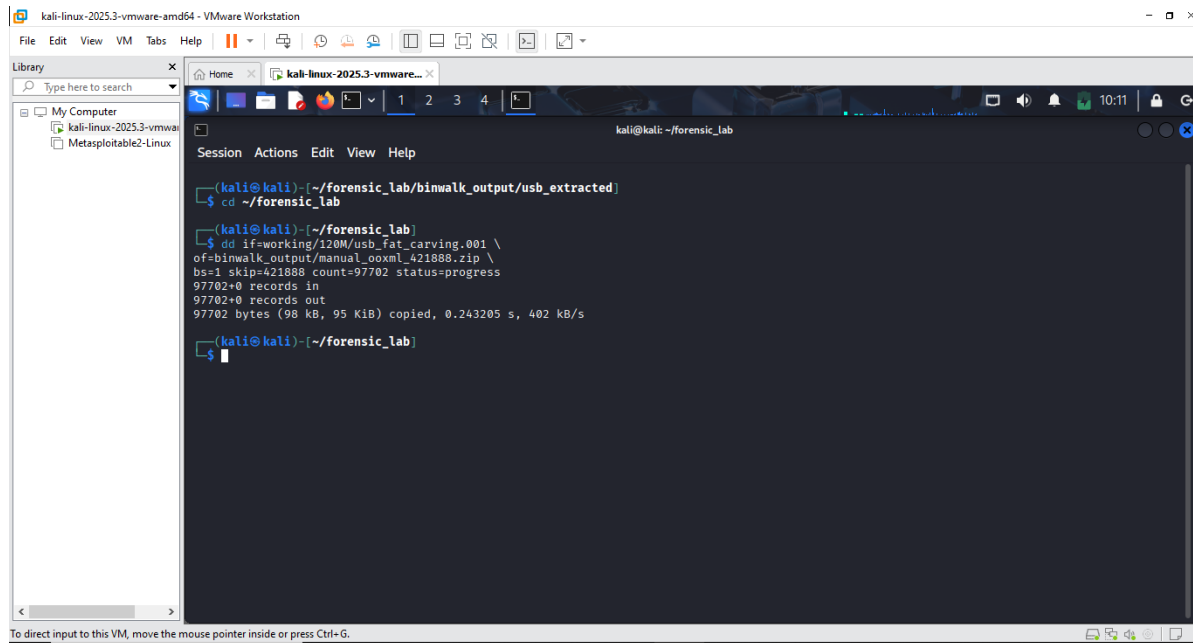
Demonstrating a manual Binwalk-guided extraction

start = 421888

end-of-ZIP record = 519568

footer length = 22

$519568 + 22 - 421888 = 97,702$ bytes

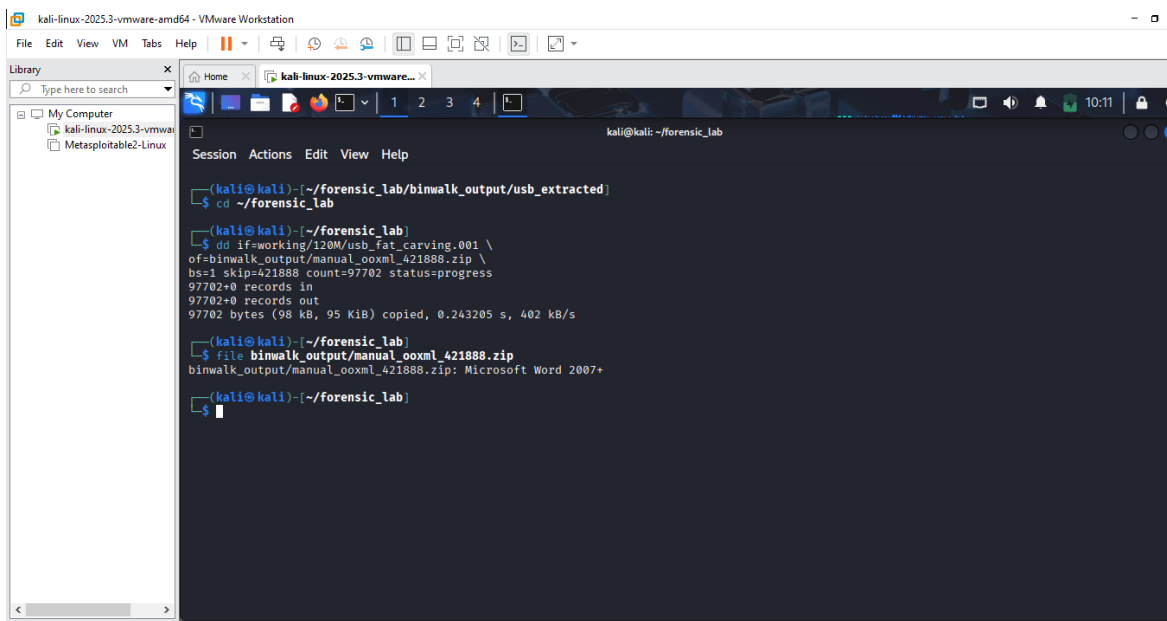


```
kali@kali:~/forensic_lab$ cd ~/forensic_lab/binwalk_output/usb_extracted
kali@kali:~/forensic_lab$ dd if=working/120M/usb_fat_carving.001 \
of=binwalk_output/manual_ooxml_421888.zip \
bs=1 skip=421888 count=97702 status=progress
97702+0 records in
97702+0 records out
97702 bytes (98 kB, 95 KiB) copied, 0.243205 s, 402 kB/s
kali@kali:~/forensic_lab$
```

Fig 5.1 identify and extracting bytes.

Explanation

Using the start and end information identified through Binwalk, dd was used to extract exactly 97,702 bytes beginning at decimal offset 421888. This created the manually recovered OOXML candidate.

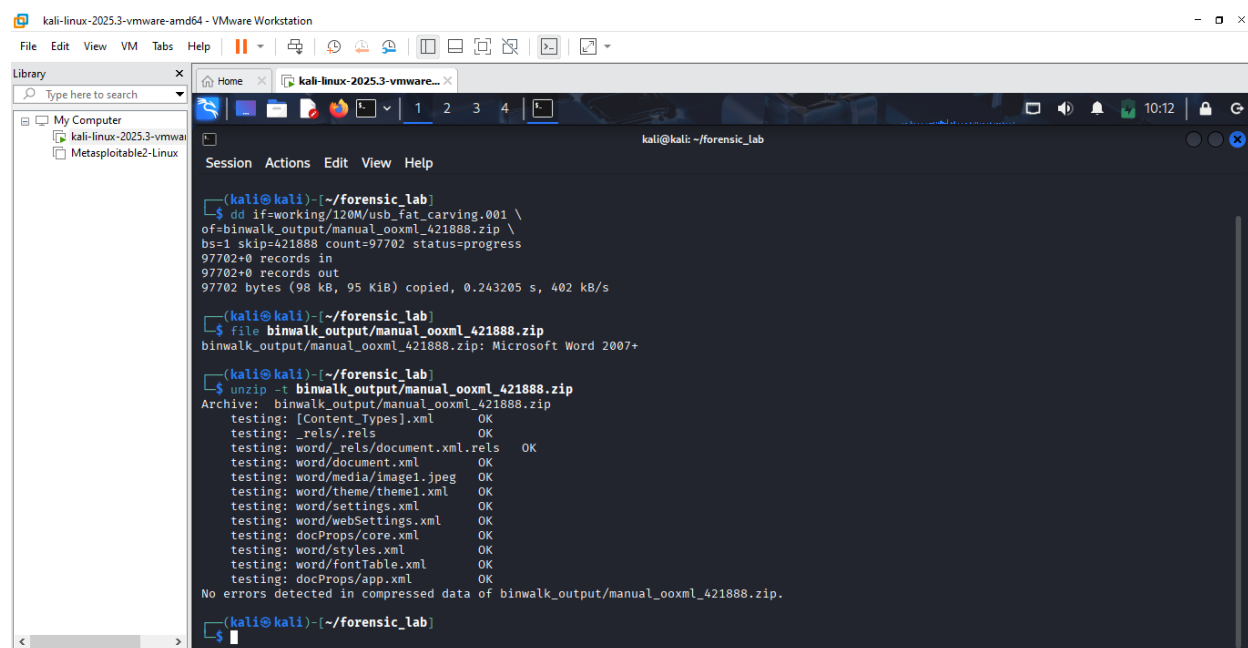


```
kali@kali:~/forensic_lab$ cd ~/forensic_lab/binwalk_output/usb_extracted
kali@kali:~/forensic_lab$ dd if=working/120M/usb_fat_carving.001 \
of=binwalk_output/manual_ooxml_421888.zip \
bs=1 skip=421888 count=97702 status=progress
97702+0 records in
97702+0 records out
97702 bytes (98 kB, 95 KiB) copied, 0.243205 s, 402 kB/s
kali@kali:~/forensic_lab$ file binwalk_output/manual_ooxml_421888.zip
binwalk_output/manual_ooxml_421888.zip: Microsoft Word 2007+
kali@kali:~/forensic_lab$
```

Fig 5.2 Testing the ZIP structure.

Explanation

The recovered object was examined with the file command and recognised as Microsoft Word 2007+. This provided an initial indication that the manually extracted data represented a valid Office Open XML document structure.



```
(kali@kali)~[/forensic_lab]
$ dd if=working/120M/usb_fat_carving.001 \
of=binwalk_output/manual_ooxml_421888.zip \
bs=1 skip=421888 count=97702 status=progress
97702+0 records in
97702+0 records out
97702 bytes (98 kB, 95 KiB) copied, 0.243205 s, 402 kB/s

(kali@kali)~[/forensic_lab]
$ file binwalk_output/manual_ooxml_421888.zip
binwalk_output/manual_ooxml_421888.zip: Microsoft Word 2007+

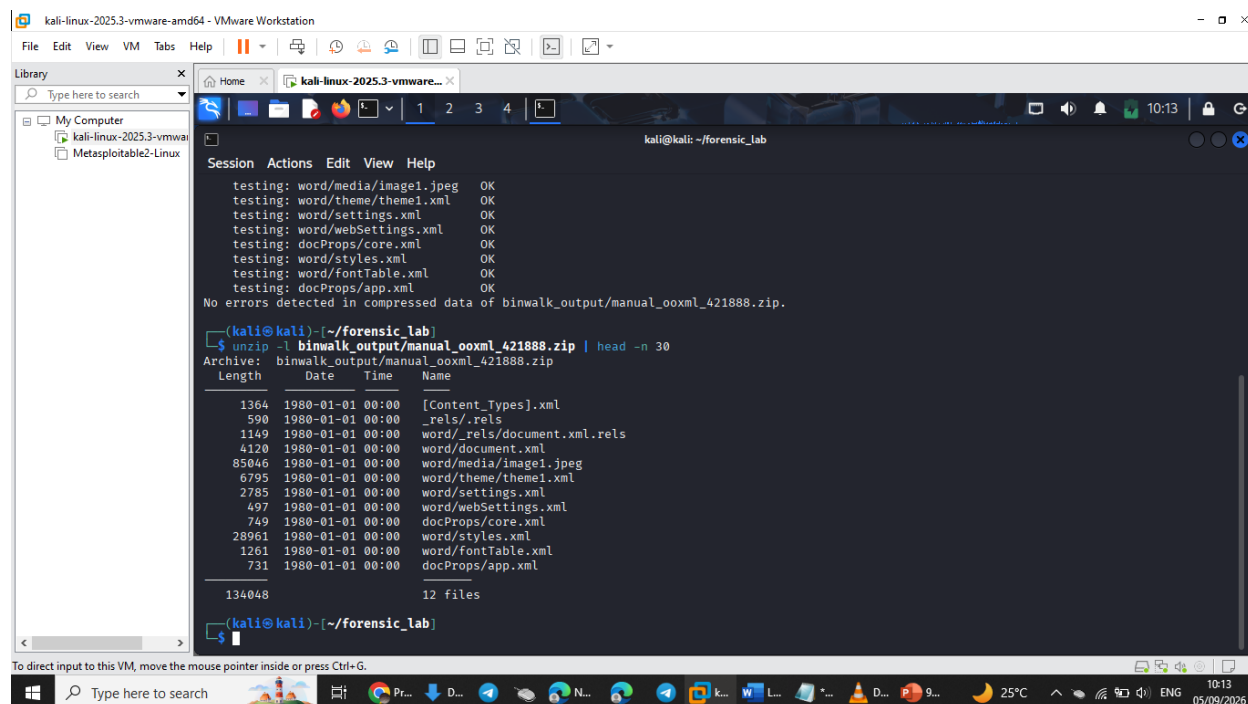
(kali@kali)~[/forensic_lab]
$ unzip -t binwalk_output/manual_ooxml_421888.zip
Archive:  binwalk_output/manual_ooxml_421888.zip
testing:  [Content.Types].xml      OK
testing:  _rels/.rels             OK
testing:  word/_rels/document.xml.rels  OK
testing:  word/document.xml       OK
testing:  word/media/image1.jpeg   OK
testing:  word/theme/theme1.xml    OK
testing:  word/settings.xml        OK
testing:  word/webSettings.xml     OK
testing:  docProps/core.xml        OK
testing:  word/styles.xml          OK
testing:  word/fontTable.xml       OK
testing:  docProps/app.xml         OK
No errors detected in compressed data of binwalk_output/manual_ooxml_421888.zip.

(kali@kali)~[/forensic_lab]
```

Fig 5.3 Inspecting its contents.

Explanation

unzip -t tested the manually recovered OOXML archive and reported no errors in the compressed data. This demonstrated that the extracted ZIP structure was internally valid.



```
(kali@kali)~[/forensic_lab]
testing: word/media/image1.jpeg      OK
testing: word/theme/theme1.xml       OK
testing: word/settings.xml           OK
testing: word/webSettings.xml        OK
testing: docProps/core.xml           OK
testing: word/styles.xml             OK
testing: word/fontTable.xml          OK
testing: docProps/app.xml            OK
No errors detected in compressed data of binwalk_output/manual_ooxml_421888.zip.

(kali@kali)~[/forensic_lab]
$ unzip -l binwalk_output/manual_ooxml_421888.zip | head -n 30
Archive:  binwalk_output/manual_ooxml_421888.zip
Length   Date       Time      Name
-----
1364    1980-01-01 00:00    [Content.Types].xml
590     1980-01-01 00:00    _rels/.rels
1149    1980-01-01 00:00    word/_rels/document.xml.rels
4120    1980-01-01 00:00    word/document.xml
85046   1980-01-01 00:00    word/media/image1.jpeg
6795    1980-01-01 00:00    word/theme/theme1.xml
2785    1980-01-01 00:00    word/settings.xml
497     1980-01-01 00:00    word/webSettings.xml
749     1980-01-01 00:00    docProps/core.xml
28961   1980-01-01 00:00    word/styles.xml
1261    1980-01-01 00:00    word/fontTable.xml
731     1980-01-01 00:00    docProps/app.xml

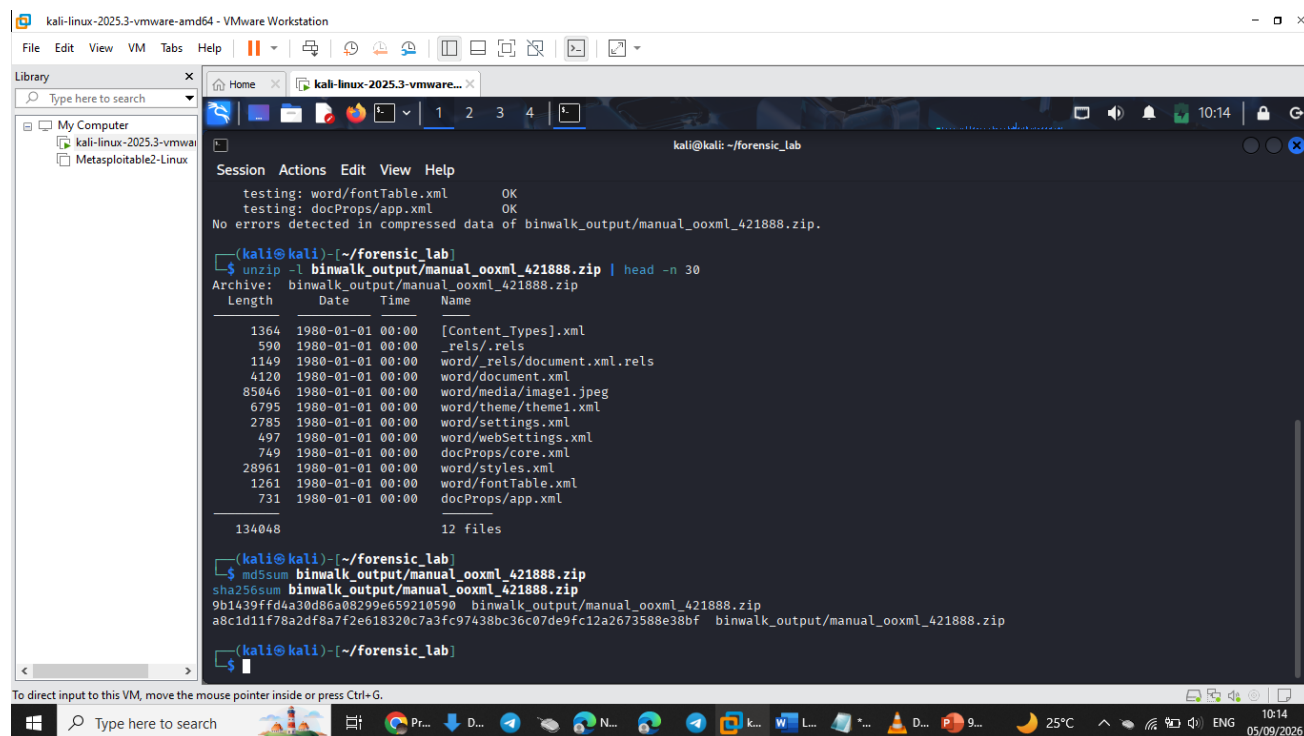
134048      12 files

(kali@kali)~[/forensic_lab]
```

Fig 5.4 Inspecting its contents page 1.

Explanation

unzip -l displayed the files stored inside the recovered object, including [Content_Types].xml, word/document.xml, relationship files, document properties, styles, and word/media/image1.jpeg. These entries are consistent with the Word OOXML structure documented in the lab.



The screenshot shows a Kali Linux terminal window with the following commands and output:

```
kali@kali: ~/forensic_lab
$ unzip -l binwalk_output/manual_ooxml_421888.zip
Archive: binwalk_output/manual_ooxml_421888.zip
  Length      Date    Time    Name
-----
1364  1980-01-01 00:00 [Content_Types].xml
 590  1980-01-01 00:00 _rels/.rels
1149  1980-01-01 00:00 word/_rels/document.xml.rels
4120  1980-01-01 00:00 word/document.xml
85046 1980-01-01 00:00 word/media/image1.jpeg
 6795 1980-01-01 00:00 word/theme/theme1.xml
 2785 1980-01-01 00:00 word/settings.xml
  497 1980-01-01 00:00 word/webSettings.xml
  749 1980-01-01 00:00 docProps/core.xml
28961 1980-01-01 00:00 word/styles.xml
 1261 1980-01-01 00:00 word/fontTable.xml
   731 1980-01-01 00:00 docProps/app.xml
-----
134048                               12 files

(kali@kali)~/forensic_lab
$ md5sum binwalk_output/manual_ooxml_421888.zip
9b1439ffd4a30d86a08299e659210590 binwalk_output/manual_ooxml_421888.zip
$ sha256sum binwalk_output/manual_ooxml_421888.zip
a8c1d11f78a2df8a7f2e618320c7a3fc97438bc36c07de9fc12a2673588e38bf binwalk_output/manual_ooxml_421888.zip
```

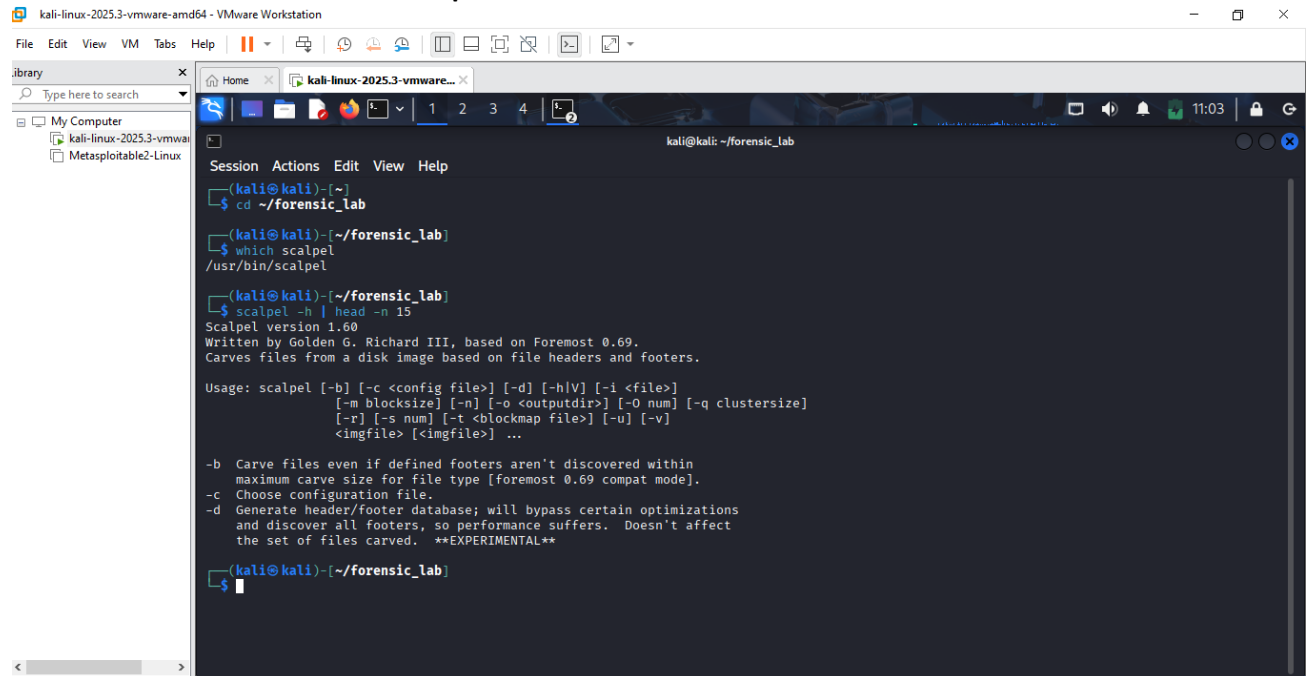
Fig 5.5 Hashing using md5sum and sha256sum.

Explanation

MD5 and SHA-256 hashes were generated for the manually extracted OOXML file. Recording these values preserved an integrity reference for the recovered object after successful validation.

The instructor-referenced File_carving.docx was not available in the supplied evidence. In accordance with the lab instructions, Binwalk practice was therefore completed using the supplied USB forensic image. Binwalk identified an Office Open XML ZIP structure beginning at decimal offset 421888 (0x67000). The object was manually extracted using dd. Validation with unzip -t reported no errors, while unzip -l confirmed a Microsoft Word OOXML structure containing word/document.xml, relationship files, document properties, styles and an embedded JPEG. Cryptographic hashes were recorded for the recovered object.

Part F - Carve Deleted Files with Scalpel



The screenshot shows a Kali Linux terminal window titled "kali@kali: ~/forensic_lab". The user has navigated to the directory ~/forensic_lab and run the command "which scalpel", which returns "/usr/bin/scalpel". The user then runs "scalpel -h | head -n 15", displaying the Scalpel version (1.60) and its usage information. The usage information includes options for configuration file, directory, file, block size, output directory, blockmap file, and image file. It also lists several flags: -b (Carve files even if defined footers aren't discovered within maximum carve size for file type [foremost 0.69 compat mode].), -c (Choose configuration file.), -d (Generate header/footer database; will bypass certain optimizations and discover all footers, so performance suffers. Doesn't affect the set of files carved. **EXPERIMENTAL**), -i (Carve files even if defined footers aren't discovered within maximum carve size for file type [foremost 0.69 compat mode].), -m (Carve files even if defined footers aren't discovered within maximum carve size for file type [foremost 0.69 compat mode].), -n (Carve files even if defined footers aren't discovered within maximum carve size for file type [foremost 0.69 compat mode].), -o (Carve files even if defined footers aren't discovered within maximum carve size for file type [foremost 0.69 compat mode].), -q (Carve files even if defined footers aren't discovered within maximum carve size for file type [foremost 0.69 compat mode].), -r (Carve files even if defined footers aren't discovered within maximum carve size for file type [foremost 0.69 compat mode].), -s (Carve files even if defined footers aren't discovered within maximum carve size for file type [foremost 0.69 compat mode].), -t (Carve files even if defined footers aren't discovered within maximum carve size for file type [foremost 0.69 compat mode].), -u (Carve files even if defined footers aren't discovered within maximum carve size for file type [foremost 0.69 compat mode].), and -v (Carve files even if defined footers aren't discovered within maximum carve size for file type [foremost 0.69 compat mode].).

```
kali@kali:~/forensic_lab$ cd ~/forensic_lab
kali@kali:~/forensic_lab$ which scalpel
/usr/bin/scalpel
kali@kali:~/forensic_lab$ scalpel -h | head -n 15
Scalpel version 1.60
Written by Golden G. Richard III, based on Foremost 0.69.
Carves files from a disk image based on file headers and footers.

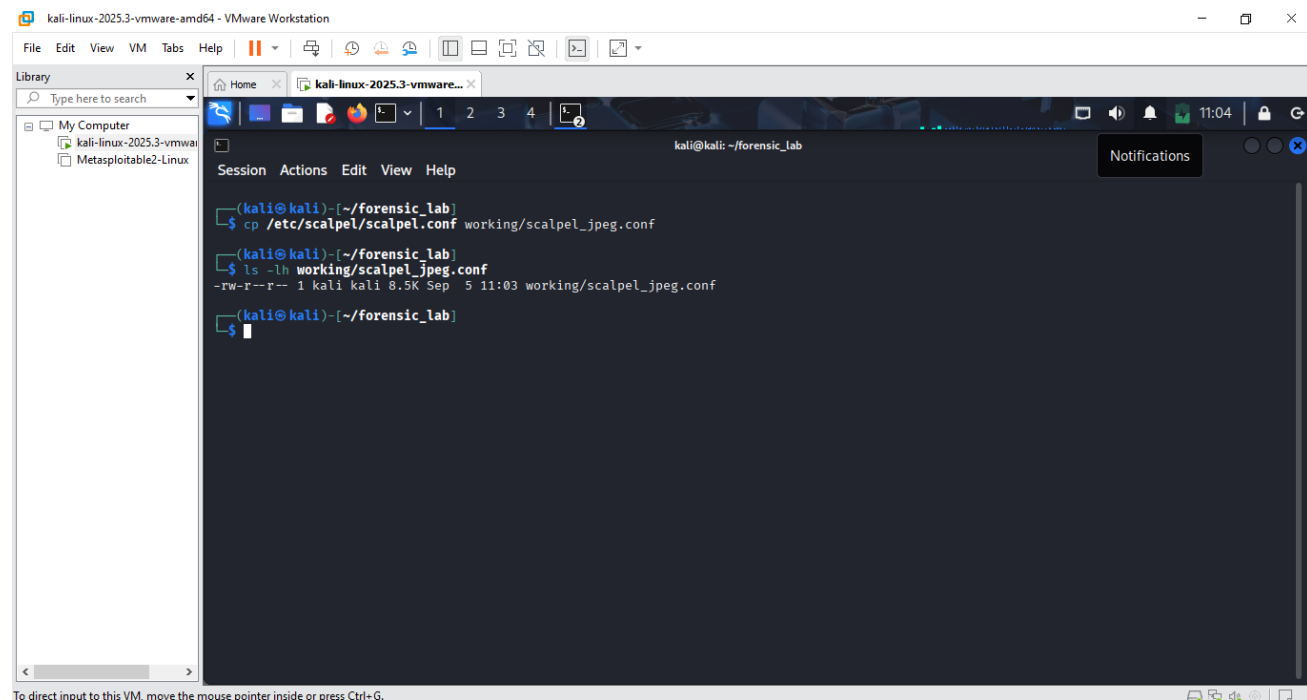
Usage: scalpel [-b] [-c <config file>] [-d] [-h|V] [-i <file>]
      [-m blocksize] [-n] [-o <outputdir>] [-O num] [-q clustersize]
      [-r] [-s num] [-t <blockmap file>] [-u] [-v]
      <imgfile> [<imgfile>] ...

-b Carve files even if defined footers aren't discovered within
  maximum carve size for file type [foremost 0.69 compat mode].
-c Choose configuration file.
-d Generate header/footer database; will bypass certain optimizations
  and discover all footers, so performance suffers. Doesn't affect
  the set of files carved. **EXPERIMENTAL**
```

Fig 5.6 Checking if Scalpel is installed.

Explanation

The system was checked for the Scalpel executable, and the help information confirmed that Scalpel was available for the carving stage of the laboratory.



The screenshot shows a Kali Linux terminal window titled "kali@kali: ~/forensic_lab". The user has run the command "cp /etc/scalpel/scalpel.conf working/scalpel_jpeg.conf", which has created a new file in the working directory. The user then runs "ls -lh working/scalpel_jpeg.conf", which shows the file's permissions, size, and creation date. The output of the ls command is: "-rw-r--r-- 1 kali kali 8.5K Sep 5 11:03 working/scalpel_jpeg.conf".

```
kali@kali:~/forensic_lab$ cp /etc/scalpel/scalpel.conf working/scalpel_jpeg.conf
kali@kali:~/forensic_lab$ ls -lh working/scalpel_jpeg.conf
-rw-r--r-- 1 kali kali 8.5K Sep 5 11:03 working/scalpel_jpeg.conf
```

Fig 5.7 Making a working copy of the Scalpel configuration.

Explanation

The system Scalpel configuration was copied into the working directory. This allowed the carving rules to be adjusted without altering the original system configuration file.

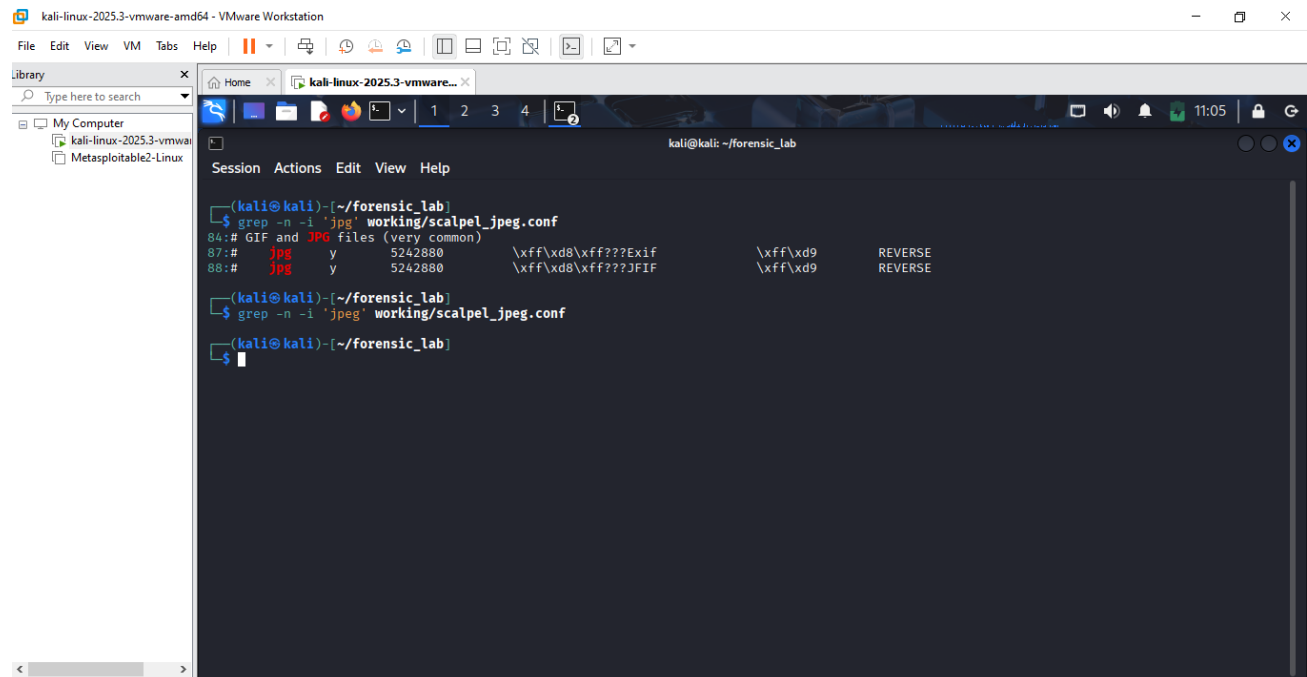


Fig 5.8 Locating all JPEG-related configuration.

Explanation

The configuration was searched for JPEG rules, revealing separate EXIF- and JFIF-based signatures. At this point, the entries were visible and ready to be enabled for the required carving exercise.

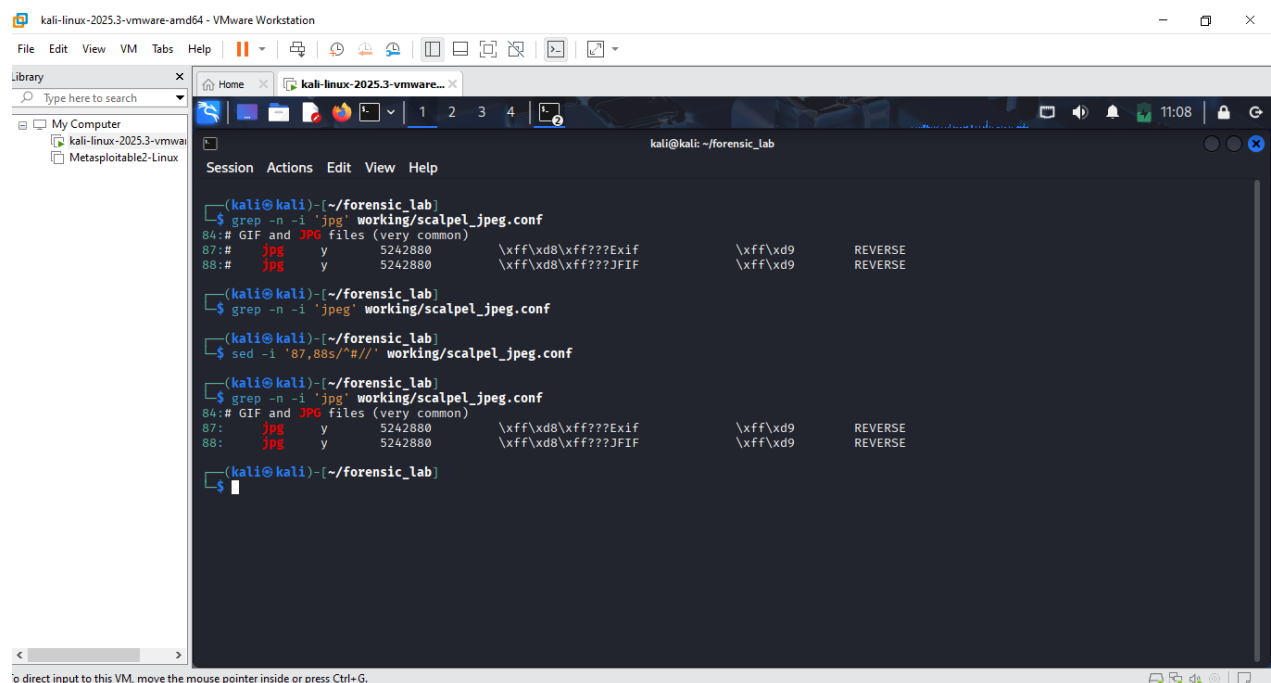


Fig 5.9 Evidence that the JPEG/JPG types were enabled in the Scalpel configuration.

Explanation

The relevant JPEG configuration lines were uncommented, enabling both EXIF- and JFIF-based JPEG carving. A second search confirmed that the rules were now active.

```
kali@kali: ~/forensic_lab
$ grep -n -i 'jpg' working/scalpel_*.conf
84: # GIF and JPG files (very common)
87: # jpg y 5242880 \\xff\\xd8\\xff???Exif \\xff\\xd9 REVERSE
88: # jpg y 5242880 \\xff\\xd8\\xff???JFIF \\xff\\xd9 REVERSE

(kali@kali)~/forensic_lab
$ grep -n -i 'jpeg' working/scalpel_*.conf

(kali@kali)~/forensic_lab
$ sed -i '87,88s/^#//' working/scalpel_*.conf

(kali@kali)~/forensic_lab
$ grep -n -i 'jpg' working/scalpel_*.conf
84: # GIF and JPG files (very common)
87: jpg y 5242880 \\xff\\xd8\\xff???Exif \\xff\\xd9 REVERSE
88: jpg y 5242880 \\xff\\xd8\\xff???JFIF \\xff\\xd9 REVERSE

(kali@kali)~/forensic_lab
$ md5sum working/scalpel_*.conf | tee hashes/scalpel_config_md5.txt
sha256sum working/scalpel_*.conf | tee hashes/scalpel_config_sha256.txt
dbafa999d0c651446268d4822a228c2f working/scalpel_*.conf
5b12b3c79012d3490a7d43ffa6fb864db736805f8e97d08377d2952be59a6630 working/scalpel_*.conf

(kali@kali)~/forensic_lab
$
```

Fig 6.0 Preserving the configured file's hash.

Explanation

MD5 and SHA-256 hashes were calculated for the modified Scalpel configuration. This created a record of the exact configuration used to produce the carving results.

```
kali@kali: ~/forensic_lab
$ mkdir -p scalpel_output

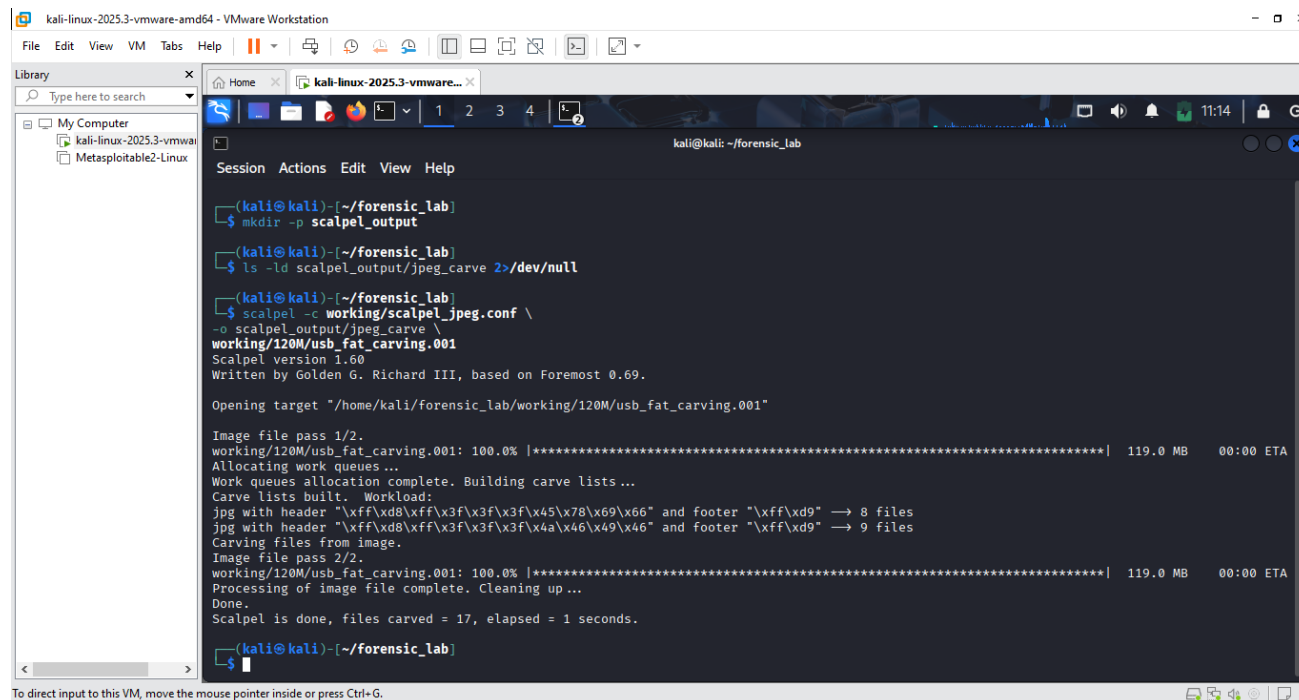
(kali@kali)~/forensic_lab
$ ls -ld scalpel_output/jpeg_carve 2>/dev/null
ls: cannot access 'scalpel_output/jpeg_carve': No such file or directory

(kali@kali)~/forensic_lab
$
```

Fig 6.1 Making sure the parent output directory exists.

Explanation

The output directory for Scalpel results was created and checked before carving began. This ensured that the recovered files and audit information would be stored in a dedicated location.



```
kali@kali: ~/forensic_lab
$ mkdir -p scalpel_output
$ ls -ld scalpel_output/jpeg_carve 2>/dev/null
$ scalpel -c working/scalpel_jpeg.conf \
-o scalpel_output/jpeg_carve \
working/120M/usb_fat_carving.001
Scalpel version 1.60
Written by Golden G. Richard III, based on Foremost 0.69.

Opening target "/home/kali/forensic_lab/working/120M/usb_fat_carving.001"

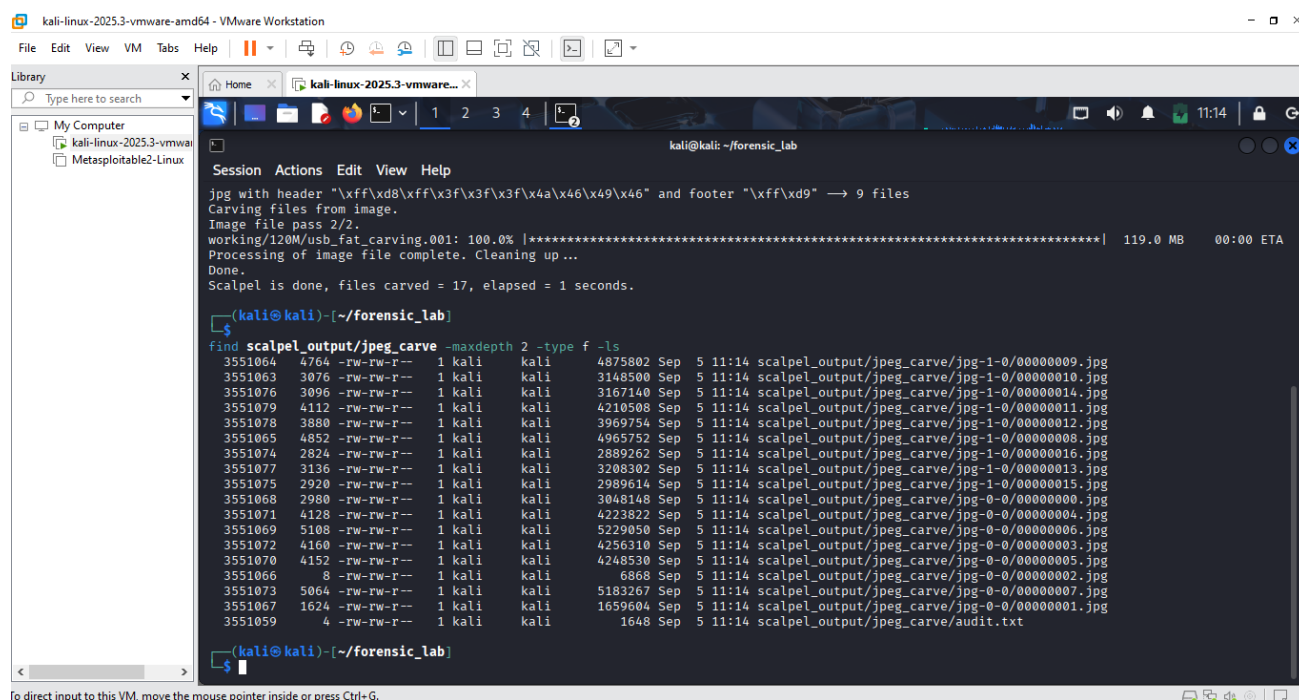
Image file pass 1/2.
working/120M/usb_fat_carving.001: 100.0% [*****] 119.0 MB 00:00 ETA
Allocating work queues ...
Work queues allocation complete. Building carve lists ...
Carve lists built. Workload:
jpg with header "\xff\xd8\xff\x3f\x3f\x3f\x45\x78\x69\x66" and footer "\xff\xd9" -> 8 files
jpg with header "\xff\xd8\xff\x3f\x3f\x3f\x4a\x46\x49\x46" and footer "\xff\xd9" -> 9 files
Carving files from image.
Image file pass 2/2.
working/120M/usb_fat_carving.001: 100.0% [*****] 119.0 MB 00:00 ETA
Processing of image file complete. Cleaning up ...
Done.
Scalpel is done, files carved = 17, elapsed = 1 seconds.

$
```

Fig 6.2 Scaple carving.

Explanation

Scalpel was run against usb_fat_carving.001 using the custom JPEG configuration. The two-pass process completed successfully and reported a total of 17 carved files.



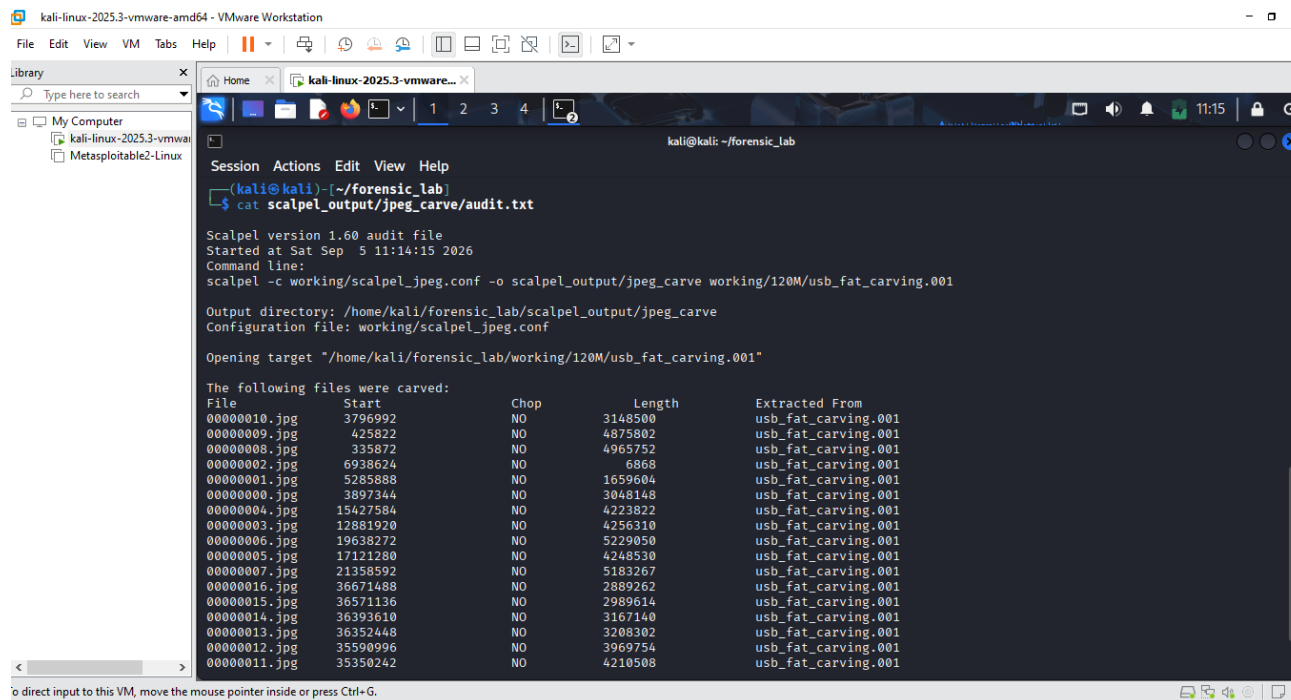
```
kali@kali: ~/forensic_lab
$ find scalpel_output/jpeg_carve -maxdepth 2 -type f -ls
3551064 4764 -rw-rw-r-- 1 kali kali 4875802 Sep 5 11:14 scalpel_output/jpeg_carve/jpg-1-0/00000009.jpg
3551063 3076 -rw-rw-r-- 1 kali kali 3148500 Sep 5 11:14 scalpel_output/jpeg_carve/jpg-1-0/00000010.jpg
3551076 3096 -rw-rw-r-- 1 kali kali 3167140 Sep 5 11:14 scalpel_output/jpeg_carve/jpg-1-0/00000014.jpg
3551079 4112 -rw-rw-r-- 1 kali kali 4210508 Sep 5 11:14 scalpel_output/jpeg_carve/jpg-1-0/00000011.jpg
3551078 3880 -rw-rw-r-- 1 kali kali 3969754 Sep 5 11:14 scalpel_output/jpeg_carve/jpg-1-0/00000012.jpg
3551065 4852 -rw-rw-r-- 1 kali kali 4965752 Sep 5 11:14 scalpel_output/jpeg_carve/jpg-1-0/00000008.jpg
3551074 2824 -rw-rw-r-- 1 kali kali 2889262 Sep 5 11:14 scalpel_output/jpeg_carve/jpg-1-0/00000016.jpg
3551077 3136 -rw-rw-r-- 1 kali kali 3208302 Sep 5 11:14 scalpel_output/jpeg_carve/jpg-1-0/00000013.jpg
3551075 2920 -rw-rw-r-- 1 kali kali 2989614 Sep 5 11:14 scalpel_output/jpeg_carve/jpg-1-0/00000015.jpg
3551068 2980 -rw-rw-r-- 1 kali kali 3048148 Sep 5 11:14 scalpel_output/jpeg_carve/jpg-0-0/00000000.jpg
3551071 4128 -rw-rw-r-- 1 kali kali 4223822 Sep 5 11:14 scalpel_output/jpeg_carve/jpg-0-0/00000004.jpg
3551069 5108 -rw-rw-r-- 1 kali kali 5229050 Sep 5 11:14 scalpel_output/jpeg_carve/jpg-0-0/00000006.jpg
3551072 4160 -rw-rw-r-- 1 kali kali 4256310 Sep 5 11:14 scalpel_output/jpeg_carve/jpg-0-0/00000003.jpg
3551070 4152 -rw-rw-r-- 1 kali kali 4248530 Sep 5 11:14 scalpel_output/jpeg_carve/jpg-0-0/00000005.jpg
3551066 8 -rw-rw-r-- 1 kali kali 6868 Sep 5 11:14 scalpel_output/jpeg_carve/jpg-0-0/00000002.jpg
3551073 5064 -rw-rw-r-- 1 kali kali 5183267 Sep 5 11:14 scalpel_output/jpeg_carve/jpg-0-0/00000007.jpg
3551067 1624 -rw-rw-r-- 1 kali kali 1659604 Sep 5 11:14 scalpel_output/jpeg_carve/jpg-0-0/00000001.jpg
3551059 4 -rw-rw-r-- 1 kali kali 1648 Sep 5 11:14 scalpel_output/jpeg_carve/audit.txt

$
```

Fig 6.3 Scaple carving for jpeg.

Explanation

The carved JPEG files were listed after the operation completed. The output shows files recovered into separate JPEG directories together with audit.txt.



```
kali@kali: ~/forensic_lab
$ cat scalpel_output/jpeg_carve/audit.txt

Scalpel version 1.60 audit file
Started at Sat Sep 5 11:14:15 2026
Command line:
scalpel -c working/scalpel_jpeg.conf -o scalpel_output/jpeg_carve working/120M/usb_fat_carving.001

Output directory: /home/kali/forensic_lab/scalpel_output/jpeg_carve
Configuration file: working/scalpel_jpeg.conf

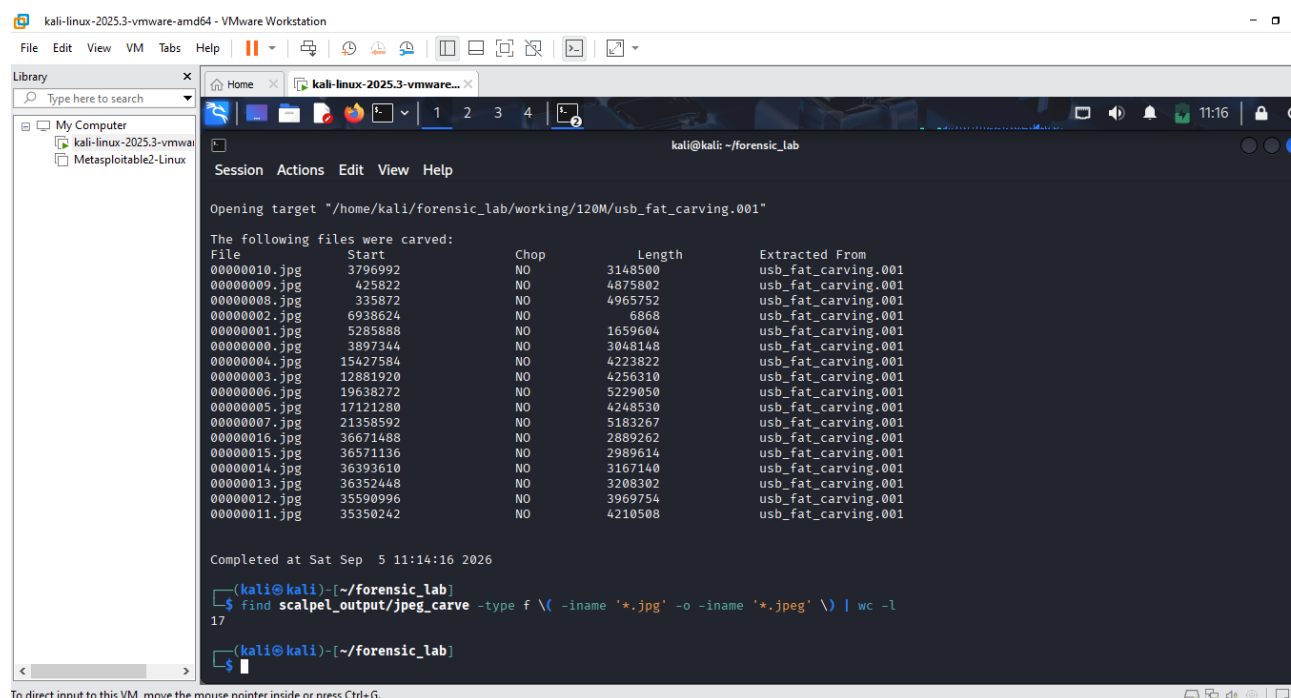
Opening target "/home/kali/forensic_lab/working/120M/usb_fat_carving.001"

The following files were carved:
File           Start      Chop      Length      Extracted From
00000010.jpg   3796992    NO        3148500     usb_fat_carving.001
00000009.jpg   425822     NO        4875802     usb_fat_carving.001
00000008.jpg   335872     NO        4965752     usb_fat_carving.001
00000002.jpg   6938624    NO        6868        usb_fat_carving.001
00000001.jpg   5285888    NO        1659604     usb_fat_carving.001
00000000.jpg   3897344    NO        3048148     usb_fat_carving.001
00000004.jpg   15427584   NO        4223822     usb_fat_carving.001
00000003.jpg   12881920   NO        4256310     usb_fat_carving.001
00000006.jpg   19638272   NO        5229050     usb_fat_carving.001
00000005.jpg   17121280   NO        4248530     usb_fat_carving.001
00000007.jpg   21358592   NO        5183267     usb_fat_carving.001
00000016.jpg   36671488   NO        2889262     usb_fat_carving.001
00000015.jpg   36571136   NO        2989614     usb_fat_carving.001
00000014.jpg   36393610   NO        3167140     usb_fat_carving.001
00000013.jpg   36352448   NO        3208302     usb_fat_carving.001
00000012.jpg   35590996   NO        3969754     usb_fat_carving.001
00000011.jpg   35350242   NO        4210508     usb_fat_carving.001
```

Fig 6.4 Locating the audit file.

Explanation

Audit.txt was opened to review the results generated by Scalpel. The audit information recorded each carved filename, start offset, file length, chop status, and the source forensic image.



```
kali@kali: ~/forensic_lab

Opening target "/home/kali/forensic_lab/working/120M/usb_fat_carving.001"

The following files were carved:
File           Start      Chop      Length      Extracted From
00000010.jpg   3796992    NO        3148500     usb_fat_carving.001
00000009.jpg   425822     NO        4875802     usb_fat_carving.001
00000008.jpg   335872     NO        4965752     usb_fat_carving.001
00000002.jpg   6938624    NO        6868        usb_fat_carving.001
00000001.jpg   5285888    NO        1659604     usb_fat_carving.001
00000000.jpg   3897344    NO        3048148     usb_fat_carving.001
00000004.jpg   15427584   NO        4223822     usb_fat_carving.001
00000003.jpg   12881920   NO        4256310     usb_fat_carving.001
00000006.jpg   19638272   NO        5229050     usb_fat_carving.001
00000005.jpg   17121280   NO        4248530     usb_fat_carving.001
00000007.jpg   21358592   NO        5183267     usb_fat_carving.001
00000016.jpg   36671488   NO        2889262     usb_fat_carving.001
00000015.jpg   36571136   NO        2989614     usb_fat_carving.001
00000014.jpg   36393610   NO        3167140     usb_fat_carving.001
00000013.jpg   36352448   NO        3208302     usb_fat_carving.001
00000012.jpg   35590996   NO        3969754     usb_fat_carving.001
00000011.jpg   35350242   NO        4210508     usb_fat_carving.001

Completed at Sat Sep 5 11:14:16 2026

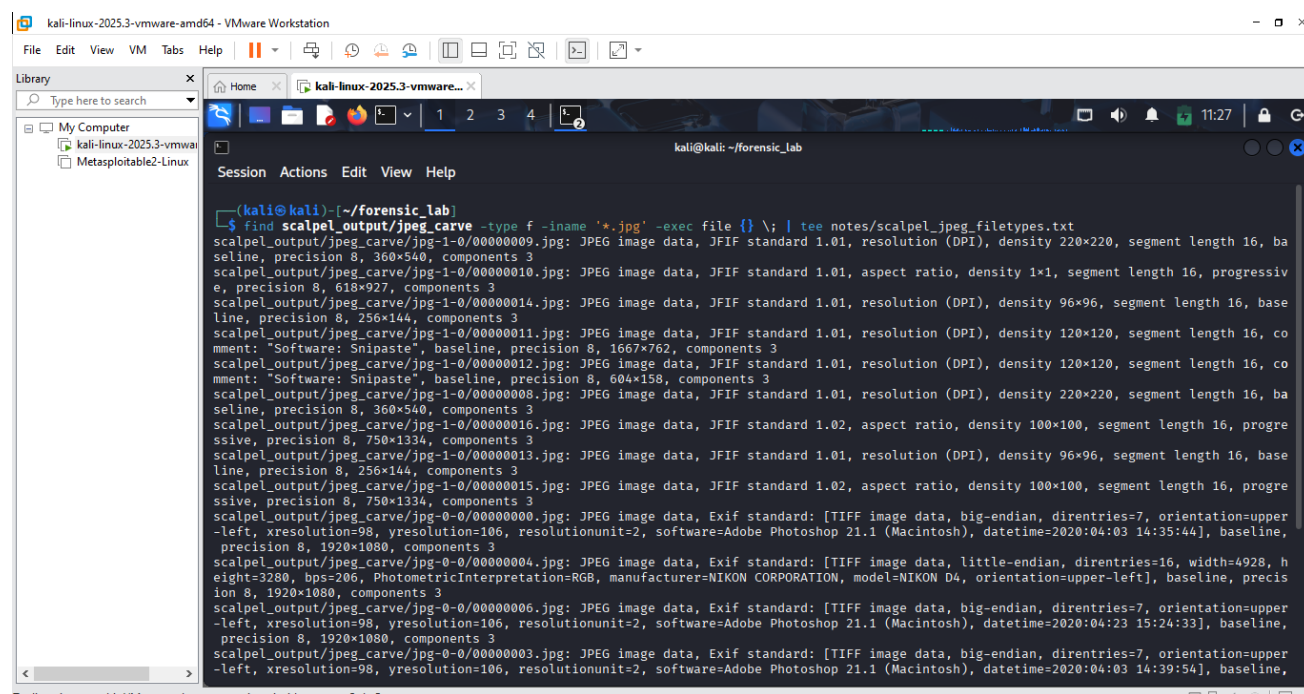
(kali@kali)~/forensic_lab
$ find scalpel_output/jpeg_carve -type f \( -iname '*.jpg' -o -iname '*.jpeg' \) | wc -l
17

(kali@kali)~/forensic_lab
$
```

Fig 6.5 counting the carved JPEGs.

Explanation

A find command was used to count files with .jpg or .jpeg extensions in the carving output. The result confirmed that 17 JPEG files had been recovered.

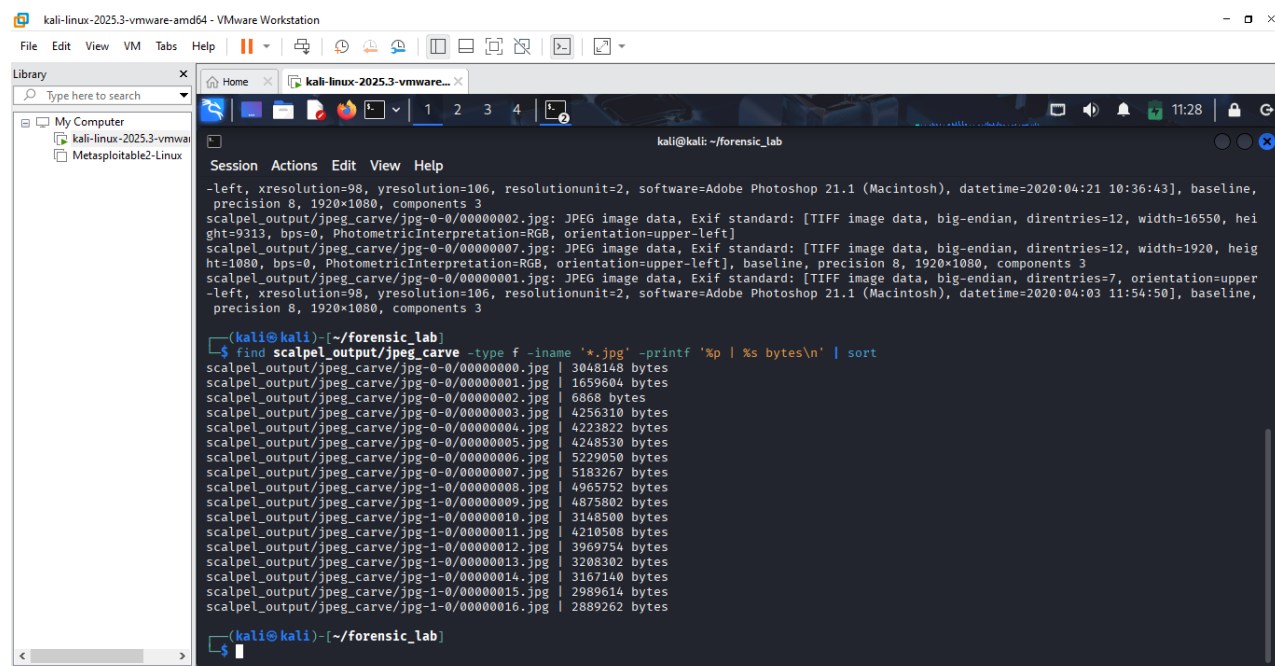


```
kali@kali: ~/forensic_lab
$ find scalpel_output/jpeg_carve -type f -iname '*.jpg' -exec file {} \; | tee notes/scalpel_jpeg_filetypes.txt
scalpel_output/jpeg_carve/jpg-1-0/00000009.jpg: JPEG image data, JFIF standard 1.01, resolution (DPI), density 220x220, segment length 16, baseline, precision 8, 360x540, components 3
scalpel_output/jpeg_carve/jpg-1-0/00000010.jpg: JPEG image data, JFIF standard 1.01, aspect ratio, density 1x1, segment length 16, progressive, precision 8, 618x927, components 3
scalpel_output/jpeg_carve/jpg-1-0/00000014.jpg: JPEG image data, JFIF standard 1.01, resolution (DPI), density 96x96, segment length 16, baseline, precision 8, 256x144, components 3
scalpel_output/jpeg_carve/jpg-1-0/00000011.jpg: JPEG image data, JFIF standard 1.01, resolution (DPI), density 120x120, segment length 16, comment: "Software: Snipaste", baseline, precision 8, 1667x762, components 3
scalpel_output/jpeg_carve/jpg-1-0/00000012.jpg: JPEG image data, JFIF standard 1.01, resolution (DPI), density 120x120, segment length 16, comment: "Software: Snipaste", baseline, precision 8, 604x158, components 3
scalpel_output/jpeg_carve/jpg-1-0/00000008.jpg: JPEG image data, JFIF standard 1.01, resolution (DPI), density 220x220, segment length 16, baseline, precision 8, 360x540, components 3
scalpel_output/jpeg_carve/jpg-1-0/00000016.jpg: JPEG image data, JFIF standard 1.02, aspect ratio, density 100x100, segment length 16, progressive, precision 8, 750x1334, components 3
scalpel_output/jpeg_carve/jpg-1-0/00000013.jpg: JPEG image data, JFIF standard 1.01, resolution (DPI), density 96x96, segment length 16, baseline, precision 8, 256x144, components 3
scalpel_output/jpeg_carve/jpg-1-0/00000015.jpg: JPEG image data, JFIF standard 1.02, aspect ratio, density 100x100, segment length 16, progressive, precision 8, 750x1334, components 3
scalpel_output/jpeg_carve/jpg-0-0/00000000.jpg: JPEG image data, Exif standard: [TIFF image data, big-endian, direntries=7, orientation=upper-left, xresolution=98, yresolution=106, resolutionunit=2, software=Adobe Photoshop 21.1 (Macintosh), datetime=2020:04:03 14:35:44], baseline, precision 8, 1920x1080, components 3
scalpel_output/jpeg_carve/jpg-0-0/00000004.jpg: JPEG image data, Exif standard: [TIFF image data, little-endian, direntries=16, width=4928, height=3280, bps=206, PhotometricInterpretation=RGB, manufacturer=NIKON CORPORATION, model=NIKON D4, orientation=upper-left], baseline, precision 8, 1920x1080, components 3
scalpel_output/jpeg_carve/jpg-0-0/00000006.jpg: JPEG image data, Exif standard: [TIFF image data, big-endian, direntries=7, orientation=upper-left, xresolution=98, yresolution=106, resolutionunit=2, software=Adobe Photoshop 21.1 (Macintosh), datetime=2020:04:03 15:24:33], baseline, precision 8, 1920x1080, components 3
scalpel_output/jpeg_carve/jpg-0-0/00000003.jpg: JPEG image data, Exif standard: [TIFF image data, big-endian, direntries=7, orientation=upper-left, xresolution=98, yresolution=106, resolutionunit=2, software=Adobe Photoshop 21.1 (Macintosh), datetime=2020:04:03 14:39:54], baseline, precision 8, 1920x1080, components 3
```

Fig 6.6 Verify the recovered JPEGs.

Explanation

The file utility was run against the recovered images to check their recognized formats and image characteristics. The output confirmed that the carved items were being identified as JPEG image data.

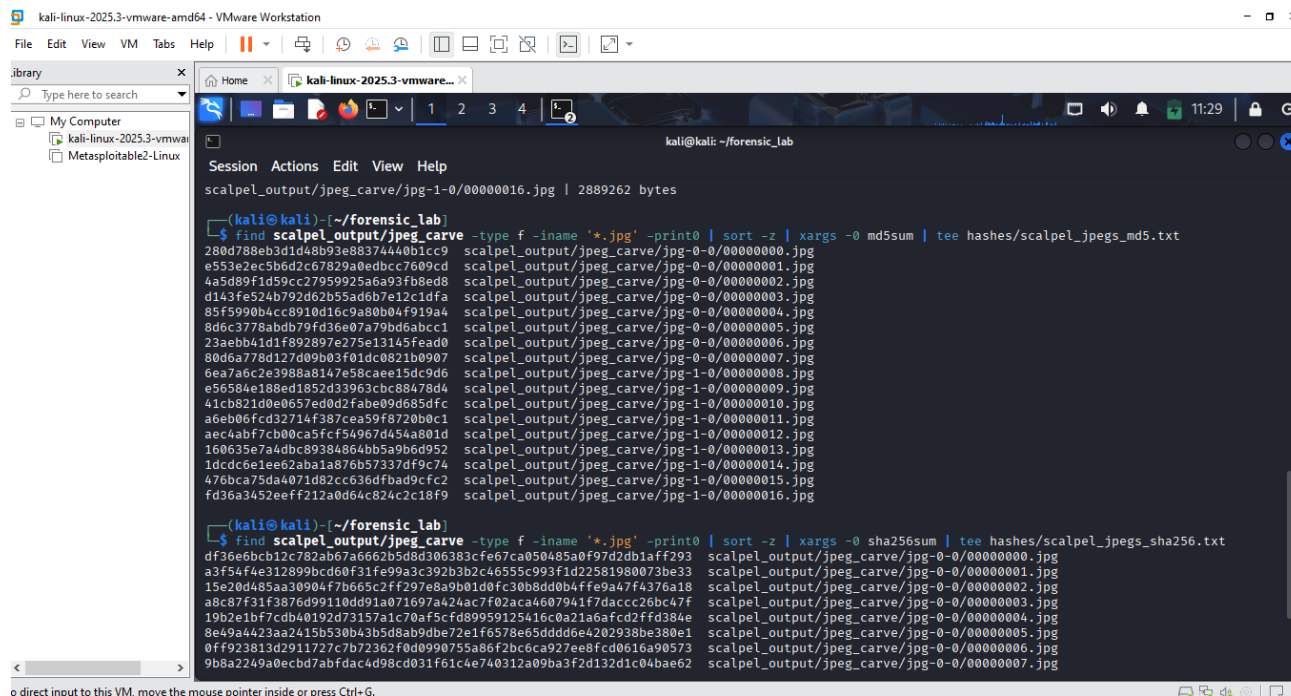


```
kali@kali: ~/forensic_lab
$ find scalpel_output/jpeg_carve -type f -iname '*.jpg' -printf '%p | %s bytes\n' | sort
scalpel_output/jpeg_carve/jpg-0-0/00000000.jpg | 3048148 bytes
scalpel_output/jpeg_carve/jpg-0-0/00000001.jpg | 1659604 bytes
scalpel_output/jpeg_carve/jpg-0-0/00000002.jpg | 6868 bytes
scalpel_output/jpeg_carve/jpg-0-0/00000003.jpg | 4256310 bytes
scalpel_output/jpeg_carve/jpg-0-0/00000004.jpg | 4223822 bytes
scalpel_output/jpeg_carve/jpg-0-0/00000005.jpg | 4248530 bytes
scalpel_output/jpeg_carve/jpg-0-0/00000006.jpg | 5229050 bytes
scalpel_output/jpeg_carve/jpg-0-0/00000007.jpg | 5183267 bytes
scalpel_output/jpeg_carve/jpg-1-0/00000008.jpg | 4965752 bytes
scalpel_output/jpeg_carve/jpg-1-0/00000009.jpg | 4875802 bytes
scalpel_output/jpeg_carve/jpg-1-0/00000010.jpg | 3148500 bytes
scalpel_output/jpeg_carve/jpg-1-0/00000011.jpg | 4210508 bytes
scalpel_output/jpeg_carve/jpg-1-0/00000012.jpg | 3969754 bytes
scalpel_output/jpeg_carve/jpg-1-0/00000013.jpg | 3208302 bytes
scalpel_output/jpeg_carve/jpg-1-0/00000014.jpg | 3167140 bytes
scalpel_output/jpeg_carve/jpg-1-0/00000015.jpg | 2989614 bytes
scalpel_output/jpeg_carve/jpg-1-0/00000016.jpg | 2889262 bytes
```

Fig 6.7 Verify the recovered JPEGs page 1.

Explanation

The carved JPEGs were listed together with their file sizes. This provided a simple inventory showing that the recovered images varied in size rather than all being identical artefacts.



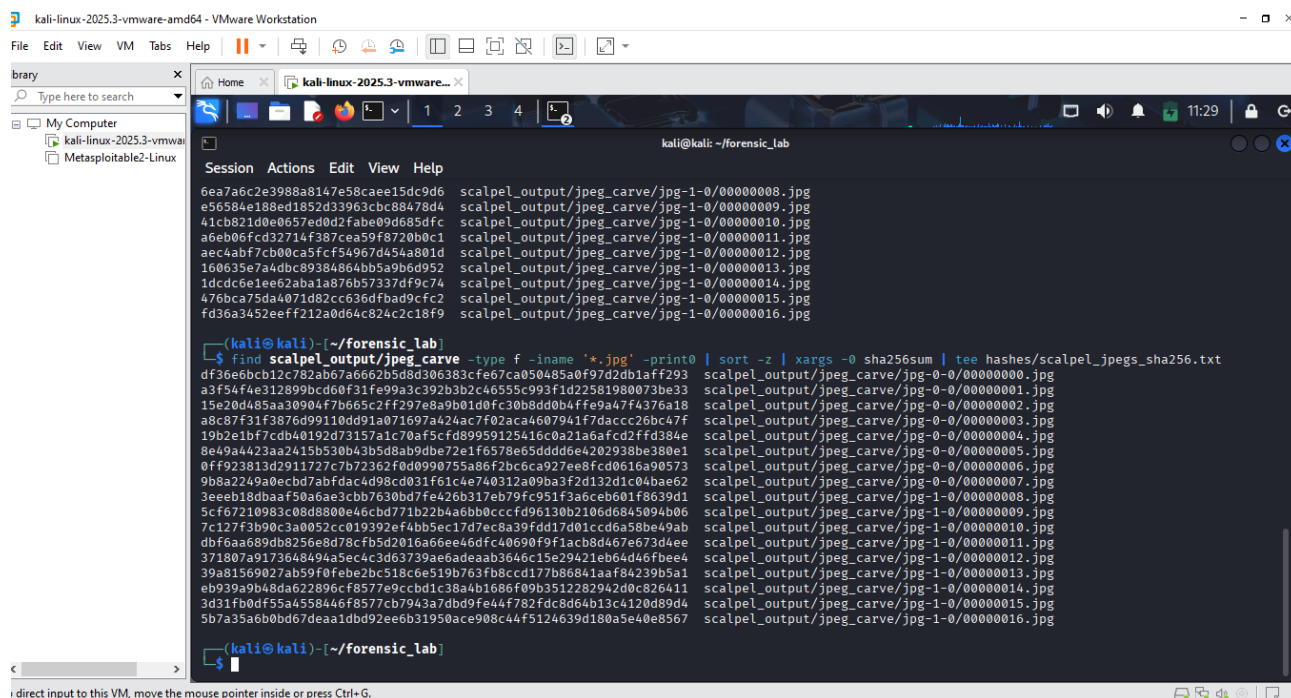
```
kali@kali: ~/forensic_lab
$ find scalpel_output/jpeg_carve -type f -iname '*.jpg' -print0 | sort -z | xargs -0 md5sum | tee hashes/scalpel_jpegs_md5.txt
scalpel_output/jpeg_carve/jpg-1-0/00000016.jpg | 2889262 bytes
scalpel_output/jpeg_carve/jpg-0-0/00000000.jpg
scalpel_output/jpeg_carve/jpg-0-0/00000001.jpg
scalpel_output/jpeg_carve/jpg-0-0/00000002.jpg
scalpel_output/jpeg_carve/jpg-0-0/00000003.jpg
scalpel_output/jpeg_carve/jpg-0-0/00000004.jpg
scalpel_output/jpeg_carve/jpg-0-0/00000005.jpg
scalpel_output/jpeg_carve/jpg-0-0/00000007.jpg
scalpel_output/jpeg_carve/jpg-1-0/00000008.jpg
scalpel_output/jpeg_carve/jpg-1-0/00000009.jpg
scalpel_output/jpeg_carve/jpg-1-0/00000010.jpg
scalpel_output/jpeg_carve/jpg-1-0/00000011.jpg
scalpel_output/jpeg_carve/jpg-1-0/00000012.jpg
scalpel_output/jpeg_carve/jpg-1-0/00000013.jpg
scalpel_output/jpeg_carve/jpg-1-0/00000014.jpg
scalpel_output/jpeg_carve/jpg-1-0/00000015.jpg
scalpel_output/jpeg_carve/jpg-1-0/00000016.jpg

(kali@kali)~/forensic_lab
$ find scalpel_output/jpeg_carve -type f -iname '*.jpg' -print0 | sort -z | xargs -0 sha256sum | tee hashes/scalpel_jpegs_sha256.txt
scalpel_output/jpeg_carve/jpg-0-0/00000000.jpg
scalpel_output/jpeg_carve/jpg-0-0/00000001.jpg
scalpel_output/jpeg_carve/jpg-0-0/00000002.jpg
scalpel_output/jpeg_carve/jpg-0-0/00000003.jpg
scalpel_output/jpeg_carve/jpg-0-0/00000004.jpg
scalpel_output/jpeg_carve/jpg-0-0/00000005.jpg
scalpel_output/jpeg_carve/jpg-0-0/00000006.jpg
scalpel_output/jpeg_carve/jpg-0-0/00000007.jpg
```

Fig 6.8 Hash all 17 carved JPEGs using md5sum and sha256sum.

Explanation

MD5 hashes were calculated for the complete set of carved JPEG files and saved to a hash record. SHA-256 processing was also started so that stronger integrity values would be retained.



```
kali@kali: ~/forensic_lab
$ find scalpel_output/jpeg_carve -type f -iname '*.jpg' -print0 | sort -z | xargs -0 sha256sum | tee hashes/scalpel_jpegs_sha256.txt
scalpel_output/jpeg_carve/jpg-1-0/00000008.jpg
scalpel_output/jpeg_carve/jpg-1-0/00000009.jpg
scalpel_output/jpeg_carve/jpg-1-0/00000010.jpg
scalpel_output/jpeg_carve/jpg-1-0/00000011.jpg
scalpel_output/jpeg_carve/jpg-1-0/00000012.jpg
scalpel_output/jpeg_carve/jpg-1-0/00000013.jpg
scalpel_output/jpeg_carve/jpg-1-0/00000014.jpg
scalpel_output/jpeg_carve/jpg-1-0/00000015.jpg
scalpel_output/jpeg_carve/jpg-1-0/00000016.jpg

(kali@kali)~/forensic_lab
$ find scalpel_output/jpeg_carve -type f -iname '*.jpg' -print0 | sort -z | xargs -0 sha256sum | tee hashes/scalpel_jpegs_sha256.txt
scalpel_output/jpeg_carve/jpg-0-0/00000000.jpg
scalpel_output/jpeg_carve/jpg-0-0/00000001.jpg
scalpel_output/jpeg_carve/jpg-0-0/00000002.jpg
scalpel_output/jpeg_carve/jpg-0-0/00000003.jpg
scalpel_output/jpeg_carve/jpg-0-0/00000004.jpg
scalpel_output/jpeg_carve/jpg-0-0/00000005.jpg
scalpel_output/jpeg_carve/jpg-0-0/00000006.jpg
scalpel_output/jpeg_carve/jpg-0-0/00000007.jpg
scalpel_output/jpeg_carve/jpg-1-0/00000008.jpg
scalpel_output/jpeg_carve/jpg-1-0/00000009.jpg
scalpel_output/jpeg_carve/jpg-1-0/00000010.jpg
scalpel_output/jpeg_carve/jpg-1-0/00000011.jpg
scalpel_output/jpeg_carve/jpg-1-0/00000012.jpg
scalpel_output/jpeg_carve/jpg-1-0/00000013.jpg
scalpel_output/jpeg_carve/jpg-1-0/00000014.jpg
scalpel_output/jpeg_carve/jpg-1-0/00000015.jpg
scalpel_output/jpeg_carve/jpg-1-0/00000016.jpg
```

Fig 6.9 Hash all 17 carved JPEGs using md5sum and sha256sum page 1.

Explanation

This screenshot completes the SHA-256 hash output for the 17 recovered JPEGs. The stored hash records provide individual integrity references for each carved image.

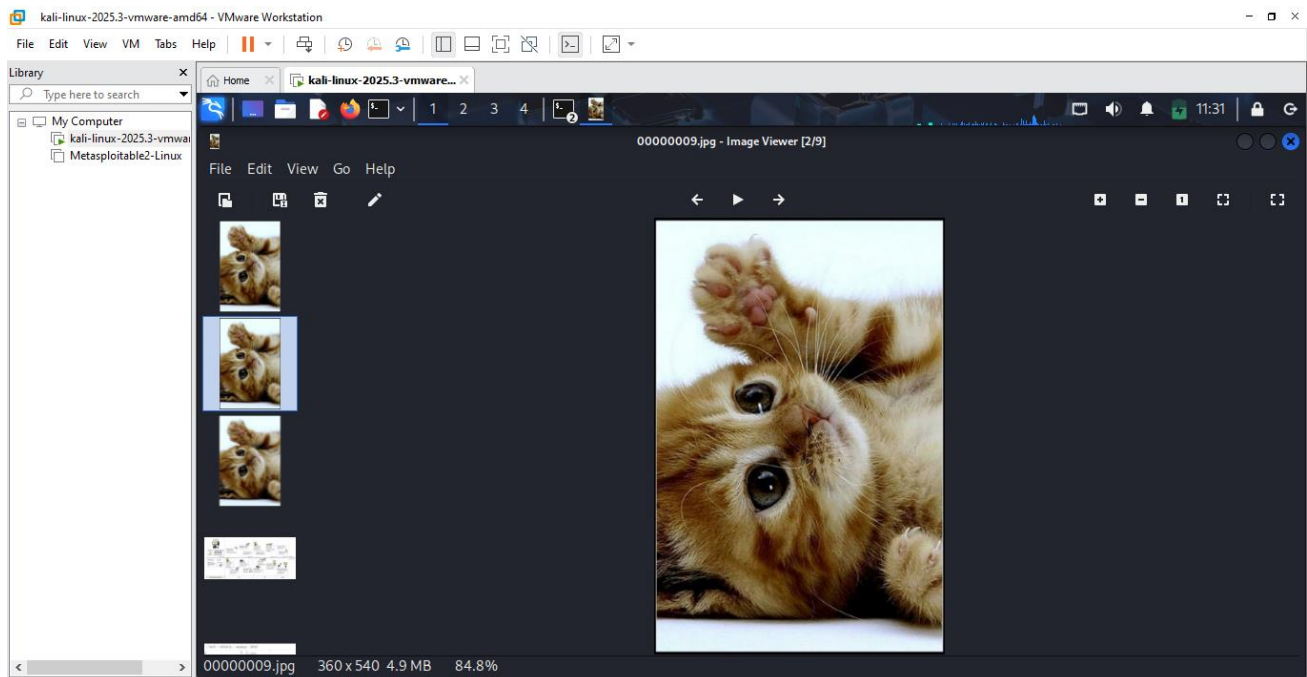


Fig 7.0 Opening recovered JPEGs 00000009.

Explanation

The first selected carved JPEG was successfully opened in the Kali image viewer and displayed a kitten image. Its successful rendering confirmed that Scalpel had recovered usable JPEG content rather than only a signature fragment.

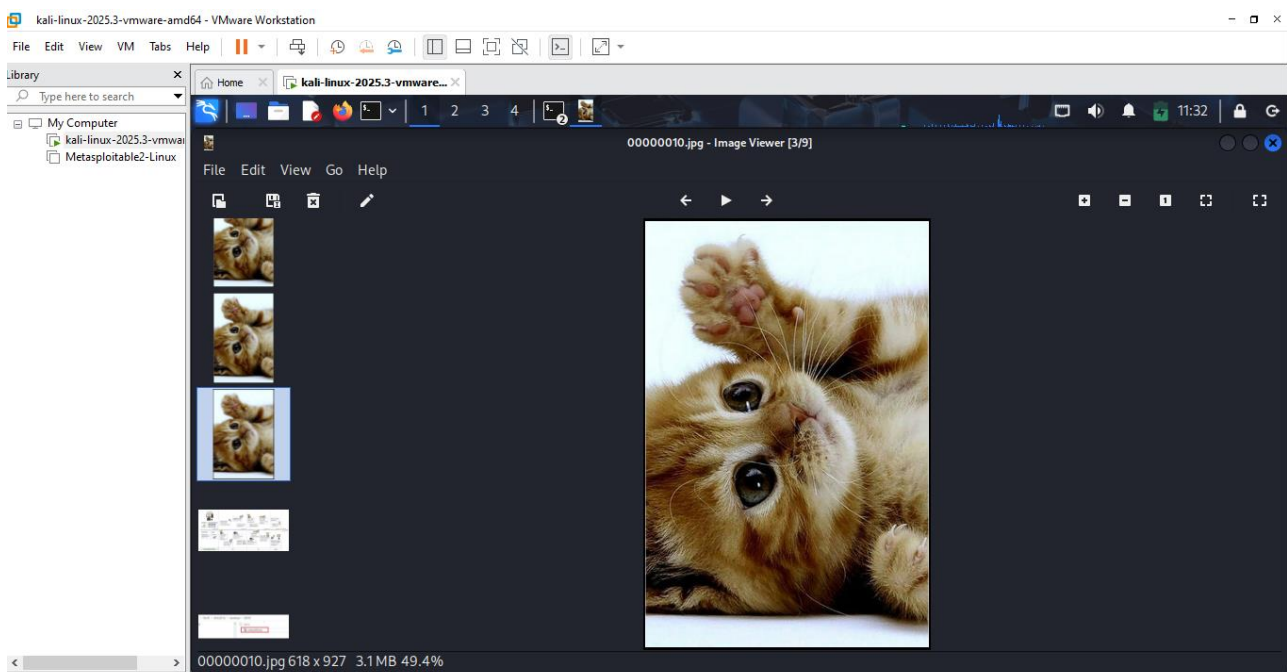
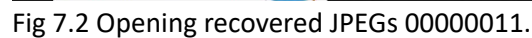


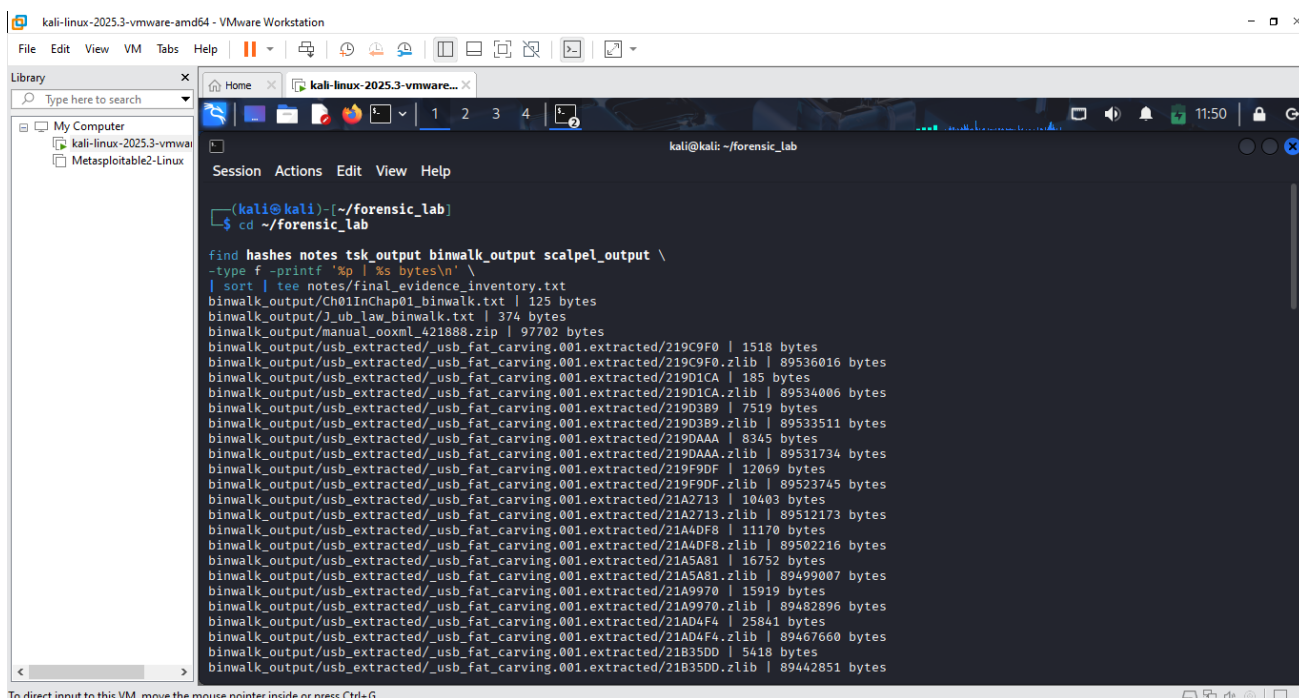
Fig 7.1 Opening recovered JPEGs 00000010.

A second recovered JPEG was opened successfully and also displayed correctly in the image viewer. This provided another visible confirmation that the carving process had produced recoverable image evidence.



Explanation

Scalpel was configured to carve both EXIF- and JFIF-based JPEG files. The carving process completed successfully and recovered 17 JPEG artefacts from the USB forensic image. audit.txt documented the start offset, file length and extraction source for each carved artefact. MD5 and SHA-256 hashes were generated for all recovered JPEG files. Multiple carved images were successfully opened using the Kali image viewer, including 00000009.jpg, 00000010.jpg and 00000011.jpg, confirming that usable image data had been recovered.



```
kali@kali: ~/forensic_lab
$ cd ~/forensic_lab

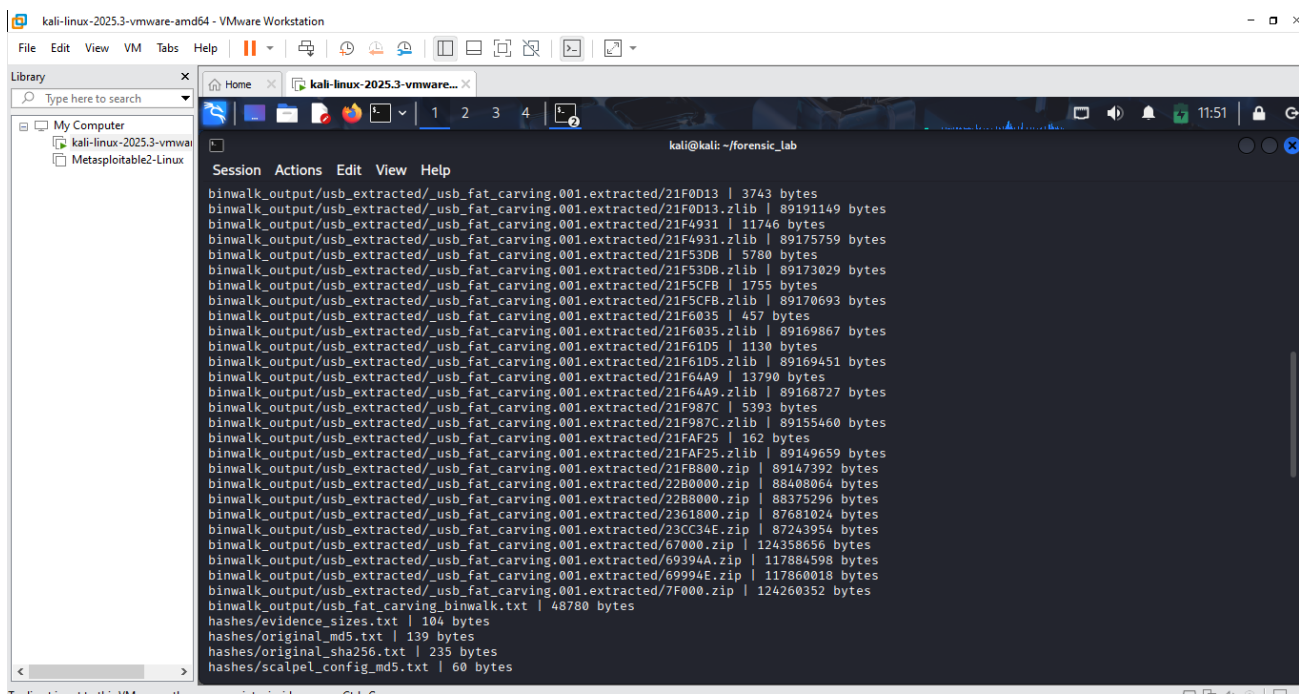
find hashes notes tsk_output binwalk_output scalpel_output \
-type f -printf '%p | %s bytes\n' \
| sort | tee notes/final_evidence_inventory.txt
binwalk_output/Ch01InChap01_binwalk.txt | 125 bytes
binwalk_output/J_ub_low_binwalk.txt | 374 bytes
binwalk_output/manual_ooxml_421888.zip | 97702 bytes
binwalk_output/usb_extracted/_usb_fat_carving.001.extracted/219C9F0 | 1518 bytes
binwalk_output/usb_extracted/_usb_fat_carving.001.extracted/219C9F0.zlib | 89536016 bytes
binwalk_output/usb_extracted/_usb_fat_carving.001.extracted/219D1CA | 185 bytes
binwalk_output/usb_extracted/_usb_fat_carving.001.extracted/219D1CA.zlib | 89534006 bytes
binwalk_output/usb_extracted/_usb_fat_carving.001.extracted/219D389 | 7519 bytes
binwalk_output/usb_extracted/_usb_fat_carving.001.extracted/219D389.zlib | 89533511 bytes
binwalk_output/usb_extracted/_usb_fat_carving.001.extracted/219DAAA | 8345 bytes
binwalk_output/usb_extracted/_usb_fat_carving.001.extracted/219DAAA.zlib | 89531734 bytes
binwalk_output/usb_extracted/_usb_fat_carving.001.extracted/219F9DF | 12069 bytes
binwalk_output/usb_extracted/_usb_fat_carving.001.extracted/219F9DF.zlib | 89523745 bytes
binwalk_output/usb_extracted/_usb_fat_carving.001.extracted/21A2713 | 10403 bytes
binwalk_output/usb_extracted/_usb_fat_carving.001.extracted/21A2713.zlib | 89512173 bytes
binwalk_output/usb_extracted/_usb_fat_carving.001.extracted/21AADF8 | 11170 bytes
binwalk_output/usb_extracted/_usb_fat_carving.001.extracted/21AADF8.zlib | 89502216 bytes
binwalk_output/usb_extracted/_usb_fat_carving.001.extracted/21A5A81 | 16752 bytes
binwalk_output/usb_extracted/_usb_fat_carving.001.extracted/21A5A81.zlib | 89499007 bytes
binwalk_output/usb_extracted/_usb_fat_carving.001.extracted/21A9970 | 15919 bytes
binwalk_output/usb_extracted/_usb_fat_carving.001.extracted/21A9970.zlib | 89482896 bytes
binwalk_output/usb_extracted/_usb_fat_carving.001.extracted/21AD4F4 | 25841 bytes
binwalk_output/usb_extracted/_usb_fat_carving.001.extracted/21AD4F4.zlib | 89467660 bytes
binwalk_output/usb_extracted/_usb_fat_carving.001.extracted/21B35DD | 5418 bytes
binwalk_output/usb_extracted/_usb_fat_carving.001.extracted/21B35DD.zlib | 89442851 bytes
```

To direct input to this VM, move the mouse pointer inside or press Ctrl+G.

Fig 7.3 Evidence packaging.

Explanation

An inventory was generated for the hashes, notes, TSK output, Binwalk output, and Scalpel results. This began the process of documenting exactly which supporting files had been generated during the investigation.



```
kali@kali: ~/forensic_lab

binwalk_output/usb_extracted/_usb_fat_carving.001.extracted/21F0D13 | 3743 bytes
binwalk_output/usb_extracted/_usb_fat_carving.001.extracted/21F0D13.zlib | 89191149 bytes
binwalk_output/usb_extracted/_usb_fat_carving.001.extracted/21F4931 | 11746 bytes
binwalk_output/usb_extracted/_usb_fat_carving.001.extracted/21F4931.zlib | 89175759 bytes
binwalk_output/usb_extracted/_usb_fat_carving.001.extracted/21F53D8 | 5780 bytes
binwalk_output/usb_extracted/_usb_fat_carving.001.extracted/21F53D8.zlib | 89173029 bytes
binwalk_output/usb_extracted/_usb_fat_carving.001.extracted/21F5CFB | 1755 bytes
binwalk_output/usb_extracted/_usb_fat_carving.001.extracted/21F5CFB.zlib | 89170693 bytes
binwalk_output/usb_extracted/_usb_fat_carving.001.extracted/21F6035 | 457 bytes
binwalk_output/usb_extracted/_usb_fat_carving.001.extracted/21F6035.zlib | 89169867 bytes
binwalk_output/usb_extracted/_usb_fat_carving.001.extracted/21F61D5 | 1130 bytes
binwalk_output/usb_extracted/_usb_fat_carving.001.extracted/21F61D5.zlib | 89169451 bytes
binwalk_output/usb_extracted/_usb_fat_carving.001.extracted/21F64A9 | 13790 bytes
binwalk_output/usb_extracted/_usb_fat_carving.001.extracted/21F64A9.zlib | 89168727 bytes
binwalk_output/usb_extracted/_usb_fat_carving.001.extracted/21F987C | 5393 bytes
binwalk_output/usb_extracted/_usb_fat_carving.001.extracted/21F987C.zlib | 89155460 bytes
binwalk_output/usb_extracted/_usb_fat_carving.001.extracted/21FAF25 | 162 bytes
binwalk_output/usb_extracted/_usb_fat_carving.001.extracted/21FAF25.zlib | 89149659 bytes
binwalk_output/usb_extracted/_usb_fat_carving.001.extracted/21F8000.zip | 89147392 bytes
binwalk_output/usb_extracted/_usb_fat_carving.001.extracted/22B8000.zip | 88408064 bytes
binwalk_output/usb_extracted/_usb_fat_carving.001.extracted/22B8000.zip | 88375296 bytes
binwalk_output/usb_extracted/_usb_fat_carving.001.extracted/2361800.zip | 87681024 bytes
binwalk_output/usb_extracted/_usb_fat_carving.001.extracted/23CC34E.zip | 87243954 bytes
binwalk_output/usb_extracted/_usb_fat_carving.001.extracted/67000.zip | 124358656 bytes
binwalk_output/usb_extracted/_usb_fat_carving.001.extracted/69394A.zip | 117884598 bytes
binwalk_output/usb_extracted/_usb_fat_carving.001.extracted/69994E.zip | 117860018 bytes
binwalk_output/usb_extracted/_usb_fat_carving.001.extracted/7F000.zip | 124260352 bytes
binwalk_output/usb_fat_carving_binwalk.txt | 48780 bytes
hashes/evidence_sizes.txt | 104 bytes
hashes/original_md5.txt | 139 bytes
hashes/original_sha256.txt | 235 bytes
hashes/scalpel_config_md5.txt | 60 bytes
```

To direct input to this VM, move the mouse pointer inside or press Ctrl+G.

Fig 7.4 Evidence packaging page 1.

Explanation

The evidence inventory continued through the large Binwalk extraction output and other generated files. File

paths and sizes were recorded so that the contents of the laboratory workspace could be reviewed.

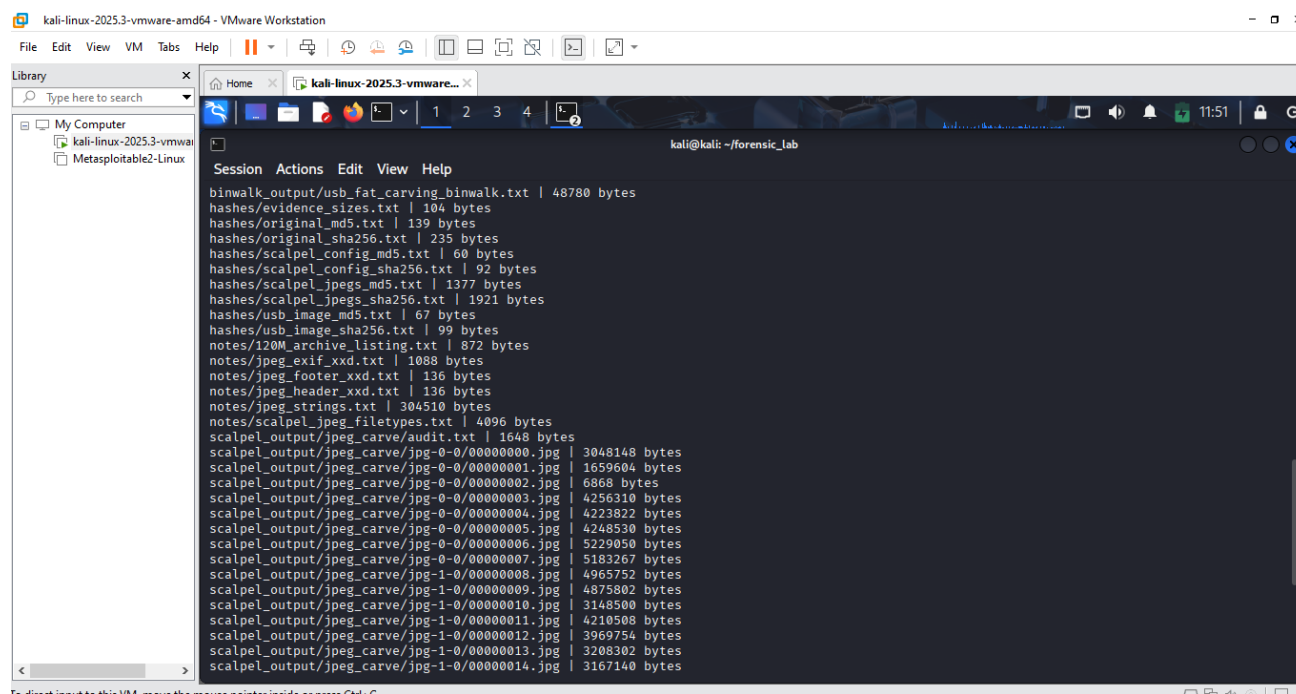


Fig 7.5 Evidence packaging page 2.

Explanation

This screenshot continues the inventory and includes hash files, notes, Scalpel output, and other supporting material. It demonstrates that the recovered and generated evidence was being catalogued before packaging.

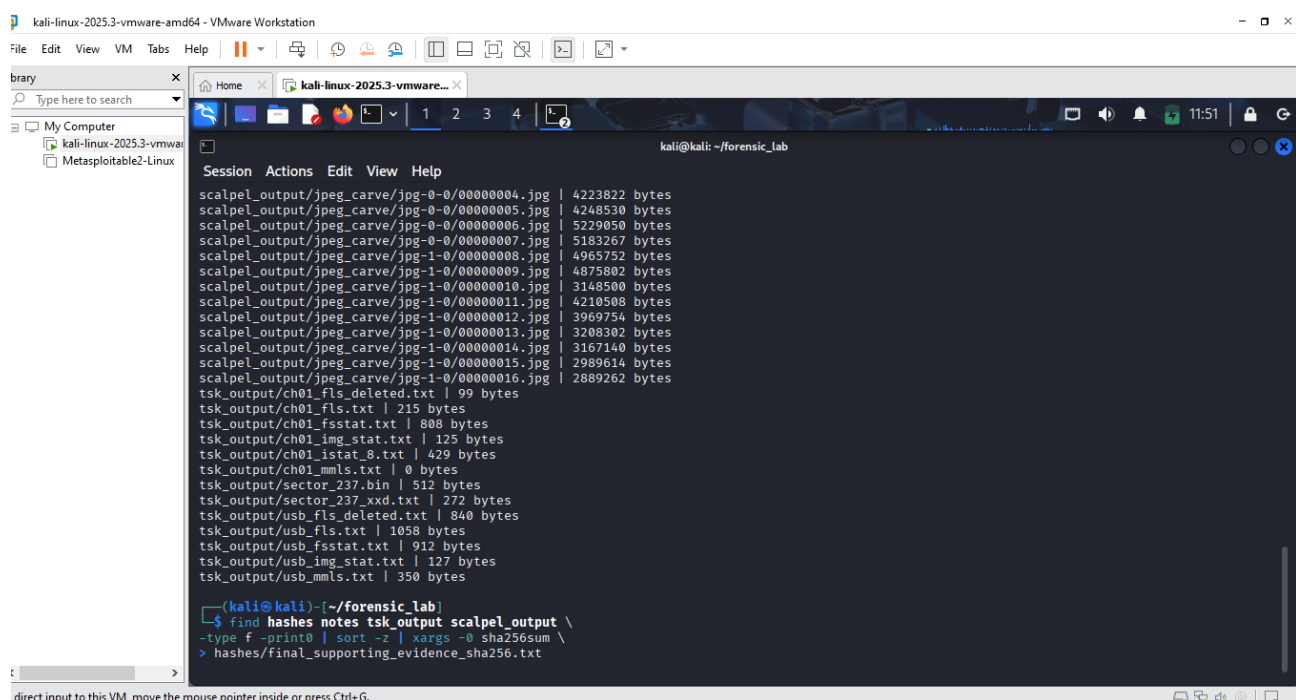
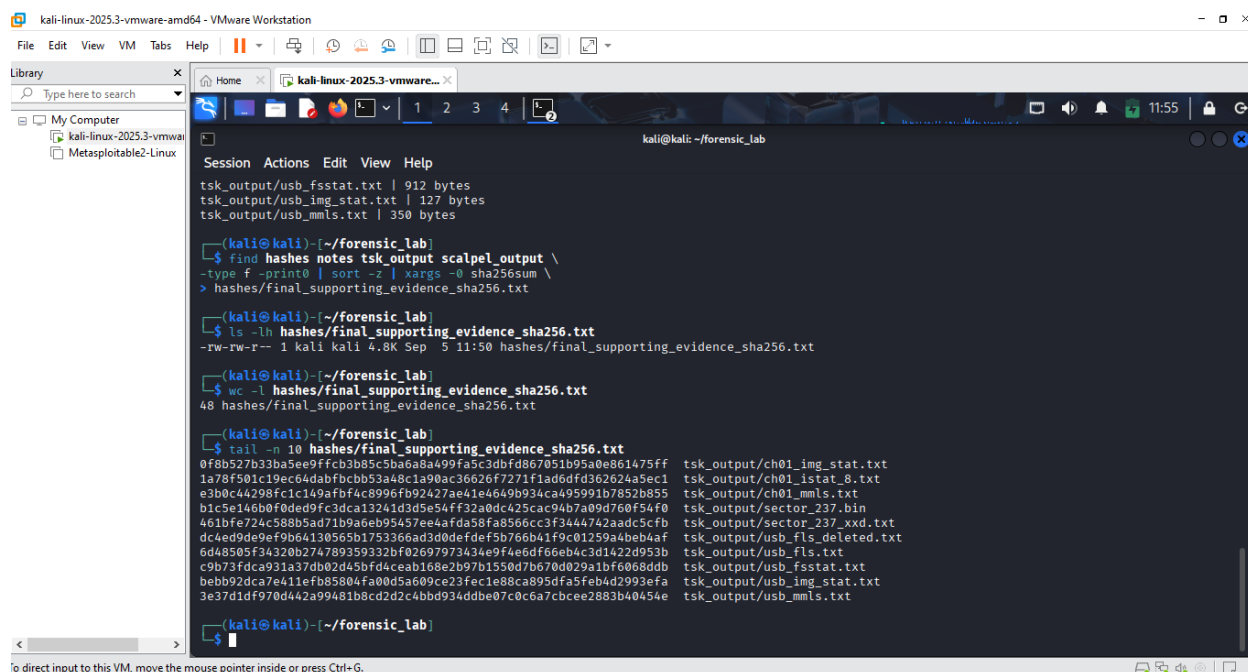


Fig 7.6 Evidence packaging page 3.

Explanation

The inventory reaches the TSK output files and additional supporting records. A SHA-256 manifest command was then started to generate integrity values for the collected supporting evidence.



```
kali@kali: ~/forensic_lab
$ find hashes notes tsk_output scalpel_output \
-type f -print0 | sort -z | xargs -0 sha256sum \
> hashes/final_supporting_evidence_sha256.txt

(kali@kali)~/forensic_lab
$ ls -lh hashes/final_supporting_evidence_sha256.txt
-rw-rw-r-- 1 kali kali 4.8K Sep  5 11:50 hashes/final_supporting_evidence_sha256.txt

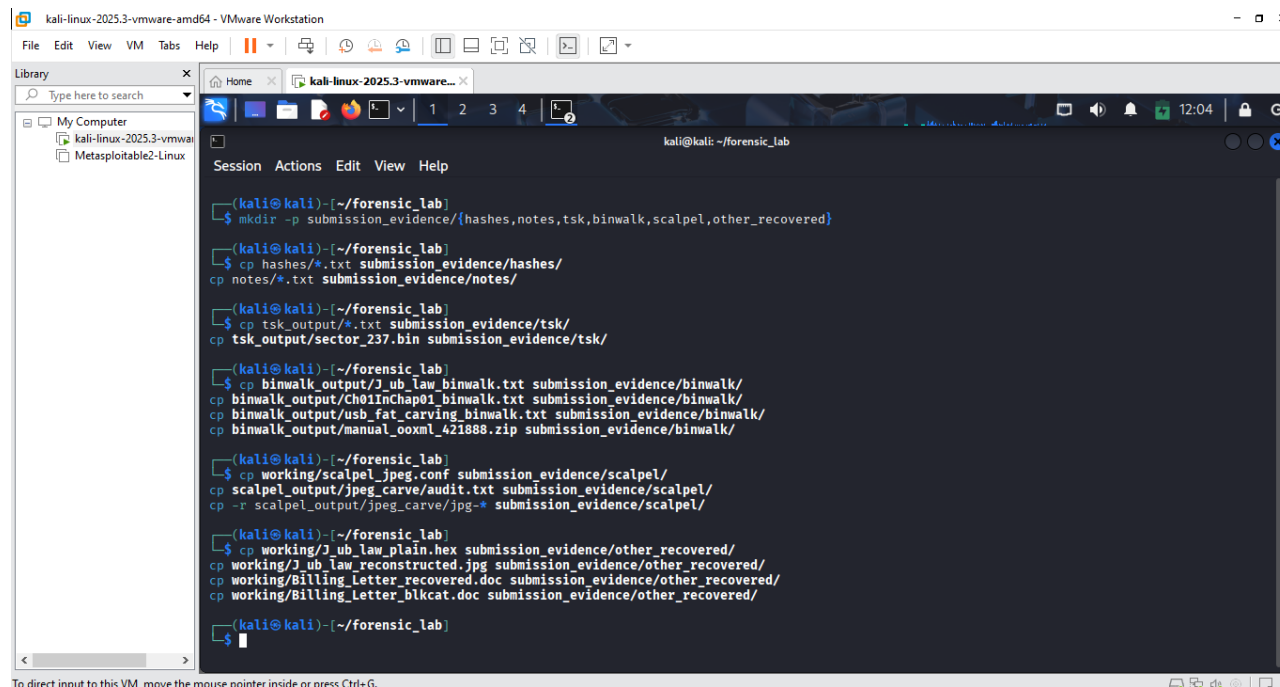
(kali@kali)~/forensic_lab
$ wc -l hashes/final_supporting_evidence_sha256.txt
48 hashes/final_supporting_evidence_sha256.txt

(kali@kali)~/forensic_lab
$ tail -n 10 hashes/final_supporting_evidence_sha256.txt
0f8b527b33ba5ee9ffc3b3b85c5ba6a8a499fa5c3dbfd867051b95a0e861475ff tsk_output/ch01_img_stat.txt
1a78f501c19ec644abfbcbb53a48c1a90ac36626f7271f1ad6dfd362624a5ec1 tsk_output/ch01_istat.8.txt
ae30bc44298f8c1c149afb4c8996fb92427ae41e4649b93ca495991b7852b855 tsk_output/ch01_mmls.txt
b1c5e146b0f0de9fc3dca13241d3d5e54ff32a0dc425cac94b7a09d760f54f0 tsk_output/sector_237.bin
461bfe724c588b5ad71b9a6eb95457ee4afda58fa8566cc3f3444742aad5c5fb tsk_output/sector_237_xdd.txt
dc4ed9de9efb64130565b1753366ad3d0defdf5b766b41f9c01259a4beb4af tsk_output/usb_fls_deleted.txt
6d48505f34320b274789359332bf02697973434e9f4e6df66eb4c3d1422d953b tsk_output/usb_fls.txt
c9b73fda931a37db02d45bdf4ceab168e2b97b1550d7b670d029a1bf6068ddb tsk_output/usb_fsstat.txt
bebb92dca7e411ef85804fa00d5a609ce23fec1e88ca895dfa5feb4d2993efa tsk_output/usb_img_stat.txt
3e37d1df970d442a99481b8cd2d2c4bbd934ddbe07c0c6a7cbcee2883b40454e tsk_output/usb_mmls.txt
```

Fig 7.7 Evidence packaging page 4.

Explanation

The completed supporting-evidence SHA-256 file was checked by displaying its size, number of entries, and final records. This provided confirmation that the supporting material had been included in a cryptographic manifest.



```
kali@kali: ~/forensic_lab
$ mkdir -p submission_evidence/{hashes,notes,tsk,binwalk,scalpel,other_recovered}

(kali@kali)~/forensic_lab
$ cp hashes/*.txt submission_evidence/hashes/
cp notes/*.txt submission_evidence/notes/

(kali@kali)~/forensic_lab
$ cp tsk_output/*.txt submission_evidence/tsk/
cp tsk_output/sector_237.bin submission_evidence/tsk/

(kali@kali)~/forensic_lab
$ cp binwalk_output/J_ub_law_binwalk.txt submission_evidence/binwalk/
cp binwalk_output/Ch01Inchap01_binwalk.txt submission_evidence/binwalk/
cp binwalk_output/usb_fat_carving_binwalk.txt submission_evidence/binwalk/
cp binwalk_output/manual_ooxml_421888.zip submission_evidence/binwalk/

(kali@kali)~/forensic_lab
$ cp working/scalpel_jpeg.conf submission_evidence/scalpel/
cp scalpel_output/jpeg_carve/audit.txt submission_evidence/scalpel/
cp -r scalpel_output/jpeg_carve/jpg-* submission_evidence/scalpel/

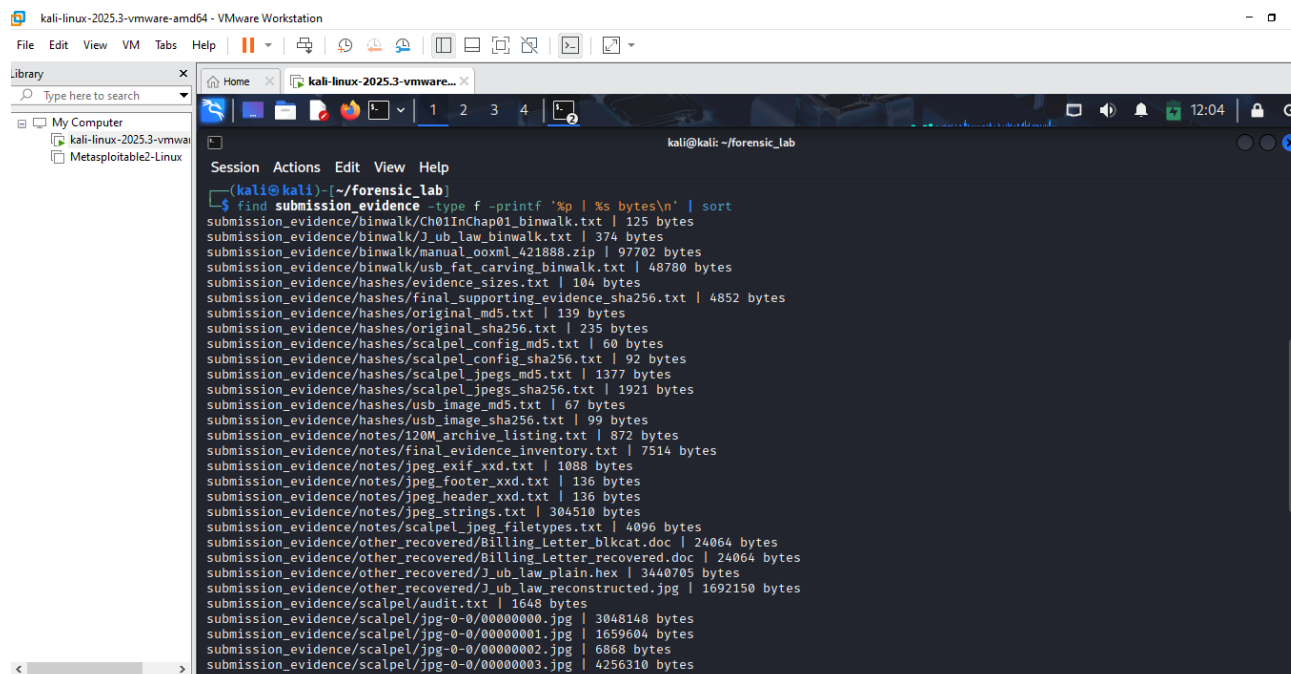
(kali@kali)~/forensic_lab
$ cp working/J_ub_law_plain.hex submission_evidence/other_recovered/
cp working/J_ub_law_reconstructed.jpg submission_evidence/other_recovered/
cp working/Billing_Letter_recovered.doc submission_evidence/other_recovered/
cp working/Billing_Letter_blkcat.doc submission_evidence/other_recovered/

(kali@kali)~/forensic_lab
$
```

Fig 7.8 Creating a clean submission-evidence folder.

Explanation

A dedicated `submission_evidence` structure was created and populated with selected hashes, notes, TSK output, Binwalk results, Scalpel evidence, and recovered files. This separated the material intended for submission from unnecessary temporary or automatically generated data.



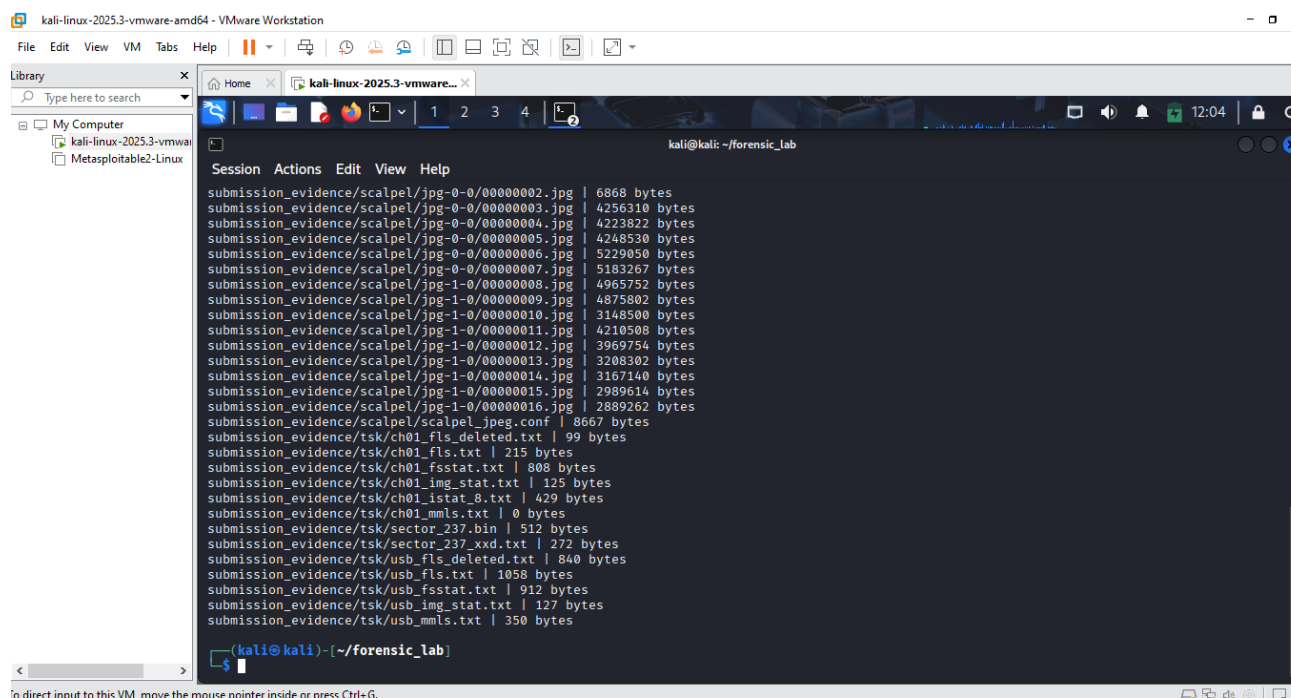
```
kali-linux-2025.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
Library
Type here to search
My Computer
kali-linux-2025.3-vmware-amd64
Metasploitable2-Linux

Session Actions Edit View Help
(kali@kali)~[/forensic_lab]
$ find submission_evidence -type f -printf '%p | %s bytes\n' | sort
submission_evidence/binwalk/Ch01InChap01_binwalk.txt | 125 bytes
submission_evidence/binwalk/1_ub_law_binwalk.txt | 374 bytes
submission_evidence/binwalk/manual_ooxml_421888.zip | 97702 bytes
submission_evidence/binwalk/usb_fat_carving_binwalk.txt | 48780 bytes
submission_evidence/hashes/evidence_sizes.txt | 104 bytes
submission_evidence/hashes/final_supporting_evidence_sha256.txt | 4852 bytes
submission_evidence/hashes/original_md5.txt | 139 bytes
submission_evidence/hashes/original_sha256.txt | 235 bytes
submission_evidence/hashes/scalpel_config_md5.txt | 60 bytes
submission_evidence/hashes/scalpel_config_sha256.txt | 92 bytes
submission_evidence/hashes/scalpel_jpegs_md5.txt | 1377 bytes
submission_evidence/hashes/scalpel_jpegs_sha256.txt | 1921 bytes
submission_evidence/hashes/usb_image_md5.txt | 67 bytes
submission_evidence/hashes/usb_image_sha256.txt | 99 bytes
submission_evidence/notes/120M_archive_listing.txt | 872 bytes
submission_evidence/notes/Final_evidence_inventory.txt | 7514 bytes
submission_evidence/notes/jpeg_exif_xxd.txt | 1088 bytes
submission_evidence/notes/jpeg_footer_xxd.txt | 136 bytes
submission_evidence/notes/jpeg_header_xxd.txt | 136 bytes
submission_evidence/notes/jpeg_strings.txt | 304510 bytes
submission_evidence/notes/scalpel_jpeg_filetypes.txt | 4096 bytes
submission_evidence/other_recovered/Billing_Letter_blkcat.doc | 24064 bytes
submission_evidence/other_recovered/Billing_Letter_recovered.doc | 24064 bytes
submission_evidence/other_recovered/1_ub_law_plain.hex | 3440705 bytes
submission_evidence/other_recovered/1_ub_law_reconstructed.jpg | 1692150 bytes
submission_evidence/scalpel/audit.txt | 1648 bytes
submission_evidence/scalpel/jpg-0-0/00000000.jpg | 3048148 bytes
submission_evidence/scalpel/jpg-0-0/00000001.jpg | 1659604 bytes
submission_evidence/scalpel/jpg-0-0/00000002.jpg | 6868 bytes
submission_evidence/scalpel/jpg-0-0/00000003.jpg | 4256310 bytes
```

Fig 7.9 Final package.

Explanation

The contents of the clean submission folder were listed with their paths and sizes. This provided a final review of the evidence selected for inclusion in the supporting archive.



```
kali-linux-2025.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
Library
Type here to search
My Computer
kali-linux-2025.3-vmware-amd64
Metasploitable2-Linux

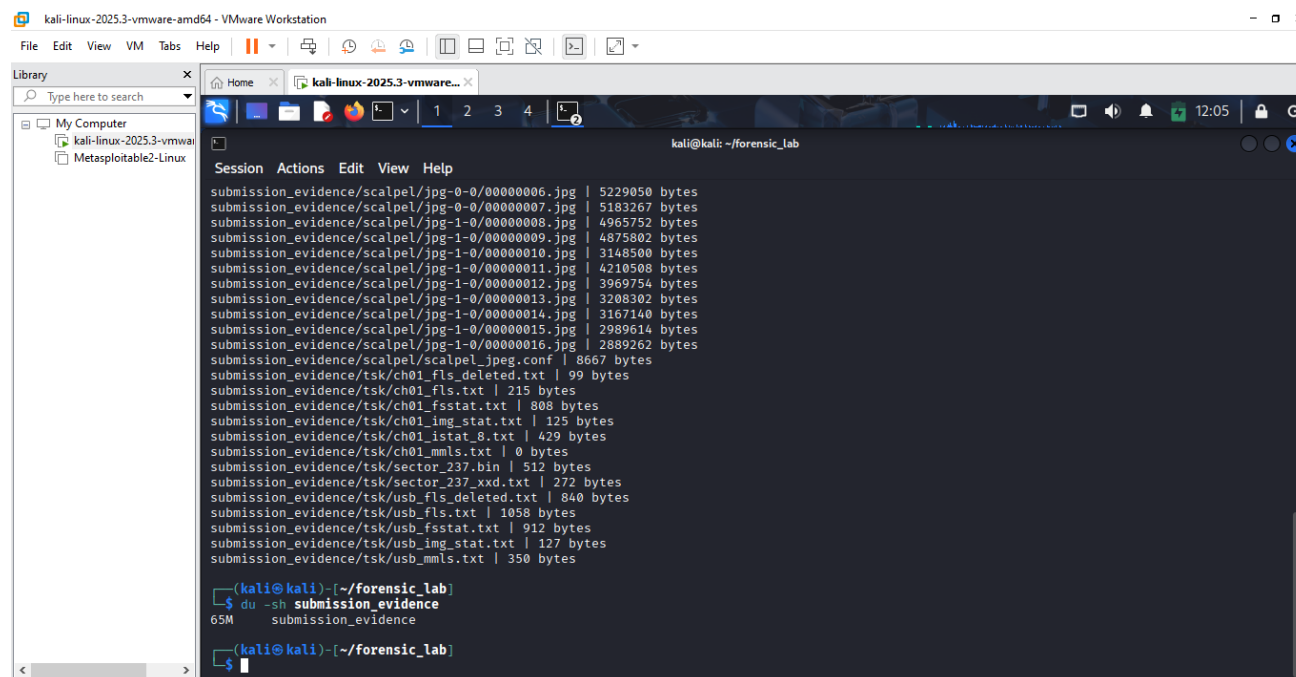
Session Actions Edit View Help
(kali@kali)~[/forensic_lab]
$ find submission_evidence -type f -printf '%p | %s bytes\n' | sort
submission_evidence/scalpel/jpg-0-0/00000002.jpg | 6868 bytes
submission_evidence/scalpel/jpg-0-0/00000003.jpg | 4256310 bytes
submission_evidence/scalpel/jpg-0-0/00000004.jpg | 4223822 bytes
submission_evidence/scalpel/jpg-0-0/00000005.jpg | 4248530 bytes
submission_evidence/scalpel/jpg-0-0/00000006.jpg | 5229050 bytes
submission_evidence/scalpel/jpg-0-0/00000007.jpg | 5183267 bytes
submission_evidence/scalpel/jpg-1-0/00000008.jpg | 4965752 bytes
submission_evidence/scalpel/jpg-1-0/00000009.jpg | 4875802 bytes
submission_evidence/scalpel/jpg-1-0/00000010.jpg | 3148500 bytes
submission_evidence/scalpel/jpg-1-0/00000011.jpg | 4210508 bytes
submission_evidence/scalpel/jpg-1-0/00000012.jpg | 3969754 bytes
submission_evidence/scalpel/jpg-1-0/00000013.jpg | 3208302 bytes
submission_evidence/scalpel/jpg-1-0/00000014.jpg | 3167140 bytes
submission_evidence/scalpel/jpg-1-0/00000015.jpg | 2989614 bytes
submission_evidence/scalpel/jpg-1-0/00000016.jpg | 2889262 bytes
submission_evidence/scalpel/scalpel_jpeg.conf | 8667 bytes
submission_evidence/tsk/ch01_fls_deleted.txt | 99 bytes
submission_evidence/tsk/ch01_fls.txt | 215 bytes
submission_evidence/tsk/ch01_fsstat.txt | 808 bytes
submission_evidence/tsk/ch01_img_stat.txt | 125 bytes
submission_evidence/tsk/ch01_istat.8.txt | 429 bytes
submission_evidence/tsk/ch01_mmls.txt | 0 bytes
submission_evidence/tsk/sector_237.bin | 512 bytes
submission_evidence/tsk/sector_237_xdd.txt | 272 bytes
submission_evidence/tsk/usb_fls_deleted.txt | 840 bytes
submission_evidence/tsk/usb_fls.txt | 1058 bytes
submission_evidence/tsk/usb_fsstat.txt | 912 bytes
submission_evidence/tsk/usb_img_stat.txt | 127 bytes
submission_evidence/tsk/usb_mmls.txt | 350 bytes
(kali@kali)~[/forensic_lab]
$
```

In direct input to this VM move the mouse pointer inside or press Ctrl+G.

Fig 8.0 Final package page 1.

Explanation

The package listing continued through the carved JPEG files and TSK outputs. It confirmed that recovered images, audit records, and forensic command outputs had been included.



```
kali@kali: ~/forensic_lab
Session Actions Edit View Help
submission_evidence/scalpel/jpg-0-0/00000006.jpg | 5229050 bytes
submission_evidence/scalpel/jpg-0-0/00000007.jpg | 5183267 bytes
submission_evidence/scalpel/jpg-1-0/00000008.jpg | 4965752 bytes
submission_evidence/scalpel/jpg-1-0/00000009.jpg | 4875802 bytes
submission_evidence/scalpel/jpg-1-0/00000010.jpg | 3148500 bytes
submission_evidence/scalpel/jpg-1-0/00000011.jpg | 4210508 bytes
submission_evidence/scalpel/jpg-1-0/00000012.jpg | 3969754 bytes
submission_evidence/scalpel/jpg-1-0/00000013.jpg | 3208302 bytes
submission_evidence/scalpel/jpg-1-0/00000014.jpg | 3167140 bytes
submission_evidence/scalpel/jpg-1-0/00000015.jpg | 2989614 bytes
submission_evidence/scalpel/jpg-1-0/00000016.jpg | 2889262 bytes
submission_evidence/scalpel/scalpel_jpeg.conf | 8667 bytes
submission_evidence/tsk/ch01_flis_deleted.txt | 99 bytes
submission_evidence/tsk/ch01_flis.txt | 215 bytes
submission_evidence/tsk/ch01_fsstat.txt | 808 bytes
submission_evidence/tsk/ch01_img_stat.txt | 125 bytes
submission_evidence/tsk/ch01_istat_8.txt | 429 bytes
submission_evidence/tsk/ch01_mmls.txt | 0 bytes
submission_evidence/tsk/sector_237.bin | 512 bytes
submission_evidence/tsk/sector_237_xdd.txt | 272 bytes
submission_evidence/tsk/usb_flis_deleted.txt | 840 bytes
submission_evidence/tsk/usb_flis.txt | 1058 bytes
submission_evidence/tsk/usb_fsstat.txt | 912 bytes
submission_evidence/tsk/usb_img_stat.txt | 127 bytes
submission_evidence/tsk/usb_mmls.txt | 350 bytes

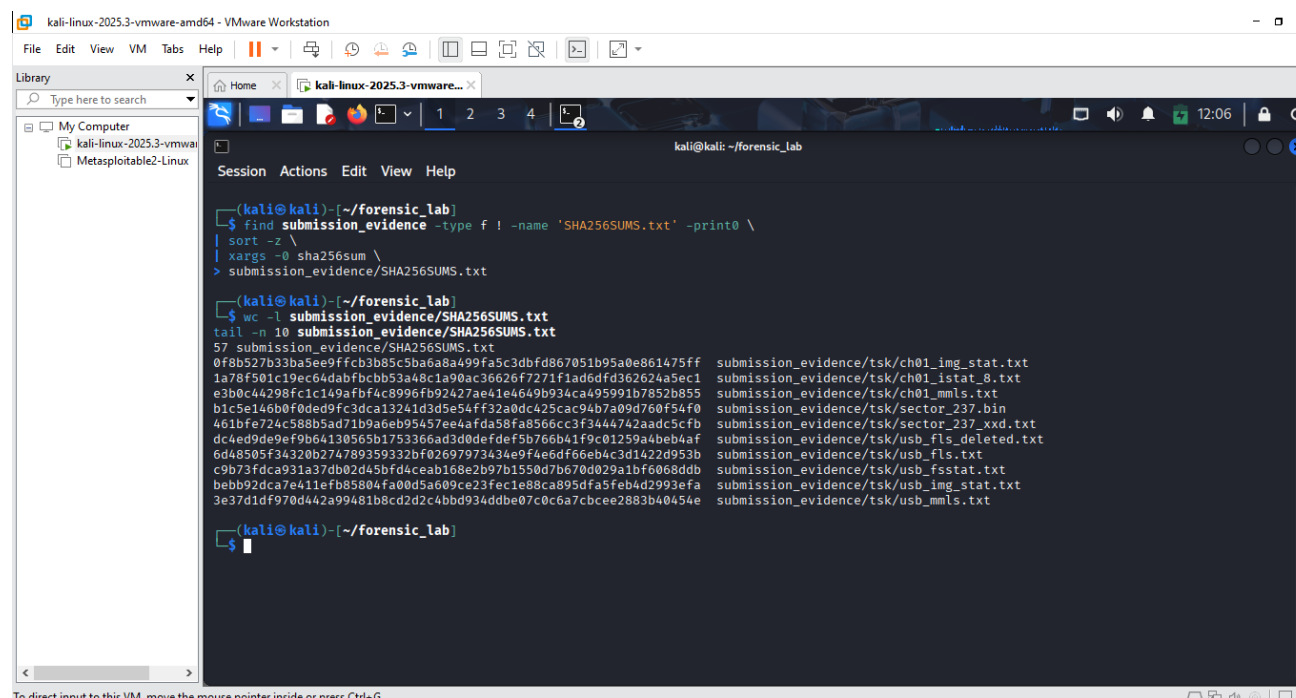
(kali@kali)~/forensic_lab
$ du -sh submission_evidence
65M submission_evidence

(kali@kali)~/forensic_lab
$
```

Fig 8.1 Final package page 2.

Explanation

The remainder of the final evidence directory was displayed and its overall size was checked. This completed the inventory stage before the final archive was created.



```
kali@kali: ~/forensic_lab
Session Actions Edit View Help

(kali@kali)~/forensic_lab
$ find submission_evidence -type f ! -name 'SHA256SUMS.txt' -print0 \
| sort -z \
| xargs -0 sha256sum \
> submission_evidence/SHA256SUMS.txt

(kali@kali)~/forensic_lab
$ wc -l submission_evidence/SHA256SUMS.txt
tail -n 10 submission_evidence/SHA256SUMS.txt
57 submission_evidence/SHA256SUMS.txt
0f8b527b33ba5e9ffcb3b85c5ba6a8a490fa5c3dbfd867051b95a0e861475ff submission_evidence/tsk/ch01_img_stat.txt
1a78f501c19ec64dabfbcbb53a48c1a90ac36626f7271f1ad6dfd362624a5ec1 submission_evidence/tsk/ch01_istat_8.txt
e3b0c44298fc1c149afb4c8996fb92427ae41e4649b934ca495991b7852b855 submission_evidence/tsk/ch01_mmls.txt
b1c5e146b0f0de9fc3dca13241d3d5e54ff32a0dc425cac94b7a09d760f54f0 submission_evidence/tsk/sector_237.bin
461bfe724c588b5ad71b9a6eb95457ee4afda58fa8566cc3f3444742aadcc5cfb submission_evidence/tsk/sector_237_xdd.txt
dc4ed9de9ef9b64130565b1753366ad3d0defdf5b766b41f9c01259a4beb4af submission_evidence/tsk/usb_flis_deleted.txt
6d48505f34320b274789359332bf02697973434e9f4e6df66eb4c3d1422d953b submission_evidence/tsk/usb_flis.txt
c9b73fdca931a37db0245b5fd4ceab168e2b97b1550d7b670d029a1bf6068ddb submission_evidence/tsk/usb_fsstat.txt
bebb92dca7e411efb85804fa00d5a609ce23fec1e88ca895dfa5feb4d2993efa submission_evidence/tsk/usb_img_stat.txt
3e37d1df970d442a99481b8cd2d2c4bbd934ddbe07c0c6a7cbcee2883b40454e submission_evidence/tsk/usb_mmls.txt

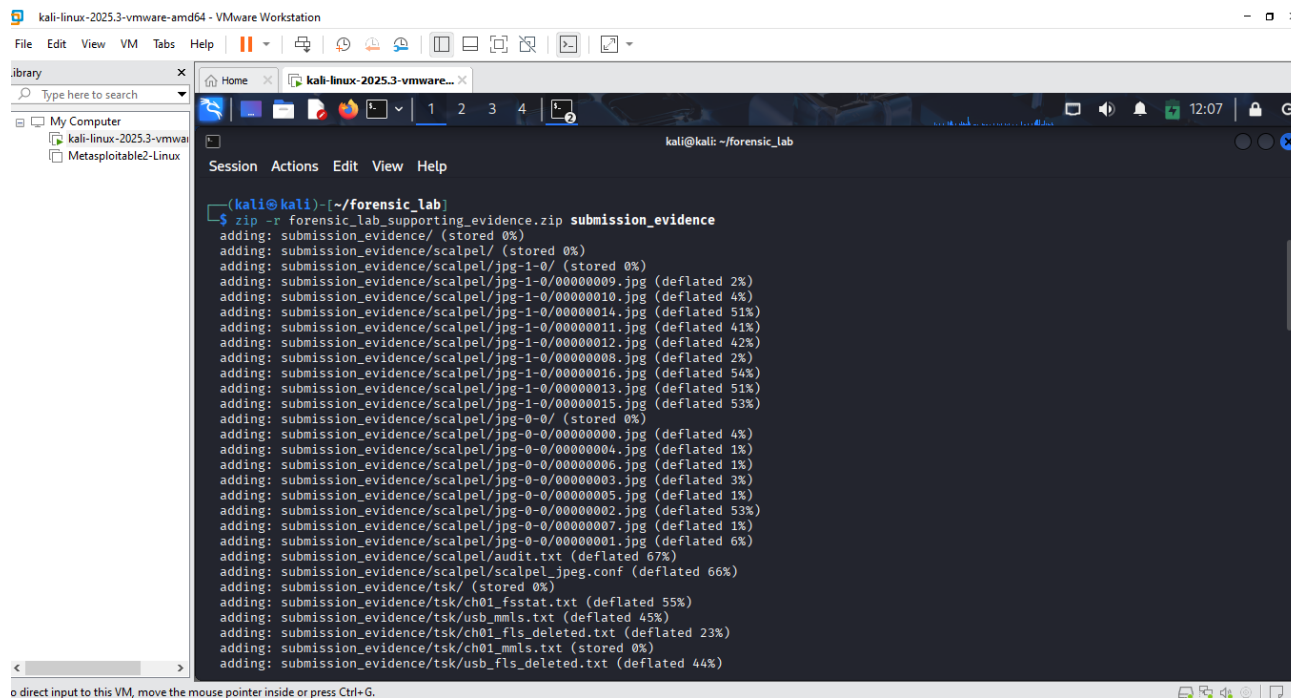
(kali@kali)~/forensic_lab
$
```

To direct input to this VM, move the mouse pointer inside or press Ctrl+G.

Fig 8.2 Hashing the submission_evidence.

Explanation

A SHA-256 manifest was generated specifically for the files inside the final submission folder. The displayed entries show that the packaged supporting evidence received individual integrity hashes before compression.

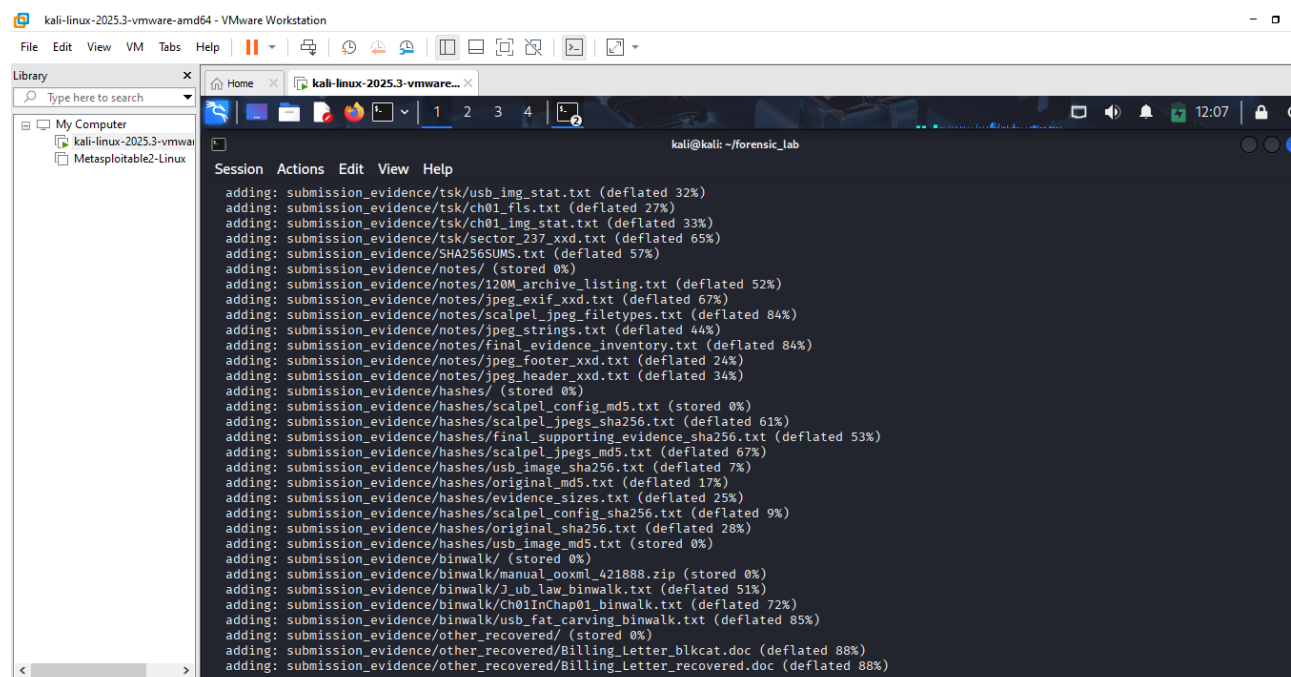


```
(kali@kali)-[~/forensic_lab]
$ zip -r forensic_lab_supporting_evidence.zip submission_evidence
adding: submission_evidence/ (stored 0%)
adding: submission_evidence/scalpel/ (stored 0%)
adding: submission_evidence/scalpel/jpg-1-0/ (stored 0%)
adding: submission_evidence/scalpel/jpg-1-0/00000009.jpg (deflated 2%)
adding: submission_evidence/scalpel/jpg-1-0/00000010.jpg (deflated 4%)
adding: submission_evidence/scalpel/jpg-1-0/00000014.jpg (deflated 51%)
adding: submission_evidence/scalpel/jpg-1-0/00000011.jpg (deflated 41%)
adding: submission_evidence/scalpel/jpg-1-0/00000012.jpg (deflated 42%)
adding: submission_evidence/scalpel/jpg-1-0/00000008.jpg (deflated 2%)
adding: submission_evidence/scalpel/jpg-1-0/00000016.jpg (deflated 54%)
adding: submission_evidence/scalpel/jpg-1-0/00000013.jpg (deflated 51%)
adding: submission_evidence/scalpel/jpg-1-0/00000015.jpg (deflated 53%)
adding: submission_evidence/scalpel/jpg-0-0/ (stored 0%)
adding: submission_evidence/scalpel/jpg-0-0/00000000.jpg (deflated 4%)
adding: submission_evidence/scalpel/jpg-0-0/00000004.jpg (deflated 1%)
adding: submission_evidence/scalpel/jpg-0-0/00000006.jpg (deflated 1%)
adding: submission_evidence/scalpel/jpg-0-0/00000003.jpg (deflated 3%)
adding: submission_evidence/scalpel/jpg-0-0/00000005.jpg (deflated 1%)
adding: submission_evidence/scalpel/jpg-0-0/00000002.jpg (deflated 53%)
adding: submission_evidence/scalpel/jpg-0-0/00000007.jpg (deflated 1%)
adding: submission_evidence/scalpel/jpg-0-0/00000001.jpg (deflated 6%)
adding: submission_evidence/scalpel/audit.txt (deflated 67%)
adding: submission_evidence/scalpel/scalpel_jpeg.conf (deflated 66%)
adding: submission_evidence/tsk/ (stored 0%)
adding: submission_evidence/tsk/ch01_fsstat.txt (deflated 55%)
adding: submission_evidence/tsk/usb_mmls.txt (deflated 45%)
adding: submission_evidence/tsk/ch01_fls_deleted.txt (deflated 23%)
adding: submission_evidence/tsk/ch01_mmls.txt (stored 0%)
adding: submission_evidence/tsk/usb_fls_deleted.txt (deflated 44%)
```

Fig 8.3 Creating the required evidence ZIP.

Explanation

The clean submission_evidence directory was compressed into the required supporting-evidence ZIP archive. The command output shows the individual files being added to the archive.

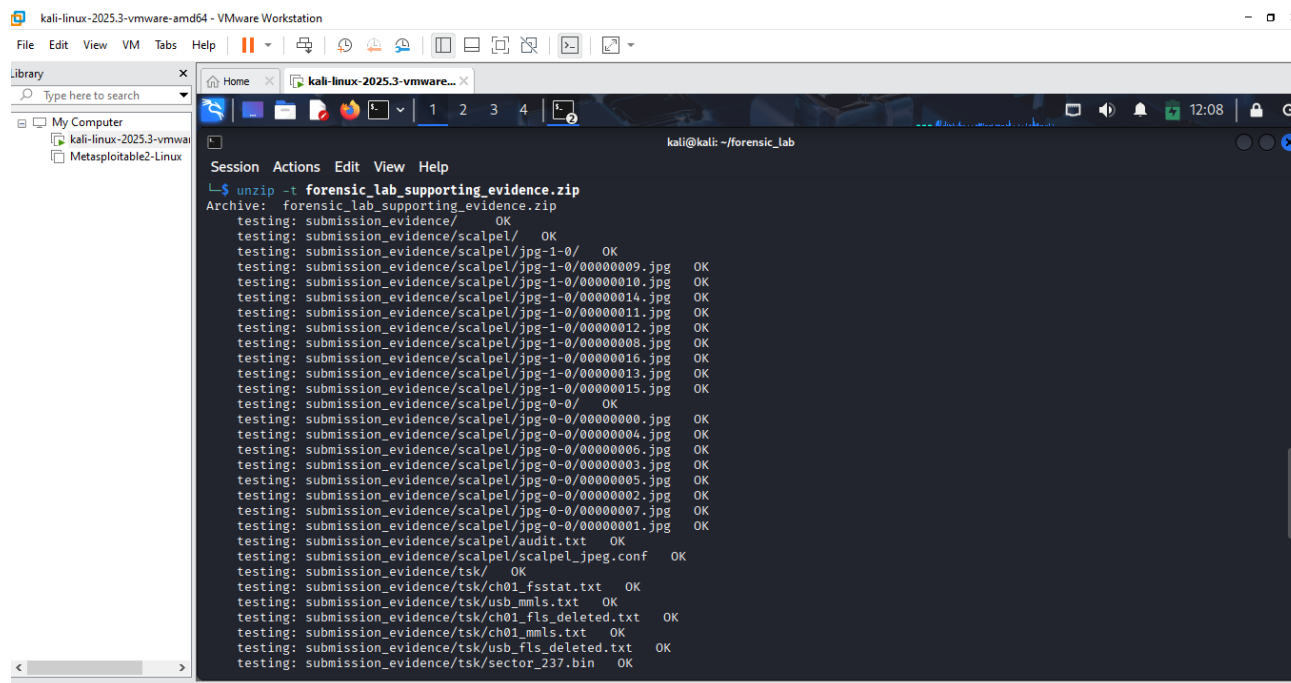


```
(kali@kali)-[~/forensic_lab]
$ zip -r forensic_lab_supporting_evidence.zip submission_evidence
adding: submission_evidence/tsk/usb_img_stat.txt (deflated 32%)
adding: submission_evidence/tsk/ch01_fls.txt (deflated 27%)
adding: submission_evidence/tsk/ch01_img_stat.txt (deflated 33%)
adding: submission_evidence/tsk/sector_237_xxd.txt (deflated 65%)
adding: submission_evidence/SHA256SUMS.txt (deflated 57%)
adding: submission_evidence/notes/ (stored 0%)
adding: submission_evidence/notes/120M_archive_listing.txt (deflated 52%)
adding: submission_evidence/notes/jpeg_exif_xxd.txt (deflated 67%)
adding: submission_evidence/notes/scalpel_jpeg_filetypes.txt (deflated 84%)
adding: submission_evidence/notes/jpeg_strings.txt (deflated 44%)
adding: submission_evidence/notes/Final_evidence_inventory.txt (deflated 84%)
adding: submission_evidence/notes/jpeg_footer_xxd.txt (deflated 24%)
adding: submission_evidence/notes/jpeg_header_xxd.txt (deflated 34%)
adding: submission_evidence/notes/ (stored 0%)
adding: submission_evidence/notes/scalpel_config_md5.txt (stored 0%)
adding: submission_evidence/notes/scalpel_jpgs_sha256.txt (deflated 61%)
adding: submission_evidence/notes/final_supporting_evidence_sha256.txt (deflated 53%)
adding: submission_evidence/notes/scalpel_jpgs_md5.txt (deflated 67%)
adding: submission_evidence/notes/usb_image_sha256.txt (deflated 7%)
adding: submission_evidence/notes/original_md5.txt (deflated 17%)
adding: submission_evidence/notes/evidence_sizes.txt (deflated 25%)
adding: submission_evidence/notes/scalpel_config_sha256.txt (deflated 9%)
adding: submission_evidence/notes/original_sha256.txt (deflated 28%)
adding: submission_evidence/notes/usb_image_md5.txt (stored 0%)
adding: submission_evidence/binwalk/ (stored 0%)
adding: submission_evidence/binwalk/manual_ooxml_421888.zip (stored 0%)
adding: submission_evidence/binwalk/J_ub_low_binwalk.txt (deflated 51%)
adding: submission_evidence/binwalk/Ch01InChap01_binwalk.txt (deflated 72%)
adding: submission_evidence/binwalk/usb_fat_carving_binwalk.txt (deflated 85%)
adding: submission_evidence/other_recovered/ (stored 0%)
adding: submission_evidence/other_recovered/Billing_Letter_blkcat.doc (deflated 88%)
adding: submission_evidence/other_recovered/Billing_Letter_recovered.doc (deflated 88%)
```

Fig 8.4 Creating the required evidence ZIP page 1.

Explanation

This screenshot continues the compression process as additional notes, hashes, Binwalk records, and recovered evidence were added to the ZIP. It documents the contents included in the final submission archive.

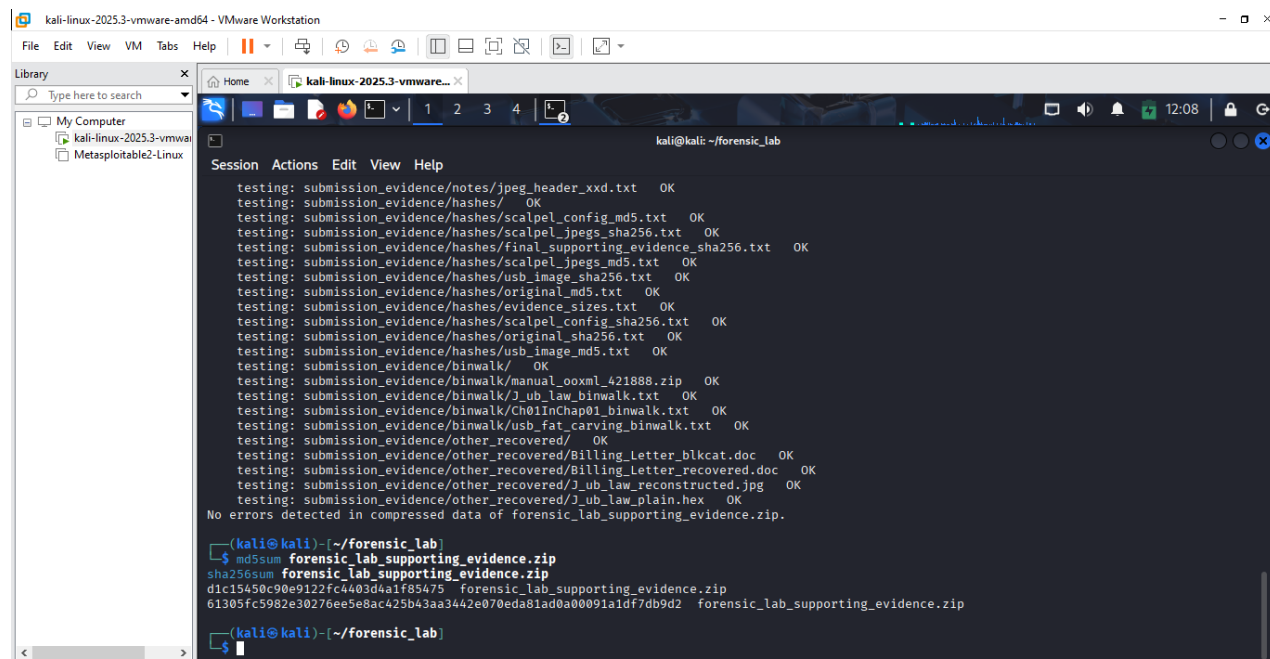


```
kali@kali: ~/forensic_lab
$ unzip -t forensic_lab_supporting_evidence.zip
Archive:  forensic_lab_supporting_evidence.zip
testing: submission_evidence/ OK
testing: submission_evidence/scalpel/ OK
testing: submission_evidence/scalpel/jpg-1-0/ OK
testing: submission_evidence/scalpel/jpg-1-0/00000009.jpg OK
testing: submission_evidence/scalpel/jpg-1-0/00000010.jpg OK
testing: submission_evidence/scalpel/jpg-1-0/00000014.jpg OK
testing: submission_evidence/scalpel/jpg-1-0/00000011.jpg OK
testing: submission_evidence/scalpel/jpg-1-0/00000012.jpg OK
testing: submission_evidence/scalpel/jpg-1-0/00000008.jpg OK
testing: submission_evidence/scalpel/jpg-1-0/00000016.jpg OK
testing: submission_evidence/scalpel/jpg-1-0/00000013.jpg OK
testing: submission_evidence/scalpel/jpg-1-0/00000015.jpg OK
testing: submission_evidence/scalpel/jpg-0-0/ OK
testing: submission_evidence/scalpel/jpg-0-0/00000000.jpg OK
testing: submission_evidence/scalpel/jpg-0-0/00000004.jpg OK
testing: submission_evidence/scalpel/jpg-0-0/00000006.jpg OK
testing: submission_evidence/scalpel/jpg-0-0/00000003.jpg OK
testing: submission_evidence/scalpel/jpg-0-0/00000005.jpg OK
testing: submission_evidence/scalpel/jpg-0-0/00000002.jpg OK
testing: submission_evidence/scalpel/jpg-0-0/00000007.jpg OK
testing: submission_evidence/scalpel/jpg-0-0/00000001.jpg OK
testing: submission_evidence/scalpel/audit.txt OK
testing: submission_evidence/scalpel/scalpel_jpeg.conf OK
testing: submission_evidence/tsk/ OK
testing: submission_evidence/tsk/ch01_fsstat.txt OK
testing: submission_evidence/tsk/usb_mmls.txt OK
testing: submission_evidence/tsk/ch01_fls_deleted.txt OK
testing: submission_evidence/tsk/ch01_mmls.txt OK
testing: submission_evidence/tsk/usb_fls_deleted.txt OK
testing: submission_evidence/tsk/sector_237.bin OK
```

Fig 8.5 Creating the required evidence ZIP page 2.

Explanation

The finished ZIP archive was tested using `unzip -t`. The displayed entries were reported as OK, showing that the archived supporting files could be read successfully without detected compression errors.



```
kali@kali: ~/forensic_lab
$ unzip -t forensic_lab_supporting_evidence.zip
testing: submission_evidence/notes/jpeg_header_xxd.txt OK
testing: submission_evidence/hashes/jpeg_header_xxd.txt OK
testing: submission_evidence/hashes/scalpel_config_md5.txt OK
testing: submission_evidence/hashes/scalpel_jpegs_sha256.txt OK
testing: submission_evidence/hashes/final_supporting_evidence_sha256.txt OK
testing: submission_evidence/hashes/scalpel_jpegs_md5.txt OK
testing: submission_evidence/hashes/usb_image_sha256.txt OK
testing: submission_evidence/hashes/original_md5.txt OK
testing: submission_evidence/hashes/evidence_sizes.txt OK
testing: submission_evidence/hashes/scalpel_config_sha256.txt OK
testing: submission_evidence/hashes/original_sha256.txt OK
testing: submission_evidence/hashes/usb_image_md5.txt OK
testing: submission_evidence/binwalk/ OK
testing: submission_evidence/binwalk/manual_ooxml_421888.zip OK
testing: submission_evidence/binwalk/j_ub_law_binwalk.txt OK
testing: submission_evidence/binwalk/Ch01InChap01_binwalk.txt OK
testing: submission_evidence/binwalk/usb_fat_carving_binwalk.txt OK
testing: submission_evidence/other_recovered/ OK
testing: submission_evidence/other_recovered/Billing_Letter_blkcat.doc OK
testing: submission_evidence/other_recovered/Billing_Letter_recovered.doc OK
testing: submission_evidence/other_recovered/j_ub_law_reconstructed.jpg OK
testing: submission_evidence/other_recovered/j_ub_law_plain.hex OK
No errors detected in compressed data of forensic_lab_supporting_evidence.zip.
(kali@kali) [~/forensic_lab]
$ md5sum forensic_lab_supporting_evidence.zip
sha256sum forensic_lab_supporting_evidence.zip
d1c15450c90e9122fc4403d4a1f85475 forensic_lab_supporting_evidence.zip
61305fc5982e30276ee5e8ac425b43aa3442e070eda81da0a00091a1df7db9d2 forensic_lab_supporting_evidence.zip
(kali@kali) [~/forensic_lab]
```

Fig 8.6 Creating the required evidence ZIP page 3.

Explanation

The ZIP verification completed without detected compressed-data errors, after which MD5 and SHA-256 hashes were calculated for the final archive. This provided the last integrity record for the supporting-evidence package before submission.

Conclusion

The laboratory successfully demonstrated how file recovery can continue even when normal file-system information is unavailable or unreliable. By combining hexadecimal examination, file-system analysis, embedded-file discovery, direct data-unit extraction, and signature-based carving, useful evidence was identified and recovered from the supplied forensic images. Integrity checks were incorporated throughout the process using MD5 and SHA-256 hashing.

The investigation also showed the value of working from the evidence actually available rather than relying on assumed offsets or placeholder values. The FAT16 partition offset was confirmed as sector 128 through mmls, while metadata addresses and sectors used with The Sleuth Kit came directly from the examined images. The absence of File_carving.docx was documented as a limitation, and the permitted alternative Binwalk workflow was completed using the USB forensic image instead.

Overall, the practical produced recoverable files, verified reconstructions, embedded-content extraction, 17 carved JPEG artefacts, documented hashes, an audit trail, and a validated supporting-evidence archive. This provided a complete and repeatable record of the forensic workflow while preserving the original evidence.

Screenshots

Fig 1.0 Creating the Lab folder.

Fig 1.1 Download the authorized training evidence files.

Fig 1.2 Download the authorized training evidence file page 1.

Fig 1.3 Download the authorized training evidence file page 2.

Fig 1.4 Checking that all three downloaded correctly.

Fig 1.5 calculating the original hashes for md5sum and sha256sum.

Fig 1.5.1 Recording the exact size for the files.

Fig 1.6 Making the originals read-only.

Fig 1.7 Making the evidence into original_evidence.

Fig 1.8 Recording the MD5 and SHA-256 hashes.

Fig 1.9 Recording the exact file sizes.

Fig 1.10 Making a working copy of the JPEG.

Fig 1.11 Examine the JPEG header with xxd.

Fig 1.12 Examine the JPEG footer.

Fig 1.13 Creating a plain hexadecimal dump.

Fig 1.14 Reconstructing the JPEG from the hexadecimal dump.

Fig 1.15 Cryptographically verify the reconstruction.

Fig 1.16 The metadata-like content examination.

Fig 1.17 The metadata-like content examination page 1.

Fig 1.18 The metadata-like content examination page 2.

Fig 1.19 The metadata-like content examination page 3.

Fig 1.20 The metadata-like content examination page 4.

Fig 1.21 The metadata-like content examination page 5.

Fig 1.22 The metadata-like content examination page 6.

Fig 1.23 The metadata-like content examination page 7.

Fig 1.24 The metadata-like content examination page 8.

Fig 1.25 The metadata-like content examination page 9.

Fig 1.26 The metadata-like content examination page 10.

Fig 1.27 The metadata-like content examination page 11.

Fig 1.28 The metadata-like content examination page 12.

Fig 1.29 The metadata-like content examination page 13.

Fig 2. Examining the Ch01InChap01.dd.

Fig 2.1 Examining the Ch01InChap01.dd page 2.

Fig 2.2 Examining the Ch01InChap01.dd page 3.

Fig 2.3 Examining the Ch01InChap01.dd page 4.

Fig 2.4 Examining the Ch01InChap01.dd page 5.

Fig 2.5 inspecting one deleted file with istat.

Fig 2.6 Extracting it with icat.

Fig 2.7 Completing the blkcat requirement.

Fig 2.8 Completing the blkcat requirement page 1.

Fig 2.9 Searching for the doc file.

Fig 3.0 Searching for the doc file page 1.

Fig 3.1 Searching for the doc file page 2.

Fig 3.1 Using binwalk for both authorize files.

Fig 3.2 Extracting into working and keeping the .7z untouched.

Fig 3.3 Seeing exactly what was extracted and identifying the image type.

Fig 3.4 Recording the exact size for usb_fat_carving.001.

Fig 3.5 Hashing the extracted forensic image with md5sum and sha256sum.

Fig 3.6 Determining the partition structure.

Fig 3.8 Inspecting the FAT16 filesystem at the confirmed offset page 1.

Fig 3.9 listing the files.

Fig 4.0 listing the files page 1.

Fig 4.1 Showing deleted entries.

Fig 4.2 Examining fat_carving.001 using binwalk.

Fig 4.3 Examining fat_carving.001 using binwalk page 1.

Fig 4.4 Examining fat_carving.001 using binwalk page 2.

Fig 4.5 Examining fat_carving.001 using binwalk page 3.

Fig 4.6 Automatic Binwalk extraction.

Fig 4.7 Automatic Binwalk extraction page 1.

Fig 4.8 Automatic Binwalk extraction page 2.

Fig 4.9 Automatic Binwalk extraction page 3.

Fig 5.0 Automatic Binwalk extraction page 4.

Fig 5.1 identify and extracting bytes.

Fig 5.2 Testing the ZIP structure.

Fig 5.3 Inspecting its contents.

Fig 5.4 Inspecting its contents page 1.

Fig 5.5 Hashing using md5sum and sha256sum.

Fig 5.6 Checking if Scalpel is installed.

Fig 5.7 Making a working copy of the Scalpel configuration.

Fig 5.8 Locating all JPEG-related configuration.

Fig 5.9 Evidence that the JPEG/JPG types were enabled in the Scalpel configuration.

Fig 6.0 Preserving the configured file's hash

Fig 6.1 Making sure the parent output directory exists.

Fig 6.2 Scaple carving.

Fig 6.3 Scaple carving for jpeg.

Fig 6.4 Locating the audit file.

Fig 6.5 counting the carved JPEGs.

Fig 6.6 Verify the recovered JPEGs.

Fig 6.7 Verify the recovered JPEGs page 1.

Fig 6.8 Hash all 17 carved JPEGs using md5sum and sha256sum.

Fig 6.9 Hash all 17 carved JPEGs using md5sum and sha256sum page 1.

Fig 7.0 Opening recovered JPEGs 00000009.

Fig 7.1 Opening recovered JPEGs 00000010.

Fig 7.2 Opening recovered JPEGs 00000011.

Fig 7.3 Evidence packaging.

Fig 7.4 Evidence packaging page 1.

Fig 7.5 Evidence packaging page 2.

Fig 7.6 Evidence packaging page 3.

Fig 7.7 Evidence packaging page 4.

Fig 7.8 Creating a clean submission-evidence folder.

Fig 7.9 Final package.

Fig 8.0 Final package page 1.

Fig 8.1 Final package page 2.

Fig 8.2 Hashing the submission_evidence.

Fig 8.3 Creating the required evidence ZIP.

Fig 8.4 Creating the required evidence ZIP page 1.

Fig 8.5 Creating the required evidence ZIP page 2.

Fig 8.6 Creating the required evidence ZIP page 3.

References

ICDFA Materials.